

The Hydrodynamic Stability of Viscous Plane Flow

Oscar Rummery

Monday 15th June, 2026

Declaration

This piece of work is a result of my own work except where it forms an assessment based on group project work. In the case of a group project, the work has been prepared in collaboration with other members of the group. Material from the work of others not involved in the project has been acknowledged and quotations and paraphrases suitably indicated.

Acknowledgements

I'd like to thank my project supervisor, Dr Laura Currie for her encouragement and guidance throughout this project.

Contents

1	Introduction	1
1.1	Hydrodynamic Stability and Parallel Shear Flows	1
1.2	Outline of this report	2
2	Governing Equations	4
2.1	Basic state and governing equations	4
2.2	Dimensionless Governing Equations	4
2.3	Linearising the governing equations	7
2.4	Normal mode analysis	8
2.5	Reducing our problem to 2 dimensions	9
2.6	The Orr-Sommerfeld equation	10
3	Sufficient Conditions for Stability	12
3.1	Equations for the wave speed and growth rate	12
3.2	Bounds on the wave speed	15
3.3	Bounds on the growth rate	18
4	Applying numerical methods to the Orr-Sommerfeld equation	25
4.1	Chebyshev polynomials	25
4.2	Approximating a function	26
4.3	Representing derivatives	29
4.4	Fourth Order System	32
4.5	Second Order System	35
4.6	First Order System	36
4.7	Boundary Conditions	37
4.8	Solving our system	38
4.9	Chebyshev Tau method	38
5	Results	40
5.1	Basic results - Poiseuille Flow	40
5.2	Basic results - Couette Flow	42
5.3	Time Comparison of Methods	44
5.4	Accuracy - Poiseuille Flow	45
5.5	Accuracy - Couette Flow	48
5.6	Eigenfunction - Poiseuille flow	49
5.7	Critical Reynolds Number - Poiseuille flow	51
6	Conclusion	53
6.1	Summary of results	53
6.2	Application of Results	54
6.3	Further Research	55

A	Code	56
A.1	.eps Saver	56
A.2	Chebyshev Python Class	59
A.3	Chebyshev Plotter	62
A.4	Chebyshev Function Approximator	63
A.5	Tau Matrices	66
A.6	Tau Method Class	69
A.7	Fourth Order Method	74
A.8	Second Order Method	76
A.9	First Order Method	78
A.10	Tau Method Tester	80
A.11	Tau Method Time Comparison	81
A.12	Eigenfunction Computation	83
A.13	Streamfunction Computation	85
A.14	Critical Reynolds Number Computation	87
	Bibliography	91

Chapter 1

Introduction

1.1 Hydrodynamic Stability and Parallel Shear Flows

What is hydrodynamic stability? Hydrodynamic stability is the area of fluid mechanics associated with the stability of flows or systems of flow. For virtually any system where a fluid is in motion, hydrodynamic stability is something that must be considered. This includes obvious examples such as rivers or pipes, but also less obvious examples such as blood flowing through vessels.

We see an example of hydrodynamic stability almost every day when we go to turn on a tap. In some cases, the stream of water appears completely clear. This is an example of *laminar flow* and is a visual example of a stable flow. In other cases, possibly even if we change how far we turn the handle / open the same tap (i.e. change the speed at which the water leaves the tap), the stream appears 'cloudy'. This is an example of unstable or *turbulent flow* and appears when the velocity profile of the stream isn't stable. Different taps will display this behaviour at different points when they are turned. This is due to their nozzles having different shapes and also having water flowing through them at different speeds. This nicely demonstrates the impact that changing the velocity and also the channel shape of a flow can have on the stability of a flow. Figure 1.1 shows an example of a tap producing a stable flow (left) versus an unstable flow (right).



Figure 1.1: Turbulent flow versus stable laminar flow from a tap ([Image source](#))

Another property of a fluid that affects its stability is *viscosity*. In the example of a tap, the most common fluid present is water which has a very low viscosity. However, if it was something

like oil or syrup leaving the tap the viscosity is much higher. Figure 1.2 shows a number of fluids of increasing viscosity being poured from identical test tubes. How does this affect stability? From what we see in real life, we'd expect viscosity to have a stabilising effect since a fluid like vegetable oil or honey seems a lot more stable when poured than water. The fluids in Figure 1.2 seem to show this too, with the least viscous fluid on the left looking more unstable than the more viscous fluids on its right.



Figure 1.2: Fluids of increasing viscosity being poured from test tubes. The least viscous fluid is on the left and the most viscous fluid is on the right. ([Image source](#))

A parallel shear flow is an axisymmetric flow with parallel streamlines. A plane flow is a 2-dimensional flow i.e. the flow is occurring only in one plane e.g. the (x, z) plane. This report aims to determine the stability of 2-dimensional viscous parallel shear flows, otherwise known as viscous plane flows.

The most common real-life example of this flow is that of a flow where a fluid is flowing through a pipe or other channel (such as a river bed). This is an important concern in the real world, due to the fact that energy equivalent to roughly 10% of the world's electricity is used moving fluids through pipes. A significant source of additional cost is turbulence since it involves the fluid swirling around itself and therefore not taking the most efficient path through the pipe. Figure 1.3 shows a comparison between turbulent flow (top) and stable flow (bottom) in a pipe.

In the inviscid case, there are many known analytic results on the stability of parallel shear flows such as *Rayleigh's inflexion point theorem* (Rayleigh, 1880) which states that a necessary condition for instability is that the velocity profile contains an inflexion point.

However, in the viscous case, the analytic results are far less concrete. Some do exist, such as the bounds found by Synge (1938); Pai (1954) and Joseph (1968, 1969), but as will be shown, they are extremely approximate, and do not tell us much about the stability of a specific flow. For this reason, numerical methods are often used instead. The aim of this report is to demonstrate both these analytic results on stability and a numerical method that can be used to more accurately determine the stability of the flow.

1.2 Outline of this report

Chapter 2 will introduce the basic state and governing equations for viscous plane flows, along with the family of flows that we are studying, plane Poiseuille and plane Couette flow. The chapter will then turn to manipulating these governing equations, reducing them to just a single equation, the Orr-Sommerfeld equation. The Orr-Sommerfeld equation is an extremely

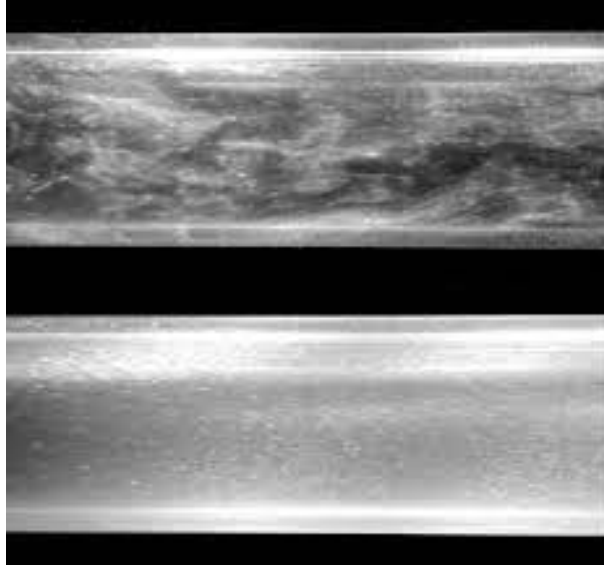


Figure 1.3: Turbulent pipe flow (top) versus stable pipe flow (bottom) - ([Image source](#))

significant equation when studying the linear stability of viscous parallel shear flows, and will be the equation that this report will use to provide both numerical and analytical results on stability.

Chapter 3 contains a discussion on how the Orr-Sommerfeld equation derived in Chapter 2 can be used to determine whether a flow is stable or not, before deriving separate equations for both the real and imaginary parts of the eigenvalue. We then turn to derive bounds on the real part of the eigenvalue, otherwise known as the wave speed. We derive bounds as seen in Pai (1954); Joseph (1968) which restrict the eigenvalues to a strip in the complex plane. We then derive bounds on the imaginary part of the eigenvalue, deriving the bounds found by Synge (1938); Joseph (1968, 1969). This gives us two bounds on the stability of a flow and allows us to get an analytic estimate of the critical Reynolds number of Couette and Poiseuille flow. However, these are extremely inaccurate, causing us to seek out a numerical method.

Chapter 4 turns to a numerical method to determine the stability of a flow. The numerical methods in question involve the use of Chebyshev polynomials and are referred to as the Chebyshev Tau methods. In the Chebyshev Tau method, we expand our function in terms of Chebyshev polynomials and represent the coefficients of this expansion in a vector. To represent the derivatives we use a matrix that transforms the coefficients of our expansion vector. This produces a matrix system that allows us to determine whether a flow is stable based on its eigenvalues. In the Tau method, the eigenvectors are the coefficients of the Chebyshev polynomial expansion representing our eigenfunctions. This chapter features 3 different ways of forming the generalised eigenvalue problem that we solve in Python, each using a larger system of lower-order differential equations. The reason we perform this split is to reduce the numerical error associated with the extremely large values present in the matrices we are using. This chapter will also explain which eigenvalue algorithm will be used to solve the generalised eigenvalue problems generated by each of our problems.

The numerical results of these methods are discussed in Chapter 5, followed by a comparison and discussion of the methods used and their advantages and disadvantages. The first set of results shown will be for the case of plane Poiseuille flow, but the chapter will then turn to discuss the results for the case of plane Couette flow. This chapter then turns to show several other key numerical results that we can get from our programs, such as the eigenfunctions (and associated streamfunctions) and the critical Reynolds number.

Chapter 2

Governing Equations

This chapter focuses on the equations governing viscous plane flows and moves on to manipulating these equations to derive the famous *Orr-Sommerfeld equation*, a famous eigenvalue equation for linear stability that (as we will see later in the report) can be solved to determine the stability of a flow. This chapter also introduces the two types of viscous plane flow this report deals with, plane Poiseuille flow and plane Couette flow.

2.1 Basic state and governing equations

A plane flow is a 2-dimensional flow represented by a velocity profile U in a single direction. For this project, we will consider U that are functions of z only and act in the x direction. Moreover, our flow will be considered to be flowing through a 'channel' that is bounded between $z = z_1$ and $z = z_2$.

We denote our basic steady flow as

$$\mathbf{U} = U(z)\mathbf{i} \quad (z_1 \leq z \leq z_2),$$

with boundaries on $z = z_1$ and $z = z_2$ that are either *rigid* or *free*:

- *rigid* - normal component of velocity must vanish
- *free* - pressure must be constant

The governing equations for this system are the same as any viscous flow, the continuity equation and the Navier-Stokes equation for an incompressible fluid.

$$\begin{aligned} \nabla \cdot \mathbf{u} &= 0, \\ \frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} &= -\frac{1}{\rho} \nabla p + \nu \Delta \mathbf{u}. \end{aligned} \tag{1}$$

Note that we denote the *kinematic viscosity* $\nu = \frac{\mu}{\rho}$ and have also set $f = 0$ in the Navier-Stokes equation, assuming there is no force acting on the fluid. The fluid discussed in this report is also considered to have a constant density i.e. $\rho = \text{const.}$

Figure 2.1 shows a schematic of the system we will be analysing in an initial stable state.

2.2 Dimensionless Governing Equations

We would like to get a dimensionless form of our governing equations (1). This is useful because it allows us to map our flow to a corresponding flow of width $z = -1$ to $z = 1$. As we will see

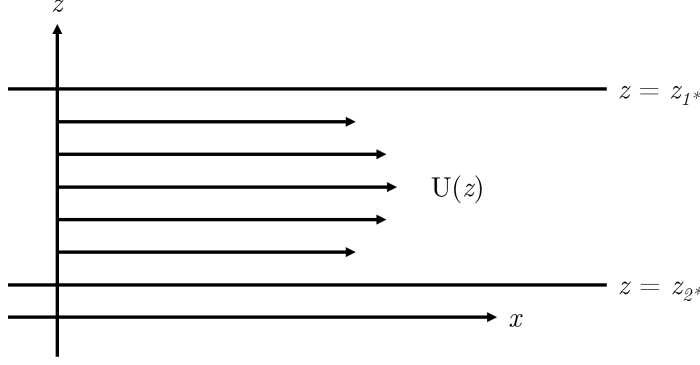


Figure 2.1: The setup of our flow

later in this chapter, this allows us to analyse families of flows that have an equivalent Reynolds number. In Chapter 4 we will also see that this range is the same as the range of the Chebyshev polynomials that are fundamental to our numerical solution.

To write the governing equations in terms of dimensionless quantities we utilise the following change of variables as in Drazin and Reid (2004):

$$t_* = \frac{tV}{L}, \quad \mathbf{x}_* = \frac{\mathbf{x}}{L}, \quad \mathbf{u}_* = \frac{\mathbf{u}}{V}, \quad p_* = \frac{p}{\rho V^2},$$

$$\mathbf{U}_* = \frac{\mathbf{U}}{V} \equiv U_*(z_*)\mathbf{i}, \quad (2)$$

where we refer to V as the *characteristic velocity* and L as the *characteristic length*, defined as

$$V = \max_{z \in [z_1, z_2]} |U(z)|,$$

$$L = \frac{1}{2}(z_2 - z_1).$$

Note: Drazin and Reid use asterisks to represent the dimensional variables, but here we use the asterisks to indicate the dimensionless variables as is the more common convention.

So in this case, the characteristic velocity of a flow is the maximum velocity of its initial velocity profile and the characteristic length is half the width of the channel. Note that the basic flow still acts in just the x direction, meaning that the dimensionless flow still has the same form as in Figure 2.1.

We can now use this transformation to find dimensionless forms of the various terms of the governing equations. By the chain rule, we have

$$\frac{\partial \mathbf{u}}{\partial t} = \frac{\partial \mathbf{u}}{\partial \mathbf{u}_*} \frac{\partial \mathbf{u}_*}{\partial t_*} \frac{\partial t_*}{\partial t} = \frac{V^2}{L} \frac{\partial \mathbf{u}_*}{\partial t_*}. \quad (3)$$

In our dimensional coordinates, ∇ is defined as

$$\nabla = \frac{\partial}{\partial x} \mathbf{i} + \frac{\partial}{\partial y} \mathbf{j} + \frac{\partial}{\partial z} \mathbf{k}.$$

To transform this operator into our new coordinates we use the chain rule to get the following

$$\nabla = \frac{\partial}{\partial x_*} \frac{\partial x_*}{\partial x} \mathbf{i} + \frac{\partial}{\partial y_*} \frac{\partial y_*}{\partial y} \mathbf{j} + \frac{\partial}{\partial z_*} \frac{\partial z_*}{\partial z} \mathbf{k} = \frac{1}{L} \nabla_*.$$

We can use this ∇_* along with our transformation (2) to get equations for $\nabla \mathbf{u}$, $\nabla \cdot \mathbf{u}$, $\Delta \mathbf{u}$ and ∇p in their dimensionless forms as follows,

$$\nabla \mathbf{u} = \frac{1}{L} \nabla_* (V \mathbf{u}_*) = \frac{V}{L} \nabla_* \mathbf{u}_*, \quad (4)$$

$$\nabla \cdot \mathbf{u} = \left[\frac{1}{L} \nabla_* \right] \cdot (V \mathbf{u}_*) = \frac{V}{L} \nabla_* \cdot \mathbf{u}_*, \quad (5)$$

$$\Delta \mathbf{u} = \nabla \cdot (\nabla \mathbf{u}) = \left[\frac{1}{L} \nabla_* \right] \cdot \left[\frac{V}{L} \nabla_* \mathbf{u}_* \right] = \frac{V}{L^2} \Delta_* \mathbf{u}_*, \quad (6)$$

$$\nabla p = \left[\frac{1}{L} \nabla_* \right] (\rho V^2 p_*) = \frac{\rho V^2}{L} \nabla_* p_*. \quad (7)$$

We can now substitute these results into our governing equations (1) to get their dimensionless forms. For the *continuity equation* we use equation (5) to get

$$0 = \nabla \cdot \mathbf{u} = \frac{V}{L} \nabla_* \cdot \mathbf{u}_*, \\ \nabla_* \cdot \mathbf{u}_* = 0.$$

Substituting our remaining equations (3), (4), (6), (7) into the *Navier-Stokes equation* we get

$$\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \Delta \mathbf{u}, \\ \frac{V^2}{L} \frac{\partial \mathbf{u}_*}{\partial t_*} + \frac{V^2}{L} \mathbf{u}_* \cdot \nabla_* \mathbf{u}_* = -\frac{V^2}{L} \nabla_* p_* + \frac{\nu V}{L^2} \Delta_* \mathbf{u}_*, \\ \frac{\partial \mathbf{u}_*}{\partial t_*} + \mathbf{u}_* \cdot \nabla_* \mathbf{u}_* = -\nabla_* p_* + \frac{\nu}{VL} \Delta_* \mathbf{u}_*.$$

Notice that $\frac{VL}{\nu}$ is the famous *Reynolds number* R . Using this in the Navier-Stokes equation leaves us with our pair of *dimensionless governing equations*. For simplicity, we drop the asterisks from our notation from now on as we will not deal with the dimensional case again. This means our dimensionless governing equations are now

$$\nabla \cdot \mathbf{u} = 0, \\ \frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} = -\nabla p + R^{-1} \Delta \mathbf{u}. \quad (8)$$

If we substitute our basic flow $\mathbf{u}(\mathbf{x}, t) = U(z)\mathbf{i}$ and basic pressure, $p(\mathbf{x}, t) = P(\mathbf{x})$ into (8), we find that $U(z)$ must satisfy the following equation of motion

$$\frac{d^2 U}{dz^2} = R \frac{dP}{dx},$$

which limits our choices of $U(z)$ and $P(\mathbf{x})$. There are two main important cases defined as follows:

- *Poiseuille flow*: $U(z) = 1 - z^2$ with $\frac{dP}{dx} = \text{const.}$
- *Couette flow*: $U(z) = z$ with $P = \text{const.}$

Schematic diagrams for these 2 flows can be seen in Figure 2.2. Poiseuille flow can be thought of as representing a velocity being applied in the centre of the channel, or the fluid 'sticking' at the edges. This is common with more viscous fluids. Couette flow on the other hand can be thought of as a representation of the flow 'shearing.'

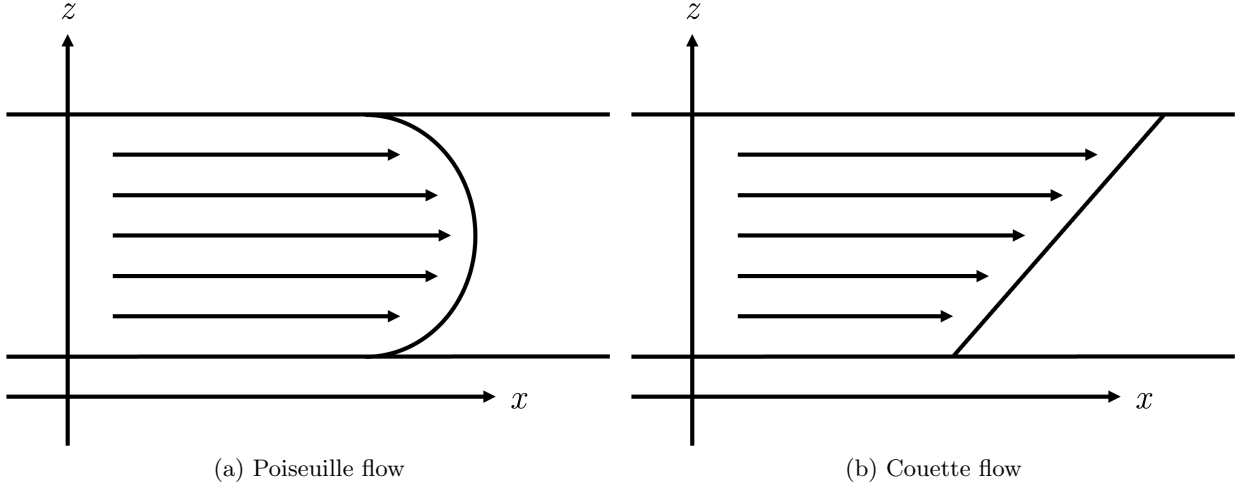


Figure 2.2: Schematics of plane Poiseuille and plane Couette flow

Before we move on to linearising our governing equations, a discussion of the Reynolds number is in order. As we can see it provides a ratio between the three main variables that define our flow: the velocity, width and viscosity. It scales proportionally to width and velocity, but inversely proportional to viscosity. This means a flow with a high Reynolds number could have one of three things: it could be travelling at a high velocity, be flowing through a very wide channel, or have a low viscosity. This means an example of a flow with a high Reynolds number could be: a flow of water or alcohol which has a low viscosity, a flow with a large channel such as a wide section of river, or a fluid being pumped quickly such as fuel being transported through a pipe. A flow with a low Reynolds number might have a high viscosity fluid, be travelling slowly or be travelling through a thinner channel. This means a flow of a fluid like honey could have a low Reynolds number.

If we can find conditions on the Reynolds number that relate to stability then this may tell us what these parameters should be set to proportional to each other. For example, if flows with a high Reynolds number are more stable, then for a more viscous flow we may want to increase the width of the channel it is being pumped through to increase its stability. If flows of a low Reynolds number are more stable, then we may want to decrease the width of the channel for flow with lower viscosity.

2.3 Linearising the governing equations

To study the stability of a plane flow we consider small perturbations to $\mathbf{u}$ and p of the form

$$\begin{aligned}\mathbf{u}(\mathbf{x}, t) &= U(z)\mathbf{i} + \mathbf{u}'(\mathbf{x}, t), \\ p(\mathbf{x}, t) &= P(\mathbf{x}) + p'(\mathbf{x}, t).\end{aligned}$$

We can now substitute these equations back into our dimensionless governing equations (8). From here we linearise, discarding any terms that are non-linear in $\mathbf{u}'$ or p' . Note that this means that for the remainder of this report, we are analysing the linear stability of a flow as all non-linear components have been removed. This has some impact on the validity of our report, as we will see when we come to discuss our results later on.

For the continuity equation, we get

$$0 = \nabla \cdot \mathbf{u} = \nabla \cdot (U\mathbf{i} + \mathbf{u}'(\mathbf{x}, t)) = \nabla \cdot \mathbf{u}', \quad (9)$$

where the term $\nabla \cdot U\mathbf{i}$ vanishes as U is a function of z only. For the Navier-Stokes equation, we linearise the individual terms separately for readability.

For $\frac{\partial \mathbf{u}}{\partial t}$,

$$\frac{\partial \mathbf{u}}{\partial t} = \frac{\partial}{\partial t}(U(z)\mathbf{i} + \mathbf{u}'(\mathbf{x}, t)) = \frac{\partial \mathbf{u}'}{\partial t}.$$

Similarly

$$\begin{aligned} \mathbf{u} \cdot \nabla \mathbf{u} &= (U\mathbf{i} + \mathbf{u}'(\mathbf{x}, t)) \cdot \nabla (U\mathbf{i} + \mathbf{u}'(\mathbf{x}, t)) \\ &= (U\mathbf{i} + \mathbf{u}'(\mathbf{x}, t)) \cdot (\nabla \mathbf{u}' + \nabla U) \\ &= U\mathbf{i} \cdot \nabla \mathbf{u}' + \mathbf{u}' \cdot \nabla \mathbf{u}' + \mathbf{u}' \nabla U + U\mathbf{i} \cdot \nabla U\mathbf{i}. \end{aligned}$$

We now omit $\mathbf{u}' \cdot \nabla \mathbf{u}'$ as it is quadratic in $\mathbf{u}'$. We are left with

$$\mathbf{u} \cdot \nabla \mathbf{u} = U \frac{\partial \mathbf{u}'}{\partial x} + w' \frac{dU}{dz} \mathbf{i},$$

where w' represents the z -component of $\mathbf{u}'$. For the right-hand side, we get

$$\begin{aligned} \nabla p &= \nabla (P(\mathbf{x}) + p') = -\nabla p', \\ \Delta \mathbf{u} &= \Delta (U\mathbf{i} + \mathbf{u}'(\mathbf{x}, t)), \\ &= \nabla \cdot \nabla (U\mathbf{i} + \mathbf{u}'(\mathbf{x}, t)), \\ &= \nabla \cdot \nabla \mathbf{u}' = \Delta \mathbf{u}'. \end{aligned}$$

We can now substitute these into the Navier-Stokes equation to get its linearised form:

$$\left[\frac{\partial}{\partial t} + U \frac{\partial}{\partial x} \right] \mathbf{u}' + w' \frac{dU}{dz} \mathbf{i} = -\nabla p' + R^{-1} \Delta \mathbf{u}'. \quad (10)$$

2.4 Normal mode analysis

Now that we have our linearised equations, we consider a normal mode solution. As in Drazin and Reid (2004), we consider the following ansatz for our perturbations $\mathbf{u}'$ and p' .

$$\begin{aligned} \mathbf{u}'(\mathbf{x}, t) &= \hat{\mathbf{u}}(z) \exp\{i(\alpha x + \beta y - \alpha ct)\}, \\ p'(\mathbf{x}, t) &= \hat{p}(z) \exp\{i(\alpha x + \beta y - \alpha ct)\}. \end{aligned} \quad (11)$$

This ansatz is derived from the fact that the coefficients in our linearised equations (9) and (10) are only dependent on z . In this ansatz α, β are the *wavenumbers* of the mode and can take any real value greater than 0, and c denotes the *phase speed* and is a complex number. The value of the phase speed is very important when it comes to analysing the stability of a flow. More details of this will be discussed in Chapter 3.

The functions $\hat{\mathbf{u}}$ and $\hat{p}$ represent how our perturbation varies through the top and bottom of the channel. We will need a way of representing the individual components of $\hat{\mathbf{u}}$ so denote $\hat{\mathbf{u}} = (\hat{u}, \hat{v}, \hat{w})$.

We now substitute our ansatz (11) into our linearised governing equations (9) and (10) which gives us

$$\begin{aligned} 0 &= \nabla \cdot \mathbf{u}' = \nabla \cdot \hat{\mathbf{u}}(z) \exp\{i(\alpha x + \beta y - \alpha ct)\}, \\ &= \left[i(\alpha \hat{u} + \beta \hat{v}) + \frac{d\hat{w}}{dz} \right] \exp\{i(\alpha x + \beta y - \alpha ct)\}. \end{aligned}$$

Looking at this equation, we see that for it to equal zero the following condition must be satisfied

$$i(\alpha\hat{u} + \beta\hat{v}) + \frac{d\hat{w}}{dz} = 0. \quad (12)$$

For the left-hand side of the Navier-Stokes equation, we get

$$\begin{aligned} \left[\frac{\partial}{\partial t} + U \frac{\partial}{\partial x} \right] \mathbf{u}' + w' \frac{dU}{dz} \mathbf{i} &= -i\alpha c \mathbf{u}' + i\alpha U \mathbf{u}' + \frac{dU}{dz} \hat{w} \mathbf{i} \\ &= \left[i\alpha(U - c) \hat{\mathbf{u}} + \frac{dU}{dz} \hat{w} \mathbf{i} \right] \exp\{i(\alpha x + \beta y - \alpha ct)\}, \end{aligned}$$

and for the right-hand side, we get

$$\begin{aligned} -\nabla p' + R^{-1} \Delta \mathbf{u}' &= \\ \left[-(i\alpha \hat{p} \mathbf{i} + i\beta \hat{p} \mathbf{j} + \frac{d\hat{p}}{dz} \mathbf{k}) + R^{-1} \hat{\mathbf{u}}(-\alpha^2 - \beta^2 - \frac{d^2 \hat{\mathbf{u}}}{dz^2}) \right] \exp\{i(\alpha x + \beta y - \alpha ct)\}. \end{aligned}$$

Equating both sides and cancelling the exponential leaves us with the following 3 equations

$$\begin{aligned} i\alpha(U - c)\hat{u} + \frac{dU}{dz}\hat{w} &= -i\alpha\hat{p} - \left[(\alpha^2 + \beta^2)\hat{u} + \frac{d^2 \hat{u}}{dz^2} \right] R^{-1}, \\ i\alpha(U - c)\hat{v} &= -i\beta\hat{p} - \left[(\alpha^2 + \beta^2)\hat{v} + \frac{d^2 \hat{v}}{dz^2} \right] R^{-1}, \\ i\alpha(U - c)\hat{w} &= \frac{d\hat{p}}{dz} - \left[(\alpha^2 + \beta^2)\hat{w} + \frac{d^2 \hat{w}}{dz^2} \right] R^{-1}. \end{aligned}$$

Now let $D = \frac{d}{dz}$ for simplicity and rearrange to get

$$(D^2 - (\alpha^2 + \beta^2) - i\alpha R(U - c))\hat{u} = R(DU)\hat{w} + i\alpha R\hat{p}, \quad (14a)$$

$$(D^2 - (\alpha^2 + \beta^2) - i\alpha R(U - c))\hat{v} = i\beta R\hat{p}, \quad (14b)$$

$$(D^2 - (\alpha^2 + \beta^2) - i\alpha R(U - c))\hat{w} = RD\hat{p}. \quad (14c)$$

What are the boundary conditions for these equations? Since we are dealing with a channel or pipe, we use the more common rigid boundary conditions. For a viscous flow, this means the boundary conditions (Drazin and Reid, 2004)

$$\hat{u} = \hat{v} = \hat{w} = 0, \text{ on } z = -1, z = 1. \quad (15)$$

Combining $\alpha(14a) + \beta(14b)$ we get

$$(D^2 - (\alpha^2 + \beta^2) - i\alpha R(U - c))(\alpha\hat{u} + \beta\hat{v}) = \alpha R(DU)\hat{w} + i\alpha^2 R\hat{p} + i\beta^2 R\hat{p}. \quad (16)$$

2.5 Reducing our problem to 2 dimensions

It turns out we can reduce our current 3-dimensional problem to an equivalent 2-dimensional problem (Drazin and Reid, 2004). Let

$$\begin{aligned} \tilde{\alpha} &= (\alpha^2 + \beta^2)^{1/2}, \quad \tilde{\alpha}\tilde{u} = \alpha\hat{u} + \beta\hat{v}, \quad \frac{\tilde{p}}{\tilde{\alpha}} = \frac{\hat{p}}{\alpha}, \\ \tilde{w} &= \hat{w}, \quad \tilde{c} = c, \quad \tilde{\alpha}\tilde{R} = \alpha R. \end{aligned}$$

This is called *Squire's Transformation* (Squire, 1933). If we use this transformation on equations (12), (14c) and (16) then we are left with equations

$$(D^2 - \tilde{\alpha}^2 - i\tilde{\alpha}\tilde{R}(U - \tilde{c}))\tilde{u} = \tilde{R}(DU)\tilde{w} + i\tilde{\alpha}\tilde{R}\tilde{p}, \quad (17a)$$

$$(D^2 - \tilde{\alpha}^2 - i\tilde{\alpha}\tilde{R}(U - \tilde{c}))\tilde{w} = \tilde{R}D\tilde{p}, \quad (17b)$$

$$i\tilde{\alpha}\tilde{u} + D\tilde{w} = 0. \quad (17c)$$

These equations have the same form as equations (12), (14) with $\beta = \hat{v} = 0$, showing that we have successfully reduced this to a 2-dimensional problem. In what follows we will use equations (12), (14) with $\beta = \hat{v} = 0$ to keep the notation simpler.

2.6 The Orr-Sommerfeld equation

To reduce our three equations (17) to just a single equation, it is convenient to use a stream function solution that satisfies

$$u' = \frac{\partial \psi'}{\partial z}, \quad w' = -\frac{\partial \psi'}{\partial x}, \quad (\mathbf{u}' = \nabla \times \psi' \mathbf{k}). \quad (18)$$

Now let $\psi'(x, z, t) = \phi(z) \exp\{i\alpha(x - ct)\}$ which leaves us with

$$\hat{u} = D\phi, \quad \hat{w} = -i\alpha\phi,$$

as the exponentials cancel out (because $\beta = 0$). If we put this solution into equation (17c) we get the following

$$i\alpha D\phi - i\alpha D\phi = 0,$$

so this equation is satisfied. For equations (17a) and (17b) we are left with

$$\frac{(D^2 - \alpha^2 - i\alpha R(U - c))D\phi}{i\alpha R} + (DU)\phi = \hat{p}, \quad (19)$$

$$-i\alpha(D^2 - \alpha^2 - i\alpha R(U - c))\phi = RD\hat{p}. \quad (20)$$

If we differentiate (19) we get

$$\frac{D((D^2 - \alpha^2 - i\alpha R(U - c))D\phi)}{i\alpha R} + (D^2 U)\phi + (DU)(D\phi) = D\hat{p}.$$

Equate (19) and (20) to get

$$\begin{aligned} & -i\alpha(D^2 - \alpha^2 - i\alpha R(U - c))\phi \\ &= \frac{D((D^2 - \alpha^2 - i\alpha R(U - c))D\phi)}{i\alpha} + [(D^2 U)\phi + (DU)(D\phi)]R. \end{aligned}$$

Multiplying by $i\alpha$ and expanding leaves us with

$$\begin{aligned} & \alpha^2 D^2 \phi - \alpha^4 \phi - i\alpha^3 R(U - c)\phi \\ &= D^4 \phi - \alpha^2 D^2 \phi + i\alpha R(-(DU)(D\phi) + U(D^2 \phi)) + c(D^2 \phi) + ((D^2 U)\phi + (DU)(D\phi)) \\ &\implies (D^4 - 2\alpha^2 D^2 + \alpha^4)\phi = i\alpha R[-\alpha^2(U - c)\phi + (U - c)D^2 \phi - (D^2 U)\phi]. \end{aligned}$$

Noting that $(D^2 - \alpha^2)^2 = D^4 - 2\alpha^2 D^2 + \alpha^4$ we have that ϕ satisfies the following fourth order differential equation

$$(i\alpha R)^{-1}(D^2 - \alpha^2)^2 \phi = (U - c)(D^2 - \alpha^2)\phi - U''\phi. \quad (21)$$

If we look at our boundary conditions (15) and at our stream function (18) then we can see that for $\hat{u} = \hat{w} = 0$ to be zero on $z = \pm 1$ we must have the boundary conditions

$$\phi(\pm 1) = \phi'(\pm 1) = 0. \quad (22)$$

Equation (21) is known as the *Orr-Sommerfeld equation* and was first derived by Orr (1907) and Sommerfeld (1909). It is an eigenvalue equation with complex eigenvalue c . Being a fourth-order equation, it is difficult to solve. However, as we will see in the next chapter, this equation allows us to determine some analytic conditions for the stability of our governing equations. The rest of this report will be focused on determining whether a solution to this equation is stable for different R and α with different flows.

Chapter 3

Sufficient Conditions for Stability

We now turn to the discussion and proof of various analytic results in the pursuit of some conditions on the stability of (21). Denote our eigenvalue c by

$$c = c_r + ic_i.$$

The real part c_r is commonly referred to as the *wave speed* and the imaginary part c_i is referred to as the *growth rate* (Drazin and Reid, 2004). The more interesting part in relation to instability is the growth rate c_i as if $c_i > 0$ then we have an unstable solution to (21). However, it is also possible to find bounds on c_r , though these do not provide any indication as to whether a type of flow is stable.

One of the first instances of this was Synge (1938), generalised by Pai (1954). Improvements were later made by Joseph (1968, 1969).

3.1 Equations for the wave speed and growth rate

To find separate bounds on the wave speed and growth rate, we need to somehow split the Orr-Sommerfeld equation in a way that gets us two separate equations, one for the wave speed and one for the growth rate. If we multiply the Orr-Sommerfeld equation (21) by ϕ_c (the complex conjugate of ϕ) and integrating with respect to $z \in (-1, 1)$ we obtain the following equation:

$$\int_{-1}^1 (i\alpha R)^{-1} (D^2 - \alpha^2)^2 \phi \phi_c dz = \int_{-1}^1 (U - c) (D^2 - \alpha^2) \phi \phi_c dz - \int_{-1}^1 U'' \phi \phi_c dz. \quad (23)$$

Throughout this chapter, we shall use the same notation as Drazin and Reid (2004) to represent some commonly occurring integrals

$$\begin{aligned} I_0^2 &= \int_{-1}^1 |\phi|^2 dz, \\ I_1^2 &= \int_{-1}^1 |D\phi|^2 dz, \\ I_2^2 &= \int_{-1}^1 |D^2\phi|^2 dz. \end{aligned}$$

As we will see later, there are inequalities on these integrals from Joseph (1969) that help with finding bounds on c_i and c_r . This means we would like to express our integral (23) in

terms of these. We can already see some instances of these integrals appearing in our integral, for example, if we multiply out the brackets on the left-hand side we end up with the term $(i\alpha R)^{-1}\alpha^4 I_0^2$. However, the terms containing $(D^4\phi)\phi_c$, $(D^2\phi)\phi_c$ and $U(D^2\phi)\phi_c$ require some manipulation to get in terms of I_0^2 , I_1^2 , I_2^2 . Our boundary conditions (22) can help with this! If we integrate by parts, they allow us to cancel out the evaluated integral term. This gives us the following results:

$$\begin{aligned}\int_{-1}^1 (D^4\phi)\phi_c \, dz &= \left[(D^3\phi)\phi_c \right]_{-1}^1 - \int_{-1}^1 (D^3\phi)(D\phi_c) \, dz \\ &= -\left[(D^2\phi)(D\phi_c) \right]_{-1}^1 + \int_{-1}^1 (D^2\phi)(D^2\phi_c) \, dz = I_2^2, \\ \int_{-1}^1 (D^2\phi)\phi_c \, dz &= \left[(D\phi)\phi_c \right]_{-1}^1 - \int_{-1}^1 (D\phi)(D\phi_c) \, dz = -I_1^2, \\ \int_{-1}^1 U(D^2\phi)\phi_c \, dz &= \left[(D\phi)U\phi_c \right]_{-1}^1 - \int_{-1}^1 (D\phi) [U'\phi_c + U(D\phi_c)] \, dz \\ &= -\int_{-1}^1 U' [(D\phi)\phi_c + U(D\phi)(D\phi_c)] \, dz,\end{aligned}$$

We have now represented most of the terms in our integral (23) in terms of I_0^2 , I_1^2 , I_2^2 , leaving us with

$$(I_2^2 + 2\alpha I_1^2 + \alpha^4 I_0^2) + i\alpha R Q = i\alpha c R (I_1^2 + \alpha I_0^2), \quad (25)$$

where

$$Q = \int_{-1}^1 U |D\phi|^2 + (\alpha^2 U + U'') |\phi|^2 \, dz + \int_{-1}^1 U' (D\phi)\phi_c \, dz.$$

To get separate equations for c_i and c_r we need to split (25) into real and imaginary parts. All of the integrals I_0^2 , I_1^2 , I_2^2 are real, meaning that the only things we need to calculate are the real and imaginary parts of Q . Taking the real part of Q :

$$\begin{aligned}\text{Re}\{Q\} &= \frac{1}{2}(Q - Q_c) = \frac{1}{2} \int_{-1}^1 U |D\phi|^2 + \alpha^2 U + U'' |\phi|^2 \, dz \\ &\quad + \frac{1}{2} \int_{-1}^1 U' [(D\phi)\phi_c + \phi(D\phi_c)] \, dz.\end{aligned}$$

We can simplify this a bit if we note that $D(\phi\phi_c) = (D\phi)\phi_c + \phi(D\phi_c)$ and integrate by parts, leading to

$$\begin{aligned}\text{Re}\{Q\} &= \int_{-1}^1 U |D\phi|^2 + \alpha^2 U + U'' |\phi|^2 \, dz + \frac{1}{2} \left[\left[U'\phi\phi_c \right]_{-1}^1 - \int_{-1}^1 U'' \phi\phi_c \, dz \right] \\ &= \int_{-1}^1 U |D\phi|^2 + \alpha^2 U + U'' |\phi|^2 \, dz - \frac{1}{2} \int_{-1}^1 U'' \phi\phi_c \, dz \\ &= \int_{-1}^1 U |D\phi|^2 + \alpha^2 U + \frac{1}{2} U'' |\phi|^2 \, dz.\end{aligned}$$

For the imaginary part of Q , we have

$$\begin{aligned}\text{Im}\{Q\} &= \frac{1}{2i}(Q - Q_c) = \frac{1}{2i} \int_{-1}^1 U' [(D\phi)\phi_c - \phi(D\phi_c)] \, dz \\ &= \frac{1}{2}i \int_{-1}^1 U' [(D\phi_c)\phi - \phi_c(D\phi)] \, dz.\end{aligned} \quad (26)$$

Now we can split our integral into real and imaginary parts as follows

$$\begin{aligned}\alpha R \operatorname{Re}\{Q\} &= -\alpha R c_r (I_1^2 + \alpha^2 I_0^2), \\ -\alpha R \operatorname{Im}\{Q\} + (I_2^2 + 2\alpha^2 I_1^2 + \alpha^4 I_0^2) &= -\alpha R c_i (I_1^2 + \alpha I_0^2).\end{aligned}$$

Rearranging these gives us our equations for c_i and c_r

$$c_r = \frac{\operatorname{Re}\{Q\}}{I_1^2 + \alpha^2 I_0^2}, \quad (27a)$$

$$c_i = \frac{(\operatorname{Im}\{Q\} - (\alpha R)^{-1}(I_2^2 + 2\alpha^2 I_1^2 + \alpha^4 I_0^2))}{I_1^2 + \alpha^2 I_0^2}. \quad (27b)$$

These equations have the same form as derived by Pai (1954), though Pai used a generalised set of boundary conditions, whereas we have used our specific boundary conditions (22).

We now have equations for both the wave speed c_r and the growth rate c_i . This chapter will now turn to using these to get bounds on both.

First though, we should introduce those bounds on I_0^2 , I_1^2 , I_2^2 that were mentioned earlier. It can be shown that the following lemma holds:

Lemma 1 (Joseph (1968, 1969)). *Let $\phi \in \bar{H}(\alpha)$. Then*

$$I_1^2 \geq \lambda_1^2(\alpha) I_0^2, \quad I_2^2 \geq \lambda_2^2(\alpha) I_1^2, \quad I_2^2 \geq \lambda_3^2(\alpha) I_0^2, \quad (28a)$$

$$\lambda_1^2(\alpha) \geq \frac{1}{4}\pi^2, \quad \lambda_2^2(\alpha) \geq \pi^2, \quad \lambda_3^2 \geq \left(\frac{4.73}{2}\right)^4, \quad (28b)$$

with equality in (30) when $\alpha = 0$.

Proof. An outline of the proof of this lemma is given here. The reader is encouraged to read Joseph (1969) for more details.

λ_1 , λ_2 , λ_3 are derived from the following minimum problems,

$$\begin{aligned}\lambda_1^2 &= \min \frac{\int_0^1 (\phi')^2 dz + \alpha \phi(1)^2}{\int_0^1 \phi^2 dz} = \min \frac{\tilde{I}_1^2}{I_0^2}, \\ \lambda_2^2 &= \min \frac{\int_0^1 (\phi'')^2 dz}{\int_0^1 (\phi')^2 dz + \alpha \phi(1)^2} = \min \frac{\tilde{I}_2^2}{I_1^2}, \\ \lambda_3^2 &= \min \frac{\int_0^1 (\phi'')^2 dz}{\int_0^1 \phi^2 dz} = \min \frac{\tilde{I}_2^2}{I_0^2}.\end{aligned} \quad (29)$$

The tilde over the I_i^2 in these minimum value problems are to indicate that these are the I_i^2 defined by Joseph.

To minimise these, Joseph considers the Euler-Lagrange equations and finds from here that λ_1 comes from the smallest positive root of the equation

$$\lambda \cos \lambda + \alpha \sin \lambda = 0.$$

λ_2, λ_3 are the smallest positive roots of the following determinants

$$\begin{vmatrix} \lambda_2(\cos \lambda_2 - 1) + \alpha \sin \lambda_2 - \alpha \lambda_2 & -\lambda_2 \sin \lambda_2 + \alpha(\cos \lambda_2 - 1) \\ -\lambda_2 \cos \lambda_2 - \alpha \sin \lambda_2 & \lambda_2 \sin \lambda_2 - \alpha \cos \lambda_2 \end{vmatrix} = 0,$$

$$\begin{vmatrix} \lambda_3(\cosh \lambda_3 - \cos \lambda_3) + \alpha(\sinh \lambda_3 - \sin \lambda_3) & \lambda_3(\sinh \lambda_3 + \sin \lambda_3) + \alpha(\cosh \lambda_3 - \cos \lambda_3) \\ \lambda_3(\cosh \lambda_3 + \cos \lambda_3) + \alpha(\sinh \lambda_3 + \sin \lambda_3) & \lambda_3(\sinh \lambda_3 - \sin \lambda_3) + \alpha(\cosh \lambda_3 + \cos \lambda_3) \end{vmatrix} = 0.$$

All of these equations come from the boundary conditions that must be satisfied by the eigenfunctions of the Euler-Lagrange equations.

However, as mentioned before, Joseph defines his I_i^2 differently from us. His definitions have boundary conditions on $z = 0, z = 1$ unlike our boundary conditions on $z = -1, z = 1$, and his definition for I_1^2 includes the term $\alpha\phi(1)^2$. So why can we use this lemma in our case?

Consider the transformation $y = (1 + z)/2$ which maps z to a range of y between 0 and 1, the same as Joseph and set $\alpha = 0$ in $\tilde{I}_1^2$. If we apply this to our definition of I_0^2, I_1^2, I_2^2 we get the following

$$\begin{aligned} I_0^2 &= 2\tilde{I}_0^2, \\ I_1^2 &= 2\tilde{I}_1^2, \\ I_2^2 &= 2\tilde{I}_2^2. \end{aligned}$$

When we substitute these into the minimum problems considered by Joseph (29) the factors cancel out, showing that our definitions for I_i^2 satisfy Lemma 1 with $\alpha = 0$. This means that the second set of inequalities (28b) become equalities and the first set of inequalities (28a) become

$$I_1^2 \geq \frac{1}{4}\pi^2 I_0^2, \quad I_2^2 \geq \pi^2 I_1^2, \quad I_2^2 \geq \left(\frac{4.73}{2}\right)^4 I_0^2. \quad (30)$$

We can now use these inequalities along with equations (27b) and (27a) to find bounds on c_r and c_i .

3.2 Bounds on the wave speed

Let us first discuss the wave speed c_r . While this quantity does not tell us anything about stability, it can be shown that it is still bounded (Synge, 1938; Pai, 1954; Joseph, 1968, 1969), which confines c to a strip in the complex plane. Rearrange (27a) to get

$$\int_{-1}^1 (U - c_r) |D\phi|^2 dz + \alpha^2 \int_{-1}^1 \left[(U - c_r) + \frac{U''}{2\alpha^2} \right] |\phi|^2 dz = 0. \quad (31)$$

If we look at the integrals in the above (31) we can see that there are a few different possible cases. First if $c_r < 0$, then depending on the sign of U there are two possibilities. Since it is a non-trivial function, U can be split into regions where it is strictly positive and strictly negative.

For ranges of z where U is strictly positive, $U - c_r > 0$, implying that the first integral must be positive which means that the second integral must be negative. This means that the statement inside the second integral must also be negative, implying that

$$\begin{aligned} (U - c_r) + \frac{U''}{2\alpha^2} &< 0 \text{ for some } z \in (-1, 1) \\ \implies c_r &> \frac{U''_{min}}{2\alpha^2} + U_{min}, \end{aligned} \quad (32)$$

where the *min* subscript indicates the minimum value taken by U or U'' on this region.

For ranges of z where U is strictly negative, $U - c_r < 0$, implying that the first integral must be negative and therefore the second integral must be positive. This gives a similar result, that

$$\begin{aligned} (U - c_r) + \frac{U''}{2\alpha^2} &> 0 \text{ for some } z \in (-1, 1) \\ \implies c_r &> \frac{U''_{max}}{2\alpha^2} + U_{max}, \end{aligned}$$

where similarly, the *max* subscript indicates the maximum value taken by U or U'' on this region.

The second case is the case where $c_r > 0$. In a region where U is strictly positive, we need that the first integral is negative to get a different inequality (and therefore an upper bound) to (32). We, therefore, impose the additional condition that $c_r \geq U_{max}$, where U_{max} is the maximum value taken by U on this region. The first integral is now negative which means that the second integral must be positive. This means that

$$\begin{aligned} (U - c_r) + \frac{U''}{2\alpha^2} &> 0 \text{ for some } z \in (-1, 1) \\ \implies c_r &< \frac{U''_{max}}{2\alpha^2} + U_{max}. \end{aligned}$$

In a region where U is negative, we want the first integral to be positive, meaning that we need the additional condition $c_r < U_{min}$, where U_{min} is the minimum value taken by U on this region. The first integral is now positive, implying that the second integral is negative. This implies that

$$\begin{aligned} (U - c_r) + \frac{U''}{2\alpha^2} &< 0 \text{ for some } z \in (-1, 1) \\ \implies c_r &> \frac{U''_{min}}{2\alpha^2} + U_{min}. \end{aligned}$$

Combining these inequalities gives the same bounds for both the strictly positive and strictly negative cases

$$\frac{U''_{min}}{2\alpha^2} + U_{min} < c_r < \frac{U''_{max}}{2\alpha^2} + U_{max}.$$

If we now expand across all the regions of positive and negative U , we notice that the *max* and *min* subscripts can safely be redefined to be the maximum and minimum values taken by U and U'' across the whole range of $z = -1$ to $z = 1$. This bound was first derived by Pai (1954), though he used generalised bounds and left out the " U is strictly negative" case, presumably because of the argument applied here.

For plane Couette flow we have that $U'' = 0$ which leaves us with

$$U_{min} < c_r < U_{max}.$$

For plane Poiseuille flow we have that $U_{min} = 0$, $U''_{max} = 0$ which gives

$$\frac{U''_{min}}{2\alpha^2} < c_r < U_{max}.$$

In our case, with the range $-1 \leq z \leq 1$ we have that for Couette flow

$$-1 < c_r < 1,$$

and for Poiseuille flow

$$\frac{-1}{\alpha^2} < c_r < 1.$$

We can improve this result by considering the mean values of U and U'' . Let a and b represent the mean values of U and U'' respectively (in the range $(-1, 1)$). From (27a) we have that

$$\begin{aligned} c_r &= \frac{(U(a) + \alpha^2 U(a))I_0^2 + \frac{1}{2}U''(b)I_0^2}{I_1^2 + \alpha^2 I_0^2} \\ &= \frac{(U(a) + \alpha^2 U(a))I_0^2 + \frac{1}{2}U''(b)I_0^2}{I_1^2 + \alpha^2 I_0^2} \\ &= U(a) + \frac{U''(b)}{2(\frac{I_1^2}{I_0^2} + \alpha^2)}. \end{aligned} \quad (33)$$

Again there are multiple cases. Let us start with the case that U'' is strictly positive. If $U''_{min} \geq 0$, we must have that $U''_{max} > 0$ and therefore that

$$0 < \frac{U''(b)}{2\left[\frac{I_1^2}{I_0^2} + \alpha^2\right]},$$

since $U_{min} < U(a) < U_{max}$ by definition. If we now use the fact that $\frac{I_1^2}{I_0^2} \geq \frac{1}{4}\pi^2$ from our inequalities on I_i^2 (30), we end up with

$$0 < \frac{2U''_{max}}{\pi^2 + 4\alpha^2}.$$

This leaves us with the bound

$$U_{min} < c_r < U_{max} + \frac{2U''_{max}}{\pi^2 + 4\alpha^2}, \quad (34)$$

for this case. Note that the lower bound cannot be improved since we don't know whether U''_{min} is at the same value of z .

If we consider a case where U'' is strictly negative, we must have that $U''_{max} \leq 0$. This implies that

$$\frac{2U''_{min}}{\pi^2 + 4\alpha^2} < \frac{U''(b)}{2\left[\frac{I_1^2}{I_0^2} + \alpha^2\right]},$$

leaving us with the following bounds in this strictly negative case

$$U_{min} + \frac{2U''_{min}}{\pi^2 + 4\alpha^2} < c_r < U_{max}. \quad (35)$$

There is one final case to consider, the case where U'' is not strictly positive or negative. In this case, we have that $U''_{min} < 0 < U''_{max}$. This means we have the following bounds on the U'' term in (33),

$$\frac{U''_{min}}{2\left[\frac{1}{4}\pi^2 + \alpha^2\right]} < \frac{U''(b)}{2\left[\frac{I_1^2}{I_0^2} + \alpha^2\right]} < \frac{U''_{max}}{2\left[\frac{1}{4}\pi^2 + \alpha^2\right]}.$$

This leaves us with our final bound on c_r ,

$$U_{min} + \frac{2U''_{min}}{\pi^2 + 4\alpha^2} < c_r < \frac{2U''_{max}}{\pi^2 + 4\alpha^2}. \quad (36)$$

Combining these three bounds (34), (35) and (36) gives us the following theorem first derived by Joseph (1969).

Theorem (Joseph (1969) Theorem 2). *Let $c(\alpha, R)$ be any eigenvalue of (21). Then the following holds:*

- $U''_{min} \geq 0$:
 $- U_{min} < c_r < U_{max} + \frac{2U''_{max}}{\pi^2 + 4\alpha^2};$
- $U''_{min} < 0 < U''_{max}$:
 $- U_{min} + \frac{2U''_{min}}{\pi^2 + 4\alpha^2} < c_r < \frac{2U''_{max}}{\pi^2 + 4\alpha^2}$
- $U''_{max} \leq 0$:
 $- U_{min} + \frac{2U''_{min}}{\pi^2 + 4\alpha^2} < c_r < U_{max}$

For Poiseuille flow between $-1 < z < 1$, we have that $U'' = -2$, which means that case 3 of the theorem is satisfied. This leaves us with the following bounds for c_r :

$$\frac{-4}{\pi^2 + 4\alpha^2} < c_r < 1.$$

In the case of Couette flow, we have that $U'' = 0$, which means that cases 1 and 3 of the theorem are satisfied. This may seem like an issue, but they both give the same bounds on c_r ,

$$-1 < c_r < 1.$$

Notice that these are the same as the bounds given by Pai, so in the case of Couette flow, Joseph's bounds don't offer any improvement. However, if we consider the mode with $\alpha = 1$ in the case of Poiseuille flow, then Joseph's bounds are an improvement. Notice that in both cases, as $\alpha \rightarrow \infty$, c_r is found in the range 0 to 1, whereas for Couette flow, c_r is found in the range -1 to 1.

Both these results limit c_r to strips in the complex plane. However, they don't give us any direct results relating to stability. For that, we now turn to the growth rate c_i .

3.3 Bounds on the growth rate

The growth rate is important because a state cannot be stable if $c_i > 0$ for any eigenvalue of (21). This is because if we look at our streamfunction solution (18) we can see that any solutions with $c_i > 0$ will blow up as t increases, while any with $c_i < 0$ will decay, implying they are stable. $c_i > 0$ is therefore generally taken as the *criterion for instability*.

First, let us show that the following inequality holds

$$|\text{Im}\{Q\}| \leq qI_1I_0,$$

where $q = U'_{max}$ and

$$I_1 = \int_{-1}^1 D\phi \, dz, \quad I_0 = \int_{-1}^1 \phi \, dz.$$

Proof. We have that from (26)

$$|\text{Im}\{Q\}| = \frac{1}{2} \int_{-1}^1 |U'(\phi(D\phi_c) - (D\phi)\phi_c)| \, dz.$$

From repeated uses of Schwarz's Inequality, we get the following

$$\begin{aligned}
|\operatorname{Im}\{Q\}| &\leq \int_{-1}^1 |U'| \left| \frac{1}{2}(\phi(D\phi_c) - (D\phi)\phi_c) \right| dz \\
&\leq \int_{-1}^1 |U'| |D\phi\phi_c| dz \\
&\leq \int_{-1}^1 |U'| |D\phi| |\phi_c| dz = \int_{-1}^1 |U'| |D\phi| \phi dz \\
&\leq \int_{-1}^1 |U'| dz I_1 I_0 \\
&\leq U'_{max} I_1 I_0 = q I_1 I_0.
\end{aligned}$$

□

Using this inequality in (27b) gives us the following important result

$$c_i \leq \frac{(q I_1 I_0 - (\alpha R)^{-1}(I_2^2 + 2\alpha^2 I_1^2 + \alpha^4 I_0^2))}{I_1^2 + \alpha^2 I_0^2}. \quad (37)$$

To improve this bound on c_i , consider a few more results as in Joseph (1968, 1969). These all use (30). First we show that $2\alpha I_0 I_1 \leq I_1^2 + \alpha^2 I_0^2$

$$\begin{aligned}
(I_1 - \alpha I_0)^2 &= I_1^2 - 2\alpha I_0 I_1 + \alpha^2 I_0^2 \geq 0 \\
\implies 2\alpha I_0 I_1 &\leq I_1^2 + \alpha^2 I_0^2.
\end{aligned} \quad (38)$$

The second result we need is to show that $\frac{I_2^2}{I_0 I_1} \geq \pi(\frac{4.73}{2})^2$.

$$\begin{aligned}
I_2^2 I_2^2 &\geq \pi^2 \left(\frac{4.73}{2}\right)^4 I_0^2 I_1^2 \implies I_2 \geq \pi \left(\frac{4.73}{2}\right)^2 I_0 I_1 \\
\implies \frac{I_2^2}{I_1 I_0} &\geq \pi \left(\frac{4.73}{2}\right)^2.
\end{aligned}$$

The final result we need is to rearrange the following fraction

$$\begin{aligned}
\frac{I_2^2 + 2\alpha^2 I_1^2 + \alpha^4 I_0^2}{I_1^2 + \alpha^2 I_0^2} &= \frac{I_2^2 + \alpha^2 I_1^2 + \alpha^2(I_1^2 + \alpha^2 I_0^2)}{I_1^2 + \alpha^2 I_0^2} \\
&= \frac{I_2^2 + \alpha^2 I_1^2}{I_1^2 + \alpha^2 I_0^2} + \alpha^2.
\end{aligned}$$

Using (30) we have

$$\begin{aligned}
\frac{I_2^2 + \alpha^2 I_1^2}{I_1^2 + \alpha^2 I_0^2} + \alpha^2 &\geq \frac{\pi^2 I_1^2 + \frac{1}{4}\pi^2 \alpha^2 I_0^2}{\frac{1}{4}\pi^2 I_0^2 + \alpha^2 I_0^2} + \alpha^2 \\
&\geq \frac{(\frac{1}{4}\pi^4 + \frac{1}{4}\pi^2 \alpha^2) I_0^2}{(\frac{1}{4}\pi^2 + \alpha^2) I_0^2} + \alpha^2 \geq \frac{\pi^2(\pi^2 + \alpha^2)}{\pi^2 + 4\alpha^2} + \alpha^2.
\end{aligned} \quad (39)$$

Substituting (38), (39) into (37) gives

$$\begin{aligned}
c_i &\leq \frac{q(\frac{1}{2\alpha}(I_1^2 + \alpha^2 I_0^2))}{I_1^2 + \alpha^2 I_0^2} - \frac{(\alpha R)^{-1}(I_2^2 + 2\alpha^2 I_1^2 + \alpha^4 I_0^2)}{I_1^2 + \alpha^2 I_0^2} \\
c_i &\leq \frac{q}{2\alpha} - (\alpha R)^{-1} \left[\frac{\pi^2(\pi^2 + \alpha^2)}{\pi^2 + 4\alpha^2} + \alpha^2 \right].
\end{aligned} \quad (40)$$

This is the first result of Joseph's Theorem 1 (Joseph, 1969).

We'd like to find bounds on α and R to see when a flow is stable. Consider a stable flow so that $c_i < 0$. If we rearrange (37), we find that the following inequality holds

$$\frac{(qI_1I_0 - (\alpha R)^{-1}(I_2^2 + 2\alpha^2I_1^2 + \alpha^4I_0^2))}{I_1^2 + \alpha^2I_0^2} < 0. \quad (41)$$

It would be nice to get a bound on stability that doesn't require us to have a solution ϕ and just depends on α and R , similar to (40). Let's start with a less accurate method by considering the following integral as in Synge (1938)

$$\begin{aligned} & \int_{-1}^1 (\phi + az\phi' + b\phi'')(\phi_c + az\phi'_c + b\phi''_c) dz \\ &= I_0^2 + \int_{-1}^1 az\phi\phi'_c + b\phi\phi''_c + az\phi'\phi_c + a^2z^2|\phi'|^2 \\ & \quad + abz\phi'\phi''_c + b\phi''\phi_c + abz\phi''\phi'_c dz + b^2I_2^2 \\ &= I_0^2 + a \int_{-1}^1 z(\phi\phi'_c + \phi'\phi_c) dz + b \int_{-1}^1 \phi\phi''_c + \phi''\phi_c dz \\ & \quad + a^2 \int_{-1}^1 z^2|\phi'|^2 dz + ab \int_{-1}^1 z(\phi'\phi''_c + \phi''\phi'_c) dz + b^2I_2^2 > 0. \end{aligned} \quad (42)$$

We now want to express some of the more 'messy' integrals in the above as I_2^2, I_1^2, I_0^2 . Using the boundary conditions and the fact that

$$\begin{aligned} (\phi\phi_c)' &= \phi\phi'_c + \phi'\phi_c, \\ (\phi'\phi'_c)' &= \phi'\phi''_c + \phi''\phi'_c, \end{aligned}$$

we have

$$\int_{-1}^1 z(\phi'\phi''_c + \phi''\phi'_c) dz = \left[z(\phi'\phi'_c) \right]_{-1}^1 - \int_{-1}^1 \phi'\phi'_c dz = -I_1^2,$$

also

$$\int_{-1}^1 z(\phi\phi'_c + \phi'\phi_c) dz = \left[z\phi\phi_c \right]_{-1}^1 - \int_{-1}^1 \phi\phi_c dz = -I_0^2,$$

and

$$\int_{-1}^1 \phi\phi''_c + \phi''\phi_c dz = \left[\phi\phi'_c + \phi'\phi_c \right]_{-1}^1 - \int_{-1}^1 \phi'\phi'_c + \phi'\phi'_c dz = -2 \int_{-1}^1 \phi'\phi'_c dz = -2I_1^2.$$

Substituting these results into (42) gives us the following

$$I_0^2 - aI_0^2 - 2bI_1^2 + a^2 \int_{-1}^1 z^2|\phi'|^2 dz - abI_1^2 + b^2I_2^2 > 0. \quad (43)$$

Now we see that

$$\int_{-1}^1 z^2|\phi'|^2 dz \leq |z^2|_{max} I_1^2 = I_1^2,$$

by Schwarz's inequality and the fact that $|z^2|_{max} = 1$. This means that (43) is less than the following

$$(1 - a)I_0^2 + (a^2 - 2b - ab)I_1^2 + b^2I_2^2 > 0.$$

Rearranging gives us the following inequality

$$b^2 I_2^2 > (2b + ab - a^2) I_1^2 + (a - 1) I_0^2.$$

Putting this into (41) leaves us with

$$b^2 \alpha R q I_0 I_1 - I_1^2 (2\alpha^2 b^2 + ab - a^2 + 2b) - I_0^2 (\alpha^4 b^2 + a - 1) < 0.$$

For this to be true, the 'polynomial' on the left-hand side must have two real roots. Taking the determinant implies that

$$(b^2 \alpha R q)^2 < 4(2\alpha^2 b^2 + ab - a^2 + 2b)(\alpha^4 b^2 + a - 1).$$

With $a, b > 0$ and satisfying:

$$2\alpha^2 b^2 + ab - a^2 + 2b > 0, \alpha^4 b^2 + a - 1 > 0.$$

From here, we select 3 different pairs of a, b as given in Synge (1938).

Select $a = b = 1$:

$$\begin{aligned} (\alpha R q)^2 &< 8\alpha^4(\alpha^2 + 1) \\ \implies \alpha R q &< 2^{\frac{3}{2}}\alpha^2(\alpha^2 + 1)^{\frac{1}{2}} = A_1. \end{aligned}$$

We have selected the positive root as $qR > 0$.

Select $a = b = 2$:

$$\begin{aligned} 16(\alpha R q)^2 &< 4(8\alpha^2 + 4)(4\alpha^4 + 1) \\ \implies \alpha R q &< (2\alpha^2 + 1)^{\frac{1}{2}}(4\alpha^4 + 1)^{\frac{1}{2}} = A_2. \end{aligned}$$

Select $a = b = \frac{1}{\alpha}$:

$$\begin{aligned} \frac{1}{\alpha^4}(\alpha R q)^2 &< 4\left(2 + \frac{2}{\alpha}\right)\left(\alpha^2 + \frac{1}{\alpha - 1}\right) \\ \implies \frac{1}{\alpha^4}(\alpha R q)^2 &< 8\left(\alpha^2 + \alpha - 1 + \frac{1}{\alpha^2}\right) \\ \implies \alpha R q &< 2^{\frac{3}{2}}\alpha(\alpha^4 + \alpha^3 - \alpha^2 + 1)^{\frac{1}{2}}. \end{aligned}$$

The names A_1, A_2, A_3 come from Synge (1938). We now have the result that a flow is definitely stable if $\alpha R q < \max[A_1, A_2, A_3]$. This result was compiled into the following theorem by Joseph (1968).

Theorem (Synge (1938) condition for stability). *No amplified disturbances of (21) exist if*

$$\alpha R q < \max[A_1, A_2, A_3],$$

where

$$\begin{aligned} A_1 &= 2^{\frac{3}{2}}\alpha^2(\alpha^2 + 1)^{\frac{1}{2}}, \\ A_2 &= (2\alpha^2 + 1)^{\frac{1}{2}}(4\alpha^4 + 1)^{\frac{1}{2}}, \\ A_3 &= 2^{\frac{3}{2}}\alpha(1 - \alpha^2 + \alpha^3 + \alpha^4)^{\frac{1}{2}}. \end{aligned} \tag{44}$$

To improve Synge's result, rearrange (41) to find that the following inequality must be satisfied by a stable flow as found by Joseph (1968)

$$\alpha Rq < \frac{I_2^2 + 2\alpha^2 I_1^2 + \alpha^4 I_0^2}{I_1 I_0}. \quad (45)$$

Now rearrange the right-hand side of the above (45) to get

$$\begin{aligned} \frac{I_2^2 + 2\alpha^2 I_1^2 + \alpha^4 I_0^2}{I_0 I_1} &= \frac{I_2^2}{I_1 I_0} + \frac{2\alpha^2 I_1^2 + \alpha^4 I_0^2}{I_1 I_0} \\ &\geq \frac{1}{2}\pi \left[\frac{4.73}{2} \right]^2 + \frac{2\alpha^2 I_1^2 + \alpha^4 I_0^2}{I_0 I_1}. \end{aligned}$$

We have that

$$\frac{2\alpha^2 I_1^2 + \alpha^4 I_0^2}{I_0 I_1} \geq \frac{2\alpha^2 I_1^2}{I_0 I_1},$$

and that

$$\frac{2\alpha^2 I_1^2 + \alpha^4 I_0^2}{I_0 I_1} = \frac{2\alpha^2}{I_0 I_1} \left[\left[I_1 - \frac{\alpha}{\sqrt{2}} I_0 \right]^2 + \frac{2\alpha}{\sqrt{2}} I_0 I_1 \right]. \quad (46)$$

The above is obtained by completing the square. Putting these together gives

$$\begin{aligned} \frac{2\alpha^2 I_1^2 + \alpha^4 I_0^2}{I_0 I_1} &\geq \begin{cases} \frac{2\alpha^2}{I_0 I_1} \left[\left[I_1 - \frac{\alpha}{\sqrt{2}} I_0 \right]^2 + \frac{2\alpha}{\sqrt{2}} I_0 I_1 \right], \\ \frac{2\alpha^2 I_1^2}{I_0 I_1}, \end{cases} \\ &\geq \begin{cases} \frac{2\alpha^2}{I_0 I_1} \frac{2\alpha}{\sqrt{2}} I_0 I_1, \\ \frac{2\alpha^2 \frac{1}{4} \pi^2 I_0^2}{\frac{1}{2} \pi I_0^2} \end{cases} \\ &\geq \begin{cases} 2^{\frac{3}{2}} \alpha^3, \\ \pi \alpha^2. \end{cases} \end{aligned}$$

Now use this result in (46) to get

$$\alpha Rq < \begin{cases} \frac{1}{2}\pi \left(\frac{4.73}{2} \right)^2 + 2^{\frac{3}{2}} \alpha^3 = M_1, \\ \frac{1}{2}\pi \left(\frac{4.73}{2} \right)^2 + \pi \alpha^2 = M_2. \end{cases}$$

This time the names M_1 , M_2 come from Joseph (1968). This gives the result that

$$\alpha Rq < \max[M_1, M_2]. \quad (47)$$

The result (40) from earlier combined with this second result (47) forms the following theorem from Joseph (1968).

Theorem (Joseph (1968) Theorem 1).

$$c_i \leq \frac{q}{2\alpha} - (\alpha R)^{-1} \left[\frac{\pi^2(\pi^2 + \alpha^2)}{\pi^2 + 4\alpha^2} + \alpha^2 \right].$$

Furthermore, no amplified circumstances of (21) exist if

$$\alpha Rq < \max[M_1, M_2],$$

where

$$\begin{aligned} M_1 &= \left[\frac{4.73}{2} \right]^2 \pi + 2^{\frac{3}{2}} \alpha^3, \\ M_2 &= \left[\left[\frac{4.73}{2} \right]^2 + \alpha^2 \right] \pi. \end{aligned} \quad (48)$$

Figure 3.1 shows a comparison between Synge's bounds (44) and Joseph's bounds (48). The lines were plotted by considering the point at which the bounds are equal. The orange line represents the bounds given by Synge, while the blue line represents the bounds given by Joseph. The area below each curve represents the areas where we would expect no instability, while the area above the curve is where we would expect instability. As such, these are referred to as *curves of marginal stability*. The area below both curves is referred to as the *area of certain linear stability* (Joseph, 1968).

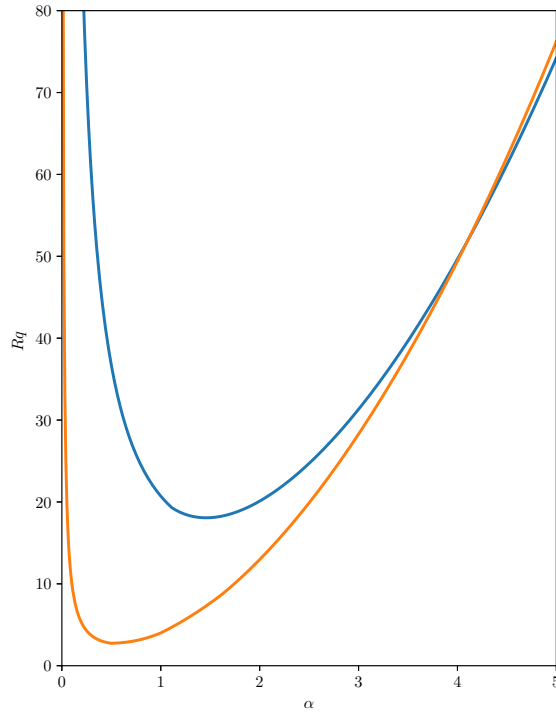


Figure 3.1: A comparison of the bounds given by Synge (44) (orange line) vs the bounds given by Joseph (48) (blue line)

In real life, we wouldn't expect a flow to be modelled by just one wave number, meaning that for a flow to be stable, it must be so for all wavenumbers. This means that these bounds predict the existence of a *critical Reynolds number*, found at the minimum point on this curve. The critical Reynolds number is the value of R above which we see instability. Synge's bound has a minimum point at $Rq = 2.74$, $\alpha = 0.50$. For Poiseuille flow, this corresponds to a critical R value of 1.37, whereas, for Couette flow, this corresponds to a critical value of 2.74 (recall that $q = 2$ for Poiseuille flow and $q = 1$ for Couette flow). Joseph's bounds predict a significantly higher critical R value of roughly 9.03 for Poiseuille flow and around 18.06 for Couette flow at $\alpha = 1.46$. These results predict that only flows with very low R will be stable and that Couette flow is more stable than Poiseuille flow. Notice that the more accurate bound (of Joseph) has a higher critical R value and a larger area of certain stability, implying that a more accurate curve may have an even higher critical R value and a larger area of certain stability.

These methods provide a bound on stability. However, they required a large number of significant approximations to derive and as such we wouldn't expect them to be very accurate. We also cannot get the values of specific eigenvalues for a flow, and it isn't easy to check whether a flow with a given R and α is stable. For this reason, we turn to a numerical method that will allow us to obtain these results along with a more accurate curve of marginal stability. The next chapter introduces the numerical method we will be using.

Chapter 4

Applying numerical methods to the Orr-Sommerfeld equation

This chapter focuses on a numerical method for finding the eigenvalues c of the Orr-Sommerfeld equation using Chebyshev polynomials. We will first introduce the Chebyshev polynomials and some of their properties before explaining how they can approximate functions and their derivatives. From there, we will discuss how we can obtain the eigenvalues of the Orr-Sommerfeld equation using the Chebyshev Tau method and how we can split the Orr-Sommerfeld equation into systems of lower-order differential equations to increase numerical accuracy.

4.1 Chebyshev polynomials

Chebyshev polynomials are a family of polynomials that are orthogonal with respect to the following inner product (Bukhari, 1997)

$$\langle f, g \rangle = \int_{-1}^1 \frac{fg}{\sqrt{1-z^2}} dz.$$

The k -th order Chebyshev polynomial is given by

$$T_k(z) = \cos(k\theta), \quad z = \cos \theta. \quad (49)$$

Specifically, if we take the inner product of two Chebyshev polynomials, we get the following result (Canuto et al., 2007),

$$\langle T_i(z), T_j(z) \rangle = \int_{-1}^1 \frac{T_i(z)T_j(z)}{\sqrt{1-z^2}} dz = \begin{cases} 0, & \text{if } i \neq j, \\ c_k \frac{\pi}{2}, & \text{if } i = j, \end{cases} \quad (50)$$

where $c_k = 2$ if $k = 0$ and $c_k = 1$ for all $k \geq 1$.

We've plotted the first 5 Chebyshev polynomials in Figure 4.1. From this, we can see that Chebyshev polynomials are odd when k is odd and even when k is even. They also appear to be well defined at $z = \pm 1$. It turns out that all Chebyshev polynomials take values of ± 1 at these endpoints (Canuto et al., 2007). The exact boundary values are as follows

$$T_k(\pm 1) = (\pm 1)^k, \quad T'_k(\pm 1) = (\pm 1)^{k+1} k^2. \quad (51)$$

The fact that Chebyshev polynomials take such well-defined values at the endpoints of our interval for z will be very useful when we come to represent the boundary conditions of the Orr-Sommerfeld equation in terms of Chebyshev polynomials in Section 4.7.

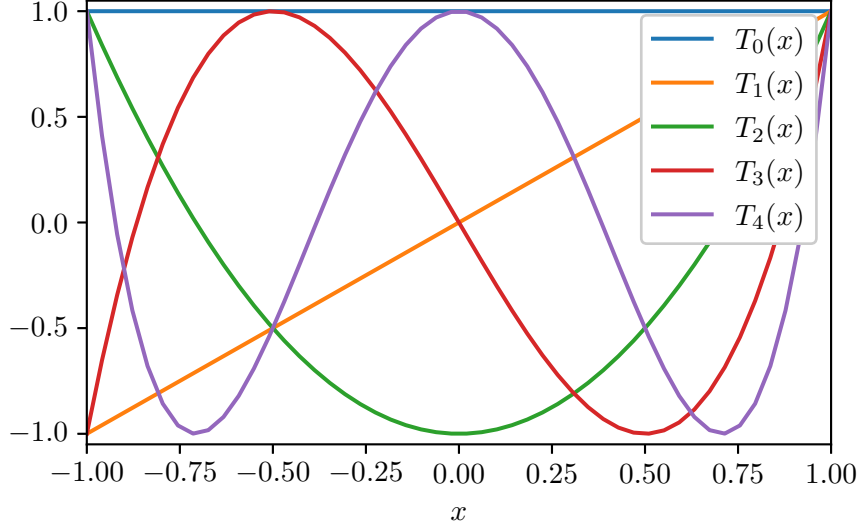


Figure 4.1: Chebyshev polynomials from $N = 0, \dots, 4$

One of the useful properties of Chebyshev polynomials is the fact that we can find recursive relations relating $T_k(z)$ of different order. Consider the trigonometric identity $\cos((k+1)\theta) + \cos((k-1)\theta) = 2\cos(\theta)\cos(k\theta)$. If we use $\theta = \arccos z$ as defined above we end up with the following expression

$$\cos((k+1)\arccos z) + \cos((k-1)\arccos z) = 2\cos(\arccos z)\cos(k\arccos z).$$

Notice that each of the terms of this expression has the form of a different order of Chebyshev polynomial. If we replace each term with its corresponding $T_k(z)$ and rearrange, we are then left with the following recursion relation

$$T_{k+1}(z) = 2zT_k(z) - T_{k-1}(z).$$

Before we move on to discussing how we will approximate our function, there is one more useful property of Chebyshev polynomials that we will need. The k th order Chebyshev polynomial has k zeros at the following points (Canuto et al., 2007)

$$z_i = \cos\left(\frac{(1+2i)\pi}{2k}\right), \quad i = 0, \dots, k-1. \quad (52)$$

We will use these in the next section for approximating integrals using the Chebyshev-Gauss quadrature.

4.2 Approximating a function

We express our function ϕ and its derivatives as a sum of $M+1$ Chebyshev polynomials

$$\frac{d^q \phi(z)}{dz^q} = \sum_{k=0}^M a_k^{(q)} T_k(z), \quad (53)$$

where $a_k^{(q)}$ are the expansion coefficients of the representation. This function expansion approximates ϕ up to a polynomial of degree M .

We represent our expansion as an $M + 1$ dimensional vector $\mathbf{a}$, where the k th component of $\mathbf{a}$ is the corresponding a_k from our expansion.

$$\mathbf{a} = \begin{pmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \dots \\ a_{M-1}^{(0)} \\ a_M^{(0)} \end{pmatrix}.$$

We will now show how to approximate the function

$$f(z) = \cos(z) = \sum_{k=0}^M a_k T_k(z), \quad (54)$$

as a sum of Chebyshev polynomials. Note that from now on, a_k with no superscript will represent $a_k^{(0)}$, the coefficients corresponding to the expansion of our undifferentiated function.

To calculate our a_k we multiply both sides of (54) by $T_i(z)/\sqrt{1-z^2}$ and then integrate between -1 and 1 . This leaves us with

$$\begin{aligned} \int_{-1}^1 \frac{\cos(z)T_i(z)}{\sqrt{1-z^2}} dz &= \int_{-1}^1 \left[\sum_{k=0}^M a_k T_k(z) \right] \frac{T_i(z)}{\sqrt{1-z^2}} dz \\ &= \sum_{k=0}^M a_k \int_{-1}^1 \frac{T_k(z)T_i(z)}{\sqrt{1-z^2}} dz. \end{aligned} \quad (55)$$

Notice that the integral inside the sum on the right-hand side of (55) corresponds to the inner product in (50). Since Chebyshev polynomials are orthogonal, we are just left with a single coefficient when $i = k$. This leaves us with

$$\int_{-1}^1 \frac{\cos(z)T_i(z)}{\sqrt{1-z^2}} dz = a_i c_i \frac{\pi}{2},$$

which can be rearranged to give us an explicit formula for our a_k as seen in Canuto et al. (2007)

$$a_k = \frac{2}{\pi c_k} \int_{-1}^1 \frac{f(z)T_k(z)}{\sqrt{1-z^2}} dz. \quad (56)$$

Note: the indices i and k have been swapped for consistency with how the coefficients are referenced in our expansion.

We now have a way of calculating the corresponding Chebyshev expansion of $\cos(z)$. However, before we can implement this in Python we need a method of approximating the integral in (56). To do this we will use a Chebyshev-Gauss quadrature method. We can approximate the weighted integral in (56) as the following sum

$$\int_{-1}^1 \frac{g(z)}{\sqrt{1-z^2}} dz = \sum_{i=0}^N g(z_i) w_i,$$

where the z_i are the $N+1$ zeros of T_{N+1} as stated in (52). It can be shown that the corresponding w_i are constant with a value of $\pi/(N+1)$, and that the degree of precision for this approximation

is $2N + 1$ (Canuto et al., 2007). Substituting this approximation into (56) leaves us with the following approximation for our a_k

$$\begin{aligned} a_k &= \frac{2}{\pi c_k} \left[\frac{\pi}{N+1} \sum_{i=0}^N f(z_i) \cos\left(\frac{k(1+2i)\pi}{2(N+1)}\right) \right] \\ &= \frac{2}{c_k(N+1)} \sum_{i=0}^N f(z_i) \cos\left(\frac{k(1+2i)\pi}{2(N+1)}\right). \end{aligned}$$

We have implemented this method in Python and the code written can be found in Appendix A.4. The approximation for different numbers of polynomials can be found in Figure 4.2.

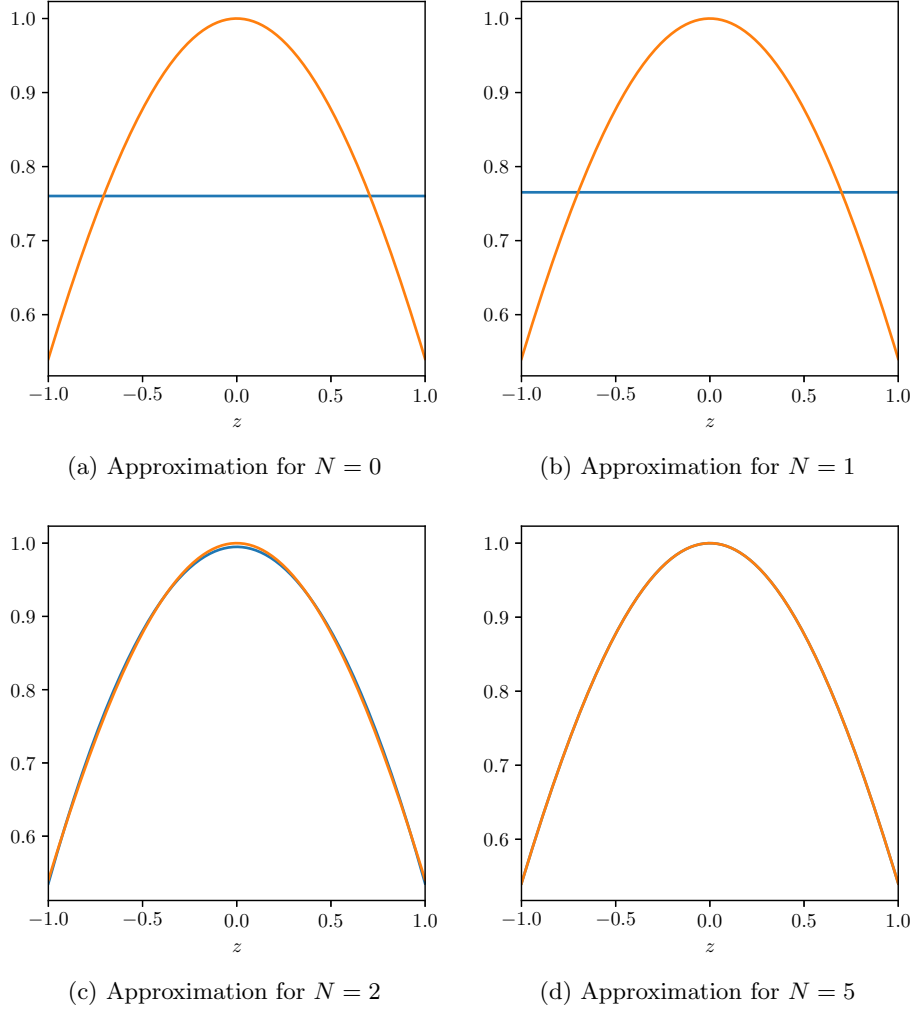
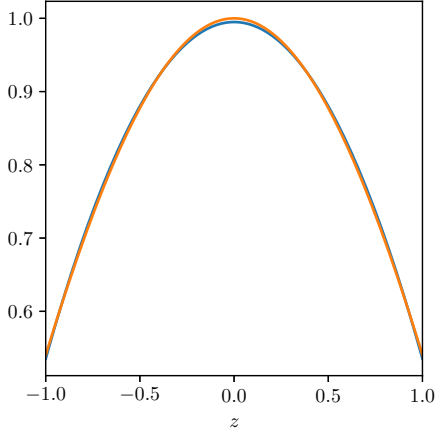
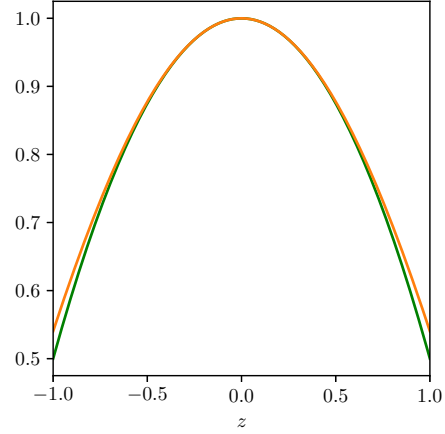


Figure 4.2: The Chebyshev approximation of $\cos(z)$ with differing numbers of polynomials (differing values of N). The orange line in each plot corresponds to $\cos(z)$, while the blue line corresponds to an approximation.

As can be seen, the approximation doesn't change between $N = 0$ and $N = 1$. This makes sense as the Taylor expansion of $\cos z$ around $z = 0$ doesn't include any odd powers of z , meaning we wouldn't expect any Chebyshev polynomials with odd order in our approximation. The Chebyshev approximation also clearly converges extremely fast, even with $N = 2$ there is very little difference between the approximation and the actual function. Figure 4.3 shows a comparison between the second-order Chebyshev approximation of $\cos(z)$ (left), and the second-order Taylor approximation of $\cos(z)$ (right).



(a) Second order Chebyshev approximation of $\cos(z)$



(b) Second order Taylor approximation of $\cos(z)$

Figure 4.3: The Chebyshev approximation of $\cos(z)$ with $N = 2$ blue line compared to the Taylor approximation of $\cos(z)$ (green line).

We can see that while the Chebyshev approximation takes an incorrect value at $z = 0$, there is a significant divergence between the Taylor approximation and $\cos(z)$ at the endpoints. This implies that the Chebyshev approximation converges to $\cos(z)$ much faster than the Taylor approximation.

Now we have a method of approximating a function, we need a method of representing the derivatives of a function in terms of some matrix D^m multiplied by our approximation vector $\mathbf{a}$, where m represents the order of derivative we are calculating. This will allow us to represent the action of differential operators on our expansion as the action of a matrix on our expansion. Calculating the form of D^m is the focus of the next section.

4.3 Representing derivatives

Differentiating (49) with respect to z implies that the first derivative of the k -th order Chebyshev polynomial has the following form

$$\frac{d}{dz} T_k(z) = \frac{-k \sin(k \arccos z)}{\sqrt{1-z^2}} = \frac{-k \sin(k\theta)}{\sin \theta}, \quad (57)$$

where $z = \cos \theta$ as before.

There are 2 important things to note from this definition:

1. We have that $T'_k(z) = T'_{-k}(z)$.
2. We have the limit $\lim_{k \rightarrow 0} T'_k(z)/k = \lim_{k \rightarrow 0} -\sin(k\theta)/\sin \theta = 0$.

If now we consider the trigonometric relation $2 \sin \theta \cos \theta = \sin(k+1)\theta - \sin(k-1)\theta$ we have that

$$\begin{aligned} 2 \cos \theta &= \frac{\sin((k+1)\theta)}{\sin \theta} - \frac{\sin((k-1)\theta)}{\sin \theta}, \\ 2T_k(z) &= \frac{1}{k+1} T'_{k+1}(z) - \frac{1}{k-1} T'_{k-1}(z), \end{aligned} \quad (58)$$

which gives us a recursive relation between different orders of derivatives of Chebyshev polynomials.

We would like to use this recursion relation to find a recursion relation between the coefficients of our undifferentiated function a_k and the coefficients of a derivative of our function $a_k^{(q)}$. If we consider the first derivative of a perfect expansion of the form in (53) (i.e. the case where $N = \infty$, we can see that the following are equivalent

$$\sum_{k=0}^{\infty} a_k^{(0)} T'_k(z) = \frac{d\phi}{dz} = \sum_{k=0}^{\infty} a_k^{(1)} T_k.$$

We can use this result along with the recursion relation (58) to get the following

$$\begin{aligned} \sum_{k=0}^{\infty} a_k^{(0)} T'_k(z) &= \frac{1}{2} \sum_{k=0}^{\infty} a_k^{(1)} \left[\frac{1}{k+1} T'_{k+1}(z) - \frac{1}{k-1} T'_{k-1}(z) \right] \\ &= \frac{1}{2} \left[\sum_{k=0}^{\infty} \frac{a_k^{(1)}}{k+1} T'_{k+1}(z) - \sum_{k=0}^{\infty} \frac{a_k^{(1)}}{k-1} T'_{k-1}(z) \right]. \end{aligned} \quad (59)$$

To get a recursion relation between our coefficients, we would like to be able to compare the terms of the sums in (59). Define $n = k+1$ and $m = k-1$. If we use these as the indices in the two sums on the right of (59) we get the following

$$\begin{aligned} \sum_{k=0}^{\infty} a_k^{(0)} T'_k(z) &= \frac{1}{2} \left[\sum_{n=1}^{\infty} \frac{a_{n-1}^{(1)}}{n} T'_n(z) - \sum_{m=-1}^{\infty} \frac{a_{m+1}^{(1)}}{m} T'_m(z) \right] \\ &= \frac{1}{2} \left[\sum_{k=1}^{\infty} \left[\frac{a_{k-1}^{(1)}}{k} - \frac{a_{k+1}^{(1)}}{k} \right] T'_k(z) + a_0^{(1)} T'_{-1}(z) - \frac{a_1^{(1)}}{0} T_0(z) \right]. \end{aligned}$$

Notice that $T'_1(z) = T'_{-1}(z)$ in the second to last term thanks to the first of the properties of the derivative (57) and that the last term vanishes thanks to the second of the properties of the derivative (57). This leaves us with

$$\sum_{k=0}^{\infty} a_k^{(0)} T'_k(z) = \frac{1}{2} \sum_{k=1}^{\infty} \left[\frac{c_{k-1} a_{k-1}^{(1)}}{k} - \frac{a_{k+1}^{(1)}}{k} T'_k(z) \right],$$

From here we extract our recursion relation for our coefficients

$$2ka_k^{(0)} = c_{k-1} a_{k-1}^{(1)} - a_{k+1}^{(1)}, \quad k \geq 1. \quad (60)$$

We would like an expression for $a_k^{(1)}$ that only uses a_p for various $p \geq 0$. If we apply (60) to itself we get

$$\begin{aligned} c_{k-1} a_{k-1}^{(1)} &= 2ka_k + a_{k+1}^{(1)} \\ &= 2ka_k + (2(k+2)a_{k+2} + a_{k+3}^{(1)}) \\ &= 2ka_k + 2(k+2)a_{k+2} + (2(k+4)a_{k+4} + a_{k+5}^{(1)}) \\ &\vdots \end{aligned}$$

If we continue this process up to a truncation point of M polynomials, we obtain the following formula for the coefficients of the first derivative of ϕ as seen in the literature (Canuto et al., 2007; Orszag, 1971)

$$a_k^{(1)} = \frac{2}{c_k} \sum_{\substack{p=k+1 \\ p \equiv 1 \pmod{2}}}^M pa_p. \quad (61)$$

To get higher derivatives we can just apply this method recursively. The second derivative is obtained by the following

$$a_k^{(2)} = \frac{2}{c_k} \sum_{\substack{p=k+1 \\ p \equiv 1 \pmod{2}}}^M p a_p^{(1)}.$$

Applying the sum (61) again, we can get an expression for the coefficients of the 2nd derivative of ϕ given by

$$a_k^{(2)} = \frac{1}{c_k} \sum_{\substack{p=k+2 \\ p \equiv 0 \pmod{2}}}^M p(p^2 - k^2) a_p.$$

In a similar way, we can compute an expression for $a_k^{(4)}$ given by

$$a_k^{(4)} = \frac{1}{c_k} \sum_{\substack{p=k+4 \\ p \equiv 0 \pmod{2}}}^N p[p^2(p^2 - 4)^2 - 3k^2p^4 + 3k^4p^2 - k^2(k^2 - 4)^2] a_k. \quad (62)$$

It is clear that for large p and k , (62) grows extremely fast. This means that numerically we can lose significance as M becomes large. To reduce this, Herbert (1977) suggested that the substitution $q = p - k$ be used inside the brackets. This transforms (62) into

$$a_k^{(4)} = \frac{1}{c_k} \sum_{\substack{p=k+4 \\ p \equiv 0 \pmod{2}}}^N p[q^4(q^2 - 8) + kq^3(6q^2 - 32) + (12k^2q^2 + 8k^3q)(q^2 - 4) + 16q^2 + 32kq] a_k.$$

We can then represent the coefficients of a derivative of ϕ by multiplying our vector $\mathbf{a}$ by an $(M+1) \times (M+1)$ matrix D^n

For $n = 1$, we have

$$D = \begin{pmatrix} 0 & 1 & 0 & 3 & 0 & 5 & 0 & 7 & 0 & \dots \\ 0 & 0 & 4 & 0 & 8 & 0 & 12 & 0 & 16 & \dots \\ 0 & 0 & 0 & 6 & 0 & 10 & 0 & 14 & 0 & \dots \\ 0 & 0 & 0 & 0 & 8 & 0 & 12 & 0 & 16 & \dots \\ 0 & 0 & 0 & 0 & 0 & 10 & 0 & 14 & 0 & \dots \\ 0 & 0 & 0 & 0 & 0 & 0 & 12 & 0 & 16 & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \end{pmatrix}.$$

For $n = 2$, we have

$$D^2 = \begin{pmatrix} 0 & 0 & 4 & 0 & 32 & 0 & 108 & 0 & 256 & \dots \\ 0 & 0 & 0 & 24 & 0 & 120 & 0 & 336 & 0 & \dots \\ 0 & 0 & 0 & 0 & 48 & 0 & 192 & 0 & 480 & \dots \\ 0 & 0 & 0 & 0 & 0 & 80 & 0 & 280 & 0 & \dots \\ 0 & 0 & 0 & 0 & 0 & 0 & 120 & 0 & 384 & \dots \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 168 & 0 & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \end{pmatrix}. \quad (63)$$

For $n = 4$, we have

$$D^4 = \begin{pmatrix} 0 & 0 & 0 & 0 & 192 & 0 & 4608 & 0 & 38400 & \dots \\ 0 & 0 & 0 & 0 & 0 & 1920 & 0 & 26880 & 0 & \dots \\ 0 & 0 & 0 & 0 & 0 & 0 & 5760 & 0 & 61440 & \dots \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 13440 & 0 & \dots \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 26880 & \dots \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \end{pmatrix}. \quad (64)$$

Notice that D^2 and D^4 are in fact multiples of D . This means in theory we can get any derivative of ϕ we want.

These matrices are implemented in Python in the code in Appendix A.5. We can then test them by multiplying the coefficients of our computed approximation for $\cos(z)$ by the matrices D, D^2, D^4 to get the coefficients of the expansion of our derivatives. Note that since D^4 is all zeroes if $N \leq 4$ we need to compute an approximation to $\cos(z)$ that is of a higher order than order 4. Figure 4.4 shows the original approximation for $N = 8$, along with the approximation for the first, second and fourth derivatives of $\cos(z)$.

From this, we can see that this method for calculating a derivative approximation is extremely accurate. This will allow us to represent differential operators as the action of a matrix on our vector $\mathbf{a}$. For example, the matrix L representing the action of the differential operator

$$\mathcal{L} = \frac{d^4}{dz^4} + 4\frac{d^2}{dz^2} - \frac{d^1}{dz^1},$$

would have the form

$$L = D^4 + 4D^2 - D.$$

We can now move on to discussing how we can apply these to represent the Orr-Sommerfeld equation in terms of these matrices.

4.4 Fourth Order System

To get the eigenvalues of the Orr-Sommerfeld equation we would like a generalised eigenvalue problem of the form

$$A\mathbf{a} = cB\mathbf{a},$$

where $\mathbf{a}$ is the $M+1$ dimensional vector representing the coefficients of the Chebyshev expansion of our eigenfunction ϕ , and A and B are $(M+1) \times (M+1)$ matrices.

To get matrix expressions for A and B we rearrange the Orr-Sommerfeld equation (21) to be in the following form

$$(i\alpha R)^{-1}(D^4 - 2\alpha^2 D^2 + \alpha^4)\phi - U(D^2 - \alpha^2)\phi + U''\phi = -c(D^2 - \alpha^2)\phi.$$

From here we can see that A and B take the following forms

$$\begin{aligned} A &= (i\alpha R)^{-1}(D^4 - 2\alpha^2 D^2 + \alpha^4 I) - U(D^2 - \alpha^2 I) + U''I, \\ B &= -(D^2 - \alpha^2 I). \end{aligned}$$

How do we represent our U and U'' ? The best way to do this is to work out some matrix that we can then multiply our coefficients for the expansion of ϕ and $D^2\phi$ by (Bukhari, 1997). To do

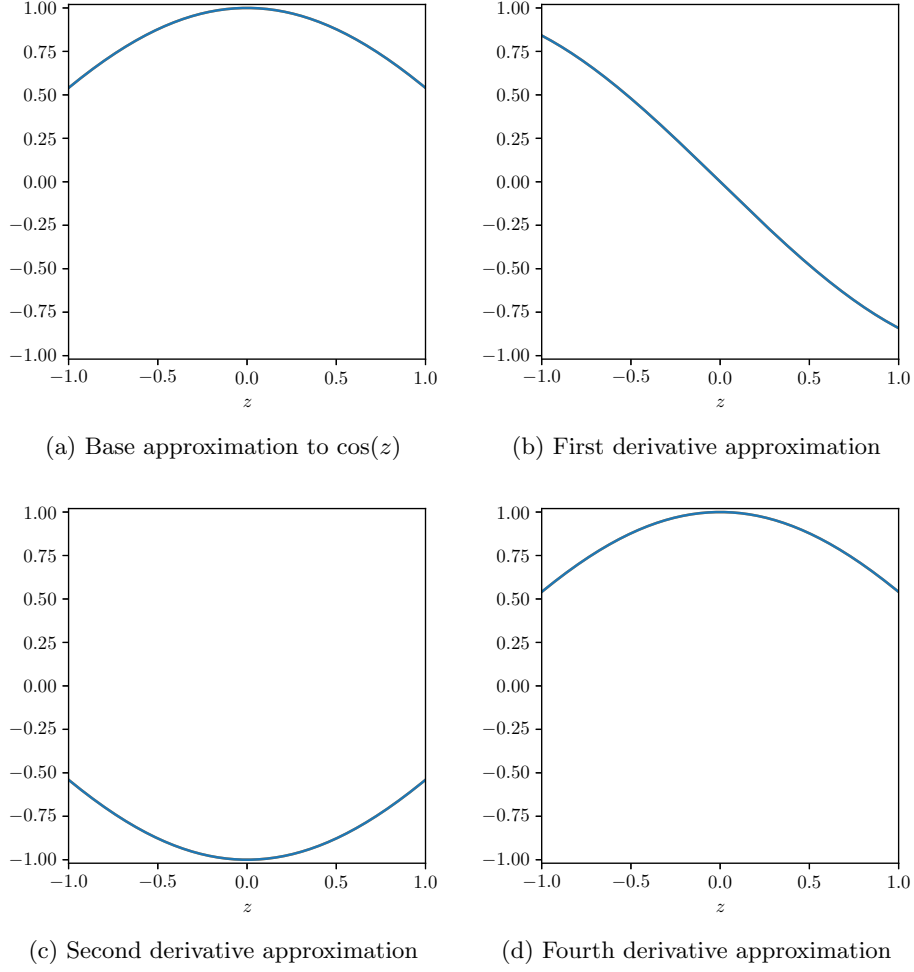


Figure 4.4: The Chebyshev approximation of $\cos(z)$ for $N = 8$, compared with the Chebyshev approximations of its derivatives. The (obscured) orange line in each plot corresponds to an exact derivative of $\cos(z)$, while the blue line corresponds to the Chebyshev approximation of that derivative.

this let us assume we can accurately compute a Chebyshev expansion for our U and U'' of the form

$$u(z) = \sum_{j=0}^{\infty} b_j T_j(z).$$

We then would like to consider the product

$$\begin{aligned} h(z) &= u(z)\phi(z) \\ &= \sum_{j=0}^{\infty} \sum_{k=0}^M a_k b_j T_j(z) T_k(z), \end{aligned} \tag{65}$$

of 2 Chebyshev expansions. Can we represent the product $T_j(z)T_k(z)$ in a nicer way? If we consider the trigonometric relation

$$\cos(i\theta) \cos(j\theta) = \frac{\cos((i+j)\theta) + \cos((i-j)\theta)}{2},$$

with $\theta = \arccos(z)$ then we end up with the following

$$\begin{aligned} T_i(z)T_j(z) &= \frac{T_{i+j}(z) + T_{i-j}(z)}{2} \\ &= \frac{T_{i+j}(z) + T_{|i-j|}(z)}{2}, \end{aligned}$$

where the $|i-j|$ just comes from the fact that $T_k(z) = T_{-k}(z)$. If we substitute this back into the product (65) we get the following

$$\begin{aligned} h(z) &= \frac{1}{2} \sum_{j=0}^{\infty} \sum_{k=0}^M a_k b_j [T_{j+k}(z) + T_{|j-k|}(z)] \\ &= \frac{1}{2} \sum_{j=0}^{\infty} \sum_{k=0}^M a_k b_j T_{j+k}(z) + \frac{1}{2} \sum_{j=0}^{\infty} \sum_{k=0}^M a_k b_j T_{|j-k|}(z). \end{aligned} \quad (66)$$

If we change indices to $n = j + k$ in the first sum, we can split the first sum into two separate sums. We have that $b_{n-k} = 0$ if $n - k < 0$. This implies that if $n < M$, we only have non-zero b_{n-k} if $k \leq n$. If $n > M$ we have that $k \leq M$. This means that the following holds

$$\frac{1}{2} \sum_{j=0}^{\infty} \sum_{k=0}^M a_k b_j T_{j+k}(z) = \frac{1}{2} \sum_{n=0}^M \sum_{k=0}^n a_k b_{n-k} T_n(z) + \frac{1}{2} \sum_{n=M}^{\infty} \sum_{k=0}^M a_k b_{n-k} T_n(z).$$

For the second sum, there are two cases to consider. If $j \leq k$, $T_{|j-k|}(z) = T_{k-j}(z)$. If we substitute $m = k - j$ then we end up with that m must take a range of between 0 and M and that k must be larger than m , meaning we have the sum

$$\sum_{m=0}^M \sum_{k=m}^M a_k b_{k-m} T_m(z).$$

If $j > k$, if we let $p = j - k$ we end up with the final sum

$$\sum_{p=1}^{\infty} \sum_{k=0}^M a_k b_{p+k} T_p(z).$$

Notice that p starts at 1 rather than 0. This is because we already have the product $a_0 b_0$ in the first of these two sums.

Substituting these results back into (66) gives us our product split into 4 sums (as seen in Bukhari (1997))

$$\begin{aligned} h(z) &= \frac{1}{2} \sum_{n=0}^M \sum_{k=0}^n a_k b_{n-k} T_n(z) + \frac{1}{2} \sum_{n=M}^{\infty} \sum_{k=0}^M a_k b_{n-k} T_n(z) \\ &\quad + \frac{1}{2} \sum_{n=0}^M \sum_{k=n}^M a_k b_{k-n} T_n(z) + \frac{1}{2} \sum_{n=1}^{\infty} \sum_{k=0}^M a_k b_{n+k} T_p(z), \end{aligned}$$

where we have made the indices the same for clarity.

We can represent this product as an $(M+1) \times (M+1)$ matrix multiplied by a vector of the $M+1$ coefficients of our function $\phi(z)$. This means each element of the matrix A_{nk} will correspond to

the sum of the b_n that are multiplied by a_k . For example, for $n = 0$, we have that the first row would represent the sum

$$a_0 b_0 + \frac{1}{2} \sum_{n=1}^M a_n b_n,$$

and would therefore have the row vector form

$$\frac{1}{2} (2b_0 \quad b_1 \quad b_2 \quad \dots \quad b_M).$$

From here on, we will represent the matrices for U'' and U by P and Q respectively as in Bukhari (1997).

For plane Poiseuille flow we have that $U'' = -2 = -2T_0(z)$ which means that we just have that $b_0 = -2$ for its expansion and $U = 1 - z^2 = \frac{1}{2}T_0(z) - \frac{1}{2}T_2(z)$, meaning that for the expansion of U we have that $b_0 = \frac{1}{2}$ and $b_2 = -\frac{1}{2}$. This means we have the following forms for P and Q

$$P = -2I, \quad Q = \begin{pmatrix} 0.5 & 0 & -0.25 & 0 & \dots \\ 0 & 0.25 & 0 & -0.25 & \dots \\ -0.5 & 0 & 0.5 & 0 & \dots \\ 0 & -0.25 & 0 & 0.5 & \dots \\ \dots & \dots & \dots & \dots & \dots \end{pmatrix}.$$

For Couette flow $U'' = 0$ and $U = z = T_1(z)$, implying that P and Q have the following forms

$$P = 0, \quad Q = \begin{pmatrix} 0 & 0.5 & 0 & 0 & \dots \\ 1 & 0 & 0.5 & 0 & \dots \\ 0 & 0.5 & 0 & 0.5 & \dots \\ 0 & 0 & 0.5 & 0 & \dots \\ \dots & \dots & \dots & \dots & \dots \end{pmatrix}.$$

Note: When P and Q are given in Bukhari (1997) for the case of Poiseuille flow, P is mistakenly stated to be $2I$ instead of $-2I$ as here.

If we use the P and Q matrices, we end up with these final forms for matrices A and B

$$\begin{aligned} A &= (i\alpha R)^{-1}(D^4 - 2\alpha^2 D^2 + \alpha^4 I) - Q(D^2 - \alpha^2 I) + P, \\ B &= -(D^2 - \alpha^2 I). \end{aligned} \tag{67}$$

We now have a generalised eigenvalue problem of the form $A\mathbf{a} = cB\mathbf{a}$ which can be solved in Python. Details of exactly how this will be done are given in 4.8.

Notice that the terms of our D^4 matrix (64) scale with $O(M^7)$ and the terms of our D^2 matrix (63) scale with $O(M^3)$. This means it is better to only use the lower-order differentiation matrices if possible. The next section will explore how to do this in the 2nd order case.

4.5 Second Order System

How do we make our system require only the use of D^2 ? The answer is to split the Orr-Sommerfeld equation into a system of 2nd order equations. We define $\phi_1 = \phi$, $\phi_2 = D^2\phi$. These two functions allow us to represent the Orr-Sommerfeld equation (21) as the following system of 2nd order differential equations

$$\begin{aligned} \phi_2 &= D^2\phi_1, \\ (i\alpha R)^{-1}((D^2 - 2\alpha^2)\phi_2 + \alpha^4\phi_1) &= (U - c)(\phi_2 - \alpha^2\phi_1) - U''\phi_1. \end{aligned} \tag{68}$$

Rearrange these to get

$$D^2\phi_1 - \phi_2 = 0, \quad (69a)$$

$$(U\alpha^2 + U'')\phi_1 + ((i\alpha R)^{-1}((D^2 - 2\alpha^2) - U)\phi_2 + \alpha^4\phi_1) = c\alpha^2\phi_1 - c\phi_2. \quad (69b)$$

We want to put these into a generalised eigenvalue problem of the form $E\mathbf{b} = cF\mathbf{b}$ like for the 4th order case.

To begin let us work out what our vector $\mathbf{b}$ is. If we denote the vector associated to our expansion of ϕ_1 by $\mathbf{a}_1$ and the vector associated to our expansion of ϕ_2 by $\mathbf{a}_2$ then we can form $\mathbf{b} = (\mathbf{a}_1, \mathbf{a}_2)^\top$. This vector is $2(M+1)$ dimensional so therefore our matrices are $2(M+1) \times 2(M+1)$ matrices.

We set E and F out as in Bukhari (1997), with the first $M+1$ rows corresponding to equation (69a) and the last $M+1$ rows corresponding to equation (69b). The first $M+1$ columns will correspond to operations done to ϕ_1 and the last $M+1$ columns will correspond to operations done to ϕ_2 . This leaves us with the following forms for E and F ,

$$E = \begin{pmatrix} D^2 & -I \\ (P - \alpha^2 Q) + (iR)^{-1}\alpha^3 I & (i\alpha R)^{-1}(D^2 - 2\alpha^2 I) - Q \end{pmatrix}, \quad F = \begin{pmatrix} 0 & 0 \\ \alpha^2 I & -I \end{pmatrix}. \quad (70)$$

This leaves us with a generalised eigenvalue problem of the form $E\mathbf{b} = cB\mathbf{b}$, which we can solve in the same way as our 4th order system. Solving this should yield significantly lower numerical error than our 4th order system. However, we can reduce the numerical error further by splitting (68) further into a system of 1st order differential equations.

4.6 First Order System

To create a system where we only use D we need to split the Orr-Sommerfeld equation (21) into a system of 1st order differential equations. To do this we define

$$\phi_1 = \phi, \quad \phi_2 = D\phi, \quad \phi_3 = D^2\phi, \quad \phi_4 = D^3\phi.$$

From here we have the following system of 1st order equations

$$\begin{aligned} D\phi_1 &= \phi_2, \\ D\phi_2 &= \phi_3, \\ D\phi_3 &= \phi_4, \\ (i\alpha R)^{-1}\phi_4 &= (i\alpha R)^{-1}(2\alpha^2\phi_3 - \alpha^4\phi_1) + (U - c)(\phi_3 - \alpha^2\phi_1) - U''\phi_1. \end{aligned}$$

Again we want these in the form $G\mathbf{c} = cH\mathbf{c}$, so we rearrange the equations of (71) to get

$$\begin{aligned} D\phi_1 - \phi_2 &= 0, \\ D\phi_2 - \phi_3 &= 0, \\ D\phi_3 - \phi_4 &= 0, \\ (i\alpha R)^{-1}D\phi_4 - (i\alpha R)^{-1}(2\alpha^2 - U)\phi_3 + ((i\alpha R)^{-1}\alpha^4 + U\alpha^2 + U'')\phi_1 &= -c\phi_3 + c\alpha^2\phi_1. \end{aligned}$$

In the same way as the second order case, we represent the vectors corresponding to the expansions of our 4 functions ϕ_i by $\mathbf{a}_i$ with $i = 1, \dots, 4$. We set $\mathbf{c} = (\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3, \mathbf{a}_4)^\top$. We now have a $4(M+1)$ dimensional vector $\mathbf{c}$ which means that G and H will be $4(M+1) \times 4(M+1)$ matrices.

We again follow the layout of G and H as in Bukhari (1997). Column $(i-1)(M+1)$ to column $i(M+1)$ correspond to the expansion coefficients of ϕ_i for $i = 1, \dots, 4$. Likewise, the rows of G and H are filled by the matrices corresponding to equations (72) in groups of $M+1$ rows.

This leaves us with the following forms for G and H ,

$$G = \begin{pmatrix} D & -I & 0 & 0 \\ 0 & D & -I & 0 \\ 0 & 0 & D & -I \\ (P + \alpha^2 Q) + (iR)^{-1}\alpha^3 & 0 & -Q - (iR)^{-1}(2\alpha^2)I & (i\alpha R)^{-1}D \end{pmatrix}, \quad H = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \alpha^2 I & 0 & -I & 0 \end{pmatrix}.$$

We now have a generalised eigenvalue problem $G\mathbf{c} = cH\mathbf{c}$ that we can again solve. This system has lower numerical error than both of the other methods we have discussed but takes much longer due to the massively increased size of the matrices involved even compared to the second-order method.

4.7 Boundary Conditions

Before we can solve the resulting systems from any of our methods, we need a way to represent our boundary conditions. The Orr-Sommerfeld equation has the following boundary conditions,

$$\phi(\pm 1) = \phi'(\pm 1) = 0.$$

By using the two properties (51) we get the following equations for these boundary conditions

$$\begin{aligned} \phi(1) &= \sum_{k=0}^M a_k, \\ \phi(-1) &= \sum_{k=0}^M (-1)^k a_k, \\ \phi'(1) &= \sum_{k=0}^M k^2 a_k, \\ \phi'(-1) &= \sum_{k=0}^M (-1)^k k^2 a_k = 0. \end{aligned} \tag{73}$$

If we combine our matrices for any of our methods with these equations we have $M+5$ equations for $M+1$ unknowns, meaning that our system is over-determined and only has the trivial solution $a_k = 0$. To solve this we follow the method that Orszag (1971) used, to do this we 4 of the rows of our matrices with boundary equations so that we have $M+1$ equations in $M+1$ unknowns.

For the 4th order system, we simply replace the last 4 rows of A and B in (67) with row vectors representing our boundary conditions in A and a row of zeroes in B .

We follow a similar procedure for the 2nd order case. In this case, all 4 boundary conditions are applied to ϕ_1 and none to ϕ_2 . This is because $\phi_2 = D^2\phi$. This means the rows inserted into matrix E of (70) will be of the form $(\mathbf{v}, \mathbf{0})$ where $\mathbf{v}$ is the $M+1$ dimensional vector representing one of the 4 sums given in (73) and $\mathbf{0}$ is the $M+1$ dimensional zero vector.

We replace the $(M-1)th$, Mth , $2(M-1)th$ and $2Mth$ rows of E and F . As with the 4th order case, the rows replaced in F are just replaced with rows of zeroes. The specific order of how the rows are inserted does not matter (Bukhari, 1997).

Finally, we deal with the 1st order system. In this case, we apply the first two boundary conditions to ϕ_1 and the first two boundary conditions again to ϕ_2 . This is because ϕ_2 represents the first derivative of ϕ and therefore we don't need to apply the boundary conditions that fix the derivative of a function.

The rows representing the boundary conditions applied to ϕ_1 have the form $(\mathbf{v}, \mathbf{0}, \mathbf{0}, \mathbf{0})$ and the rows representing the boundary conditions applied to ϕ_2 have the form $(\mathbf{0}, \mathbf{v}, \mathbf{0}, \mathbf{0})$. They are inserted into the Mth , $2Mth$, $3Mth$ and $4Mth$ rows of G and H . Again we insert just rows of zeroes into H .

We now can move on to solving our systems.

4.8 Solving our system

For each of the above methods we now have a generalised eigenvalue problem of the form

$$A\mathbf{b} = cB\mathbf{b}. \quad (74)$$

We implement this system in Python (the code for this is found in the appendix), but we have a problem! The matrix B is singular due to the rows of zeros added by the boundary conditions, which means we cannot rearrange (74) to be in the form $AB^{-1}\mathbf{b} = c\mathbf{b}$ and therefore cannot be solved using just any eigenvalue method.

To solve them we use the QZ algorithm of Moler and Stewart (1973), as suggested by Dongarra et al. (1996a) which allows us to get the eigenvalues of this problem even though the matrix B is singular. In Python, this is implemented in the `scipy.linalg.eig` function in the `scipy` package, a link to which is found [here](#).

The reason that the QZ algorithm is preferred is two-fold. Firstly, the QZ algorithm can find all of the eigenvalues of the system (74) and does not require knowledge of the position of the eigenvalues as with shooting or search methods. This is useful for this case as we are searching for any eigenvalue with an imaginary part above 0 and therefore want all possible eigenvalues of our system. Secondly, as shown in Dongarra et al. (1996a), using a search technique such as Orszag (1971) can 'miss out' eigenvalues.

The QZ algorithm transforms A and B using Householder transformations and then calculates their eigenvalues as α_i and β_i , where the α_i are the eigenvalues of A and the β_i are the eigenvalues of B . The eigenvalues of our system are then obtained by dividing each α_i by its respective β_i . As our system is singular, we would expect some of our eigenvalues to be 'infinite' or 'spurious' eigenvalues caused by this singularity. An additional advantage of the QZ algorithm is that we can filter out these 'spurious' eigenvalues by checking the β_i . Specifically, we remove any that are zero.

We now have several numerical methods implemented that can be used to assess the stability of flows for various values of R . The focus of the next chapter will be on the results that we obtain along with a comparison of the numerical accuracy of these methods.

4.9 Chebyshev Tau method

It's also worth mentioning why this is referred to as the Chebyshev Tau method. This is because this method doesn't exactly solve the Orr-Sommerfeld equation, but exactly solves the Orr-Sommerfeld equation with the following additional terms on the right-hand side (in the fourth order case)

$$\tau_1 T_{M-3}(z) + \tau_2 T_{M-2}(z) + \tau_3 T_{M-1}(z) + \tau_4 T_M(z).$$

These τ_i can be used to get error bounds if this method is used to get a solution to a differential equation (Fox and Parker, 1968), but cannot be used in this way if used to solve for eigenvalues as we are doing here.

In the case of the second-order and first-order methods, they are actually solving their systems of equations with two tau coefficients on the right-hand side in the second-order case and one tau coefficient on the left-hand side in the first-order case. Since the approximation converges Orszag (1971), we have a strong implication that the higher-order methods are more accurate. Part of the next chapter will be verifying that this is the case.

The tau method was first derived by Lanczos (1988).

Chapter 5

Results

Now that we have a few methods for getting the eigenvalues of the Orr-Sommerfeld equation, we can move on to calculating some results for stability. This chapter begins by showing the basic spectra we get for both Poiseuille and Couette flow, before moving on to showing the differences between each of our 3 methods. We will conclude by showing that this method can be used to easily compute the eigenfunction of a flow, and by computing a numerical bound on R to compare to the analytic bounds found by Synge and Joseph that we derived back in Chapter 3. From this bound, we get a value for the famous critical Reynolds number for plane Poiseuille flow.

Our method is implemented in the class Appendix A.2, with the matrices for each of our methods implemented in Appendix A.7, Appendix A.8 and Appendix A.9. The code for the differentiation matrices and P and Q matrices for each flow is implemented in Appendix A.5.

To create the spectra in the chapter we take the eigenvalues returned by our program and bound them using Josephs bounds on c_r and c_i derived in Chapter 3. As there is no lower bound on c_i , we choose the lower bound of $c_i = -1$ as is the most common convention in other literature. All of our spectra contain eigenvalues with imaginary parts below -1 , but any solutions related to these decay very fast and are therefore very stable. We are also much more interested in eigenvalues with imaginary parts close to zero so it makes little sense to include eigenvalues with large negative imaginary parts.

Note: Unless mentioned otherwise, the programs in this chapter were run with $\alpha = 1$.

5.1 Basic results - Poiseuille Flow

To plot a spectrum of eigenvalues, we will plot the real part c_r along the x -axis and the imaginary part c_i along the y -axis. This produces a two-dimensional spectrum for us to analyse.

To begin let's examine how these methods can be used to determine whether a flow is stable or not. If a flow is unstable it will have an eigenvalue c that has imaginary part $c_i > 0$. Figure 5.1 shows the spectra for $R = 1000$ and $R = 7500$ produced using the second-order method with 200 polynomials. We can see from these that Poiseuille flow is stable for $R = 1000$ but is unstable for $R = 7500$.

The spectrum for $R = 10000$ and $M = 200$ can be seen in Figure 5.2 and forms a 'Y' shape. This is a classic case when it comes to Poiseuille flow and this spectrum matches those seen in papers such as Dongarra et al. (1996a); Bukhari (1997); Antar (1976). We can see that there are 3 distinct 'families' of eigenvalues. The bottom straight line is referred to as the S family and contains a pattern of alternating odd and even eigenvalues (Dongarra et al., 1996a), the

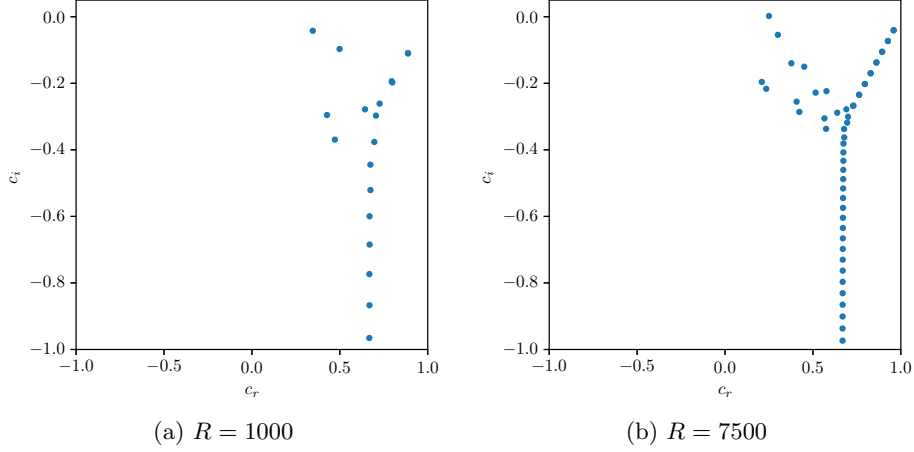


Figure 5.1: Eigenvalue spectra for a stable Poiseuille flow ($R = 1000$) and an unstable flow ($R = 7500$)

top right branch is referred to as the P family and consists of overlapping pairs of odd and even eigenvalues (Dongarra et al., 1996a) and the 'spray' of eigenvalues forming the left branch is referred to as the A family of eigenvalues. These names come from Mack (1976).

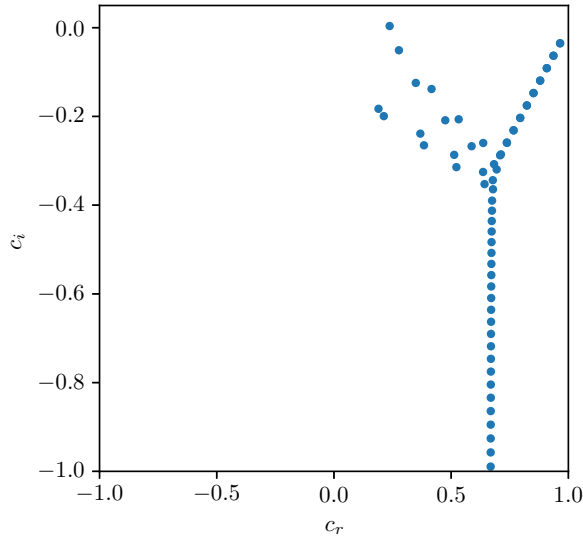


Figure 5.2: Eigenvalue spectra for plane Poiseuille flow with $R = 10000$

We can see that Poiseuille flow is unstable at $R = 10000$ because there is one 'critical eigenvalue' that has an imaginary part just slightly above 0. This is consistent with what other papers have found, most notably Orszag (1971). We will discuss how accurately the different methods reproduce the value of this critical eigenvalue in Section 5.4.

Finally, we should verify that all three of our methods produce similar results. Figure 5.3 shows a comparison between all three methods for Poiseuille flow at $\alpha = 1$, $R = 10000$ with 200 polynomials ($M = 200$). As can be seen, the spectra all look similar, showing that the methods appear to be consistent. However, there are significant differences between these methods which can become more pronounced at higher and lower values of R and M . A more thorough comparison is the focus of Section 5.4.

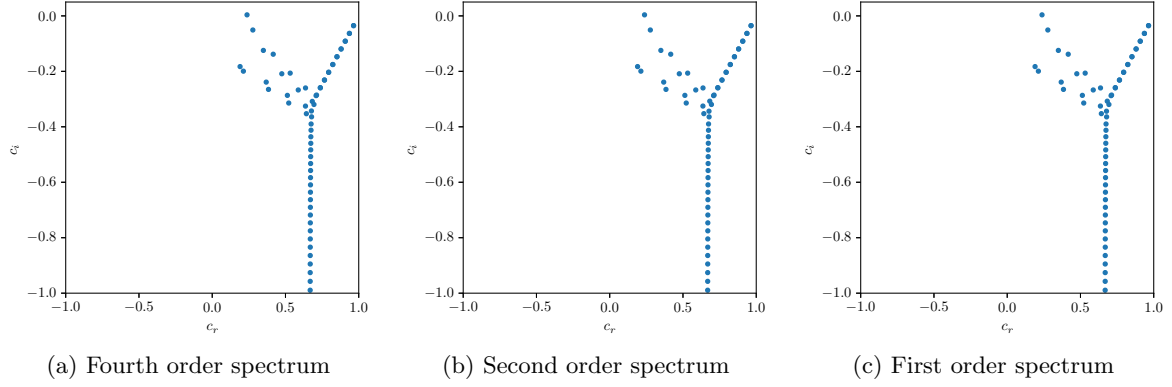


Figure 5.3: The spectrum of Poiseuille flow for $\alpha = 1$, $R = 10000$ using different orders of method. They are consistent with each other which implies that these methods all work well for this value of R .

5.2 Basic results - Couette Flow

If we run the second order method for Couette flow with $R = 8000$, $M = 200$ we get the spectrum shown in Figure 5.4. However there is an obvious discrepancy in the spectrum when compared to the one in papers such as Bukhari (1997), the spectrum is not symmetric.

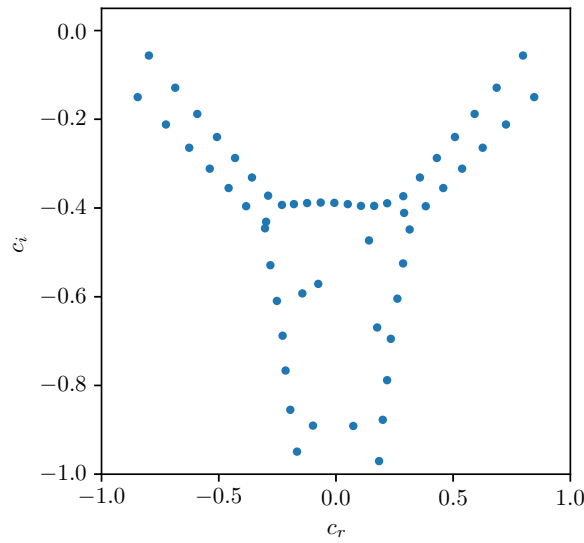


Figure 5.4: Spectrum of Couette flow with $R = 8000$. There are some interesting discrepancies in this spectrum

It turns out this is due to the boundary conditions we have used causing numerical error. As noted in Canuto et al. (2007) in the case of plane channel flow, the odd and even terms in the sums representing our boundary conditions can be decoupled. This leaves us with an alternative set of boundary conditions

$$\sum_{\substack{k=0 \\ k \equiv 0 \pmod{2}}}^M a_k = \sum_{\substack{k=1 \\ k \equiv 1 \pmod{2}}}^M a_k = \sum_{\substack{k=0 \\ k \equiv 0 \pmod{2}}}^M k^2 a_k = \sum_{\substack{k=1 \\ k \equiv 1 \pmod{2}}}^M k^2 a_k = 0.$$

The first two conditions here correspond to the boundary conditions on $\phi(z)$, while the second two correspond to the boundary conditions on $\phi'(z)$. Using these alternative boundary conditions

again with $R = 8000$, $M = 200$ produces the spectrum seen in Figure 5.5. As can be seen, the result is now symmetric which fits the spectra seen in other literature such as Bukhari (1997); Dongarra et al. (1996b).

The reason why these boundary conditions fix the spectrum is a little unclear. The most likely is due to the very large range in values in the matrices caused by the $(-1)^k k^2$ in the original boundary conditions causing additional numerical error.

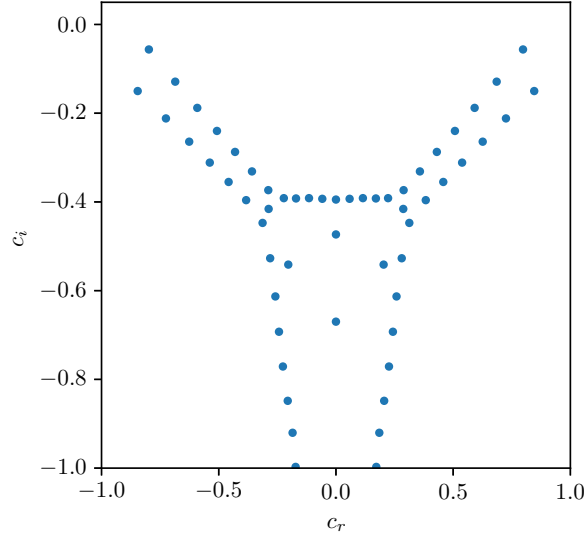


Figure 5.5: Spectrum for Couette flow with $R = 8000$ with alternative boundary conditions. The discrepancies seen in Figure 5.4 are not present here

If we look at Figure 5.5, we can see that Couette flow appears to be stable for this value of R . If we test other values of R too, we find that Couette flow is stable for all values of R . This matches the findings of Bukhari (1997) and other literature.

It turns out that these methods only work for very low R in the case of Couette flow. As seen in Dongarra et al. (1996a), all Couette flow spectra should look similar to the spectrum shown in Figure 5.6a, with a single straight tail at around $c_r = 0$ that splits two symmetric sprays of eigenvalues as $c_i \rightarrow 0$. Dongarra et al. (1996b) observed divergence from this spectrum as early as $R = 3500$, which our program reproduces as seen Figure 5.6b.

A comparison between our three methods for Couette flow with $R = 3000$ with $M = 200$ is shown in Figure 5.7. As can be seen, unlike in the case of Poiseuille flow, the spectra produced vary significantly. The first order method looks the most accurate to the plot of $R = 3000$ in Dongarra et al. (1996b) as expected, with the second order method looking extremely similar. On the other hand, the fourth-order method shows a significant divergence from the expected spectrum. This implies that the fourth-order method is not very good in the case of Couette flow.

Dongarra et al. (1996b) was able to get an accurate spectrum up until $R = 13000$ by using 128-bit floating point arithmetic rather than the 64-bit arithmetic that our program uses. We can also reduce the divergence by tweaking the number of polynomials used. We will discuss this further in the next section where we compare the three different methods and their accuracy.

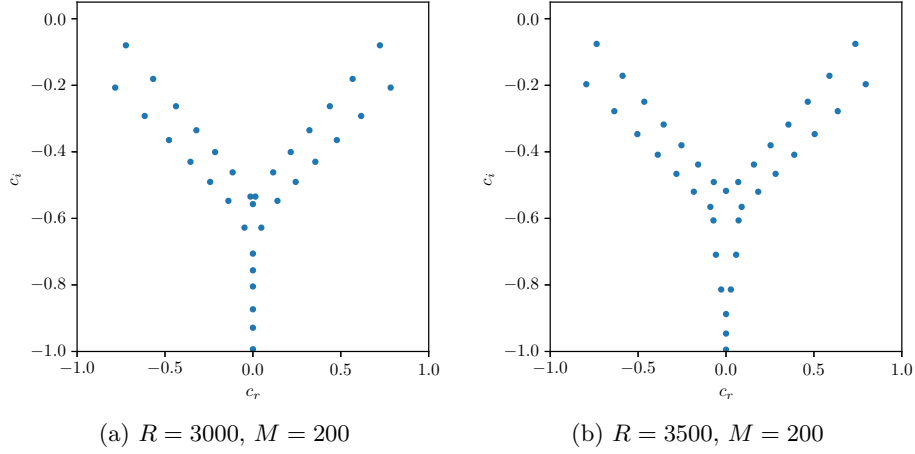


Figure 5.6: The spectrum of Couette flow with different values of R . The left plot shows an expected plot, while the right plot shows a divergence from the expected spectrum.

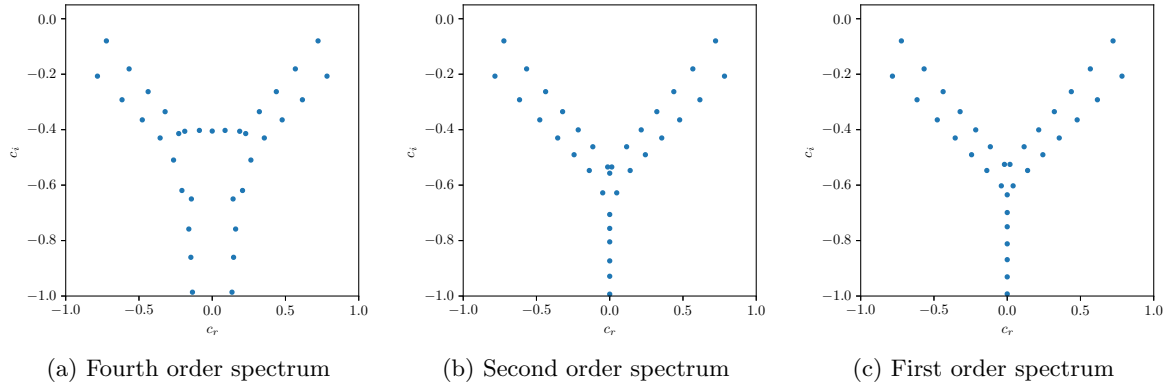


Figure 5.7: The spectrum of Couette flow for $R = 3000$ using different orders of method. The fourth-order plot is significantly more diverged than the others, while the first-order method appears to be the most accurate.

5.3 Time Comparison of Methods

Now that we have verified the basic results of our program for both Poiseuille and Couette flow, we can move on to discussing how the 3 different methods differ. There are advantages and disadvantages to each, so is there an obviously best choice?

To start with let's compare perhaps the most obvious difference that we notice when running these programs, the difference in computation time. As mentioned in Chapter 4, each method requires larger and larger matrices, and therefore more operations to both set up and solve. The Python program in Appendix A.11 calculates the time taken for each method for different values of M . Figure 5.8 shows the results of this program, a time comparison between the fourth-order method (green line), second-order method (orange line) and first-order method (blue line) for various values of M .

As we can see there is a significant difference in the time taken for each of the methods, particularly between the first-order method and the other two. Even using just $M = 200$ the first order method is significantly slower than the other two - taking over 2 minutes to run at $M = 500$! This graph also highlights the main advantage of the fourth-order method, its speed. Notice also that while slower than the fourth order method, the second order method is still

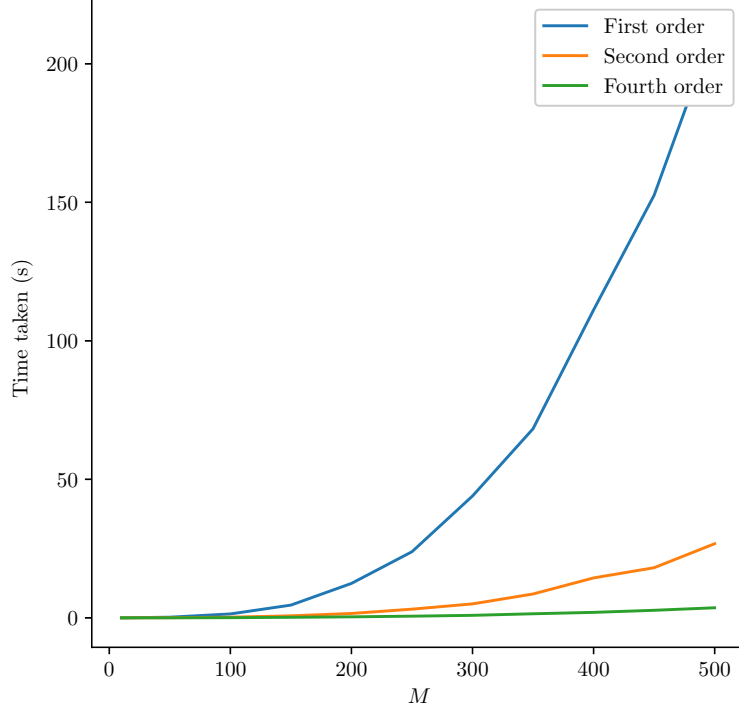


Figure 5.8: A comparison of times for the methods of different orders.

much faster than the first order method, taking less than 20 seconds at $M = 500$.

So if the fourth-order method is that much faster than the other two methods, why bother using the others at all? The answer is due to the amount of accuracy that is lost at higher values of M or even R . As we will see, each method of decreasing order is more accurate than the one before. But how does this inaccuracy present itself in the cases of Poiseuille and Couette flow?

5.4 Accuracy - Poiseuille Flow

To start with let's examine these methods in the case of Poiseuille flow. Figure 5.9 shows eigenvalue spectra for the second order method run for $R = 20000$ with $M = 150$, 200, and 1000. The $M = 200$ plot looks as we would expect, but we see some interesting features in the $M = 150$ and $M = 1000$ plot.

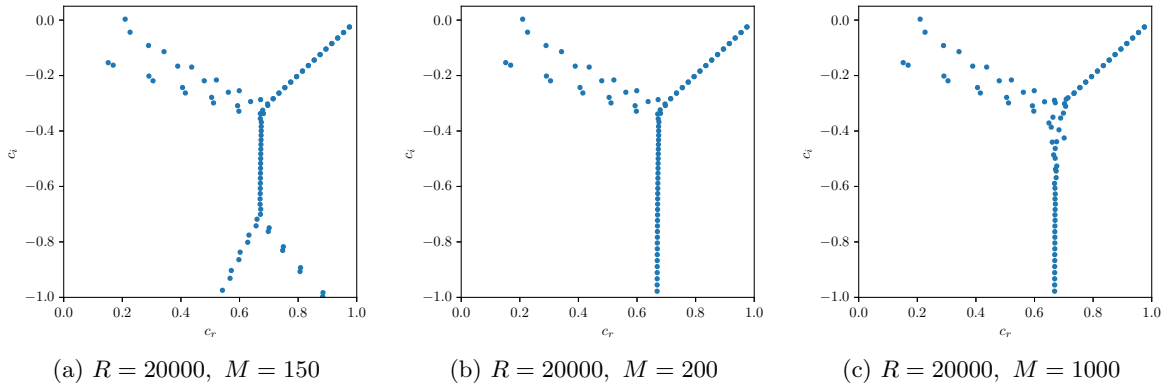


Figure 5.9: Eigenvalue spectra for various values of M with $R = 20000$

In the $M = 150$ plot we can see a 'split' in the S family of eigenvalues. This tail is caused by using a number of polynomials that is too low to accurately determine eigenvalues with a lower imaginary part (Dongarra et al., 1996a). In fact, if we extended the graph down on the $M = 200$ and $M = 1000$ plots we would see a similar split. The reason that this isn't a concern is that we are concerned with the stability of the solution, and therefore care more about eigenvalues with imaginary parts that are close to 0. The modes with eigenvalues with imaginary parts much below 0 decay extremely fast and therefore are very stable.

If we look at the $M = 1000$ plot we can see a small triangle in the centre of the spectra, where the 3 branches meet. This is sometimes referred to as the 'triangle of instability' and is an indication that numerical error is present in the spectra (Dongarra et al., 1996a). This occurs when either R or M is too high. We can use this to easily visualise the difference in numerical error between our different methods. Figure 5.10 shows the spectra for $R = 30000$ with $M = 500$ generated by the fourth-order, second order and first-order methods respectively.

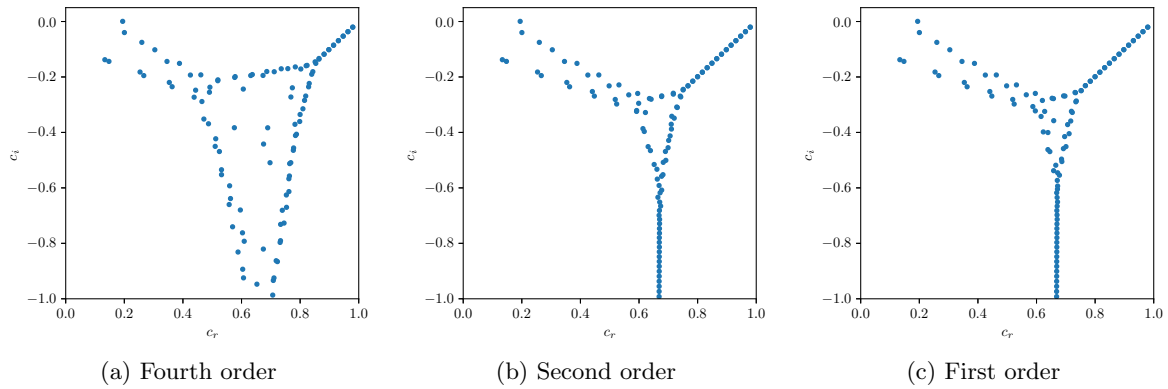


Figure 5.10: Eigenvalue spectra for $R = 30000$, $M = 500$ generated using methods of decreasing order (left to right)

As can be seen, the difference in numerical accuracy between the fourth-order method (left spectrum) and second-order method (centre spectrum) is massive, the triangle of instability heads off of the bottom of the screen in this example. However while noticeable, the plots imply that the numerical error between the second-order method (centre spectra) and the first-order method (right spectra) is far less significant. This is an extreme example however, it turns out that the triangle of numerical error can never be removed from the spectrum of $R = 30000$ no matter how high the number of polynomials used. A solution may be to use higher precision floating points, as Dongarra et al. (1996a) show in the $R = 28000$ case this can get rid of the triangle of instability altogether.

This along with Figure 5.9 imply two important things about these methods. First of all, the number of polynomials (value of M) chosen affects the accuracy of the produced spectra significantly. This must be remembered if the aim is to get an accurate representation of a specific portion of the spectra for a given value of R . Secondly, if the value of R is high enough there is no way of completely removing the numerical instability. This implies that for high values of R , this method is not very useful for determining the eigenvalue spectrum.

However, there is another more important way that numerical error can present itself. This is in the accuracy of the critical eigenvalue. This value must be computed accurately as it often has an imaginary part very close to zero and if computed incorrectly we could end up drawing the wrong conclusion about the stability of a flow.

Orszag (1971) computed the critical eigenvalue for Poiseuille flow with $R = 10000$, $\alpha = 1$ and

(to 8 decimal places) found a value of

$$0.23752649 + 0.00373967i.$$

Table 5.1 shows a side-by-side comparison of the critical eigenvalue produced by each method at different values of M for Poiseuille flow with $R = 10000$, $\alpha = 1$. We can see a few interesting things here. First, all three methods obtain the correct eigenvalue at around $M = 60$, though the first-order method produces the value accurately slightly earlier at around $M = 50$. Secondly, as M gets larger the fourth-order and second-order methods lose accuracy. The fourth-order method is only accurate to 5 decimal places by the time $M = 200$, whereas the second-order and first-order methods are still accurate to 8 decimal places. By the time $M = 500$, the fourth-order method is only accurate to 4 decimal places. At this point, the second-order method has lost some accuracy and is now only accurate to 7 decimal places, but the first-order method is still accurate to 8 decimal places. This implies that the numerical error in the first-order method has a magnitude of less than 10^{-8} at $M = 500$. This shows that the lower-order methods are indeed more accurate, as is expected with the tau method.

M	Fourth-order	Second-order	First-order
10	0.74378020+0.63468457i	0.94669728+0.38207669i	0.94669728+0.38207669i
20	0.73281177+0.0921027i	0.64429837+0.06460278i	0.64429837+0.06460278i
30	0.23690887+0.00365516i	0.23757258+0.00374383i	0.23747672+0.00364734i
40	0.23752676+0.00373427i	0.23752741+0.00374191i	0.23752761+0.00373901i
50	0.23752651+0.00373968i	0.23752648+0.00373967i	0.23752649+0.00373967i
60	0.23752649+0.00373967i	0.23752649+0.00373967i	0.23752649+0.00373967i
⋮	⋮	⋮	⋮
200	0.23752588+0.00374036i	0.23752649+0.00373967i	0.23752649+0.00373967i
⋮	⋮	⋮	⋮
500	0.2374707+0.00378162i	0.23752646+0.00373950i	0.23752649+0.00373967i

Table 5.1: Comparison of critical eigenvalue produced by each method for $R = 10000$ for increasing values of M .

Let's examine the convergence of the first-order method more closely for values above $M = 50$. Table 5.2 contains the critical eigenvalues produced by the first-order method for M between 50 and 120. We can see that for $M > 70$, the first 11 decimals are the same. As the tau method is expected to converge as M increases, we can confidently say that this is a more accurate value for the critical eigenvalue for Poiseuille with $R = 10000$. Our estimate for the critical eigenvalue to 11 decimal places is

$$0.23752648882 + 0.00373967062i.$$

It is difficult to confirm that this is definitely the critical eigenvalue of Poiseuille flow with $R = 10000$ to 11 decimal places since in other literature, the authors only attempt to reproduce the 8 decimal place value found by Orszag (1971). Still, Bukhari (1997) found that error was lowest at around this range of M and this value does match the value given by Orszag (1971) to 8 decimal places. The author of this report believes that this is a more accurate eigenvalue than that found by Orszag (1971).

We can also see that there appears to be a 'sweet spot' for accurately determining the critical eigenvalue for all of our methods at around $M = 60$. This is interesting because this value of M is too low to obtain the whole spectrum correctly. This means there is a trade-off between accurately determining the critical eigenvalue for a flow compared to getting more accurate

M	Critical Eigenvalue - First-order
50	0.23752648798891834+0.0037396745437875040i
60	0.23752648882528551+0.0037396706118949263i
70	0.23752648882056760+0.0037396706231014325i
80	0.23752648882017588+0.0037396706228313510i
90	0.23752648882049590+0.0037396706231066680i
100	0.23752648882102453+0.0037396706230743916i
110	0.23752648882018287+0.0037396706236932730i
120	0.23752648882051130+0.0037396706231617796i

Table 5.2: Critical eigenvalues for the first-order method.

results for other eigenvalues. The more eigenvalues we try and determine, the less accurate our value for the critical eigenvalue is.

However, for high values of R the numerical error in the critical eigenvalue is too high and as such we get that after a certain point, all flows are computed to be stable. This is the case no matter the method. For example, when $R = 35000$, all of our methods return that the flow is stable. But, we know that flows above a certain critical value of R are unstable from our analytical bounds derived in Chapter 3. This means that since we've found that Poiseuille flow is unstable for $R = 10000$, it must also be unstable for $R = 35000$, indicating that our program is incorrect. This isn't a concern, however, as we can still search for the critical value of R as it must be below 10000, and our methods accurately determine the stability of flows for lower R values. We will compute this value in Section 5.7.

5.5 Accuracy - Couette Flow

Are the forms of error seen in Poiseuille also seen in the case of Couette flow? Figure 5.11 shows the spectra produced using the second-order method using different numbers of polynomials. We can see that in the low M plot (Figure 5.11a) we have a split in the tail of the spectrum similar to the split seen in Poiseuille flow at low polynomial counts. This split gets larger as M gets lower. In the high M case (Figure 5.11c) we can see that in the centre of the plot, the eigenvalues have spread out into a 'diamond' which is the same as the 'triangle of instability' that appears when the value of M is too high with Poiseuille flow. The centre plot does not have these errors, which implies that similar to Poiseuille flow, there is a best number of polynomials required to get an accurate spectrum for Couette flow.

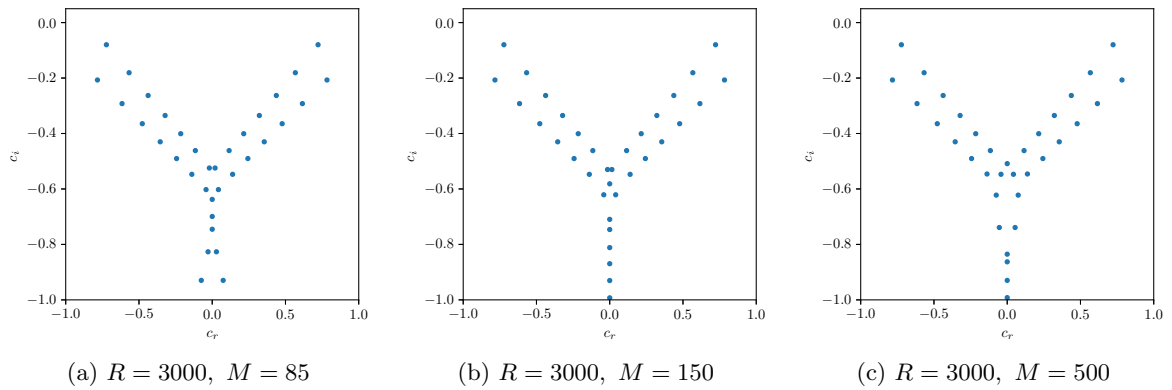


Figure 5.11: Eigenvalue spectra for various values of M with $R = 3000$

This eigenvalue method will always find that Couette flow is stable, which is the expected result (Dongarra et al., 1996b). This means that there is no 'critical eigenvalue' that we would particularly like to determine and test. We have also already seen in Figure 5.7 that there is a significant difference in the results given based on the order of method used. This indicates that despite its significantly slower computation time, the best method for Couette flow is the first-order method.

5.6 Eigenfunction - Poiseuille flow

One of the key advantages of using the QZ algorithm to solve our system is that we can get the eigenvectors of our system extremely easily, as they are calculated during the process of calculating the eigenvalues (Moler and Stewart, 1973). However, in the case of the first and second-order systems, only the first $M+1$ entries correspond to our function ϕ and the remaining entries correspond to various derivatives of ϕ . This means that our eigenfunction is found in this section of the eigenvector.

To get the eigenfunction from this eigenvector recall that the k th element corresponds to the coefficient of the k th order Chebyshev polynomial. Therefore to calculate our eigenfunction all we have to do is multiply each element by its corresponding Chebyshev polynomial.

Figure 5.12 shows the eigenfunction of the critical mode for plane Poiseuille flow with $R = 10000$, $M = 200$. In this plot, the eigenfunction has been normalised so that it takes the value $\phi = 1 + 0i$ at $z = 0$ as in Thomas (1953). The function is symmetric which is expected since the critical mode for plane Poiseuille flow is an even mode. The section of the graph with $z > 0$ also looks identical to the graph in Drazin and Reid (2004) of the eigenfunction computed by Thomas (1953), with one notable difference - the imaginary part is negative in Figure 5.12 whereas, in Drazin and Reid, it is positive. This is because Thomas computed his eigenvalue with a different sign. This also explains why his critical eigenvalue has a negative imaginary part rather than the more common positive imaginary part.

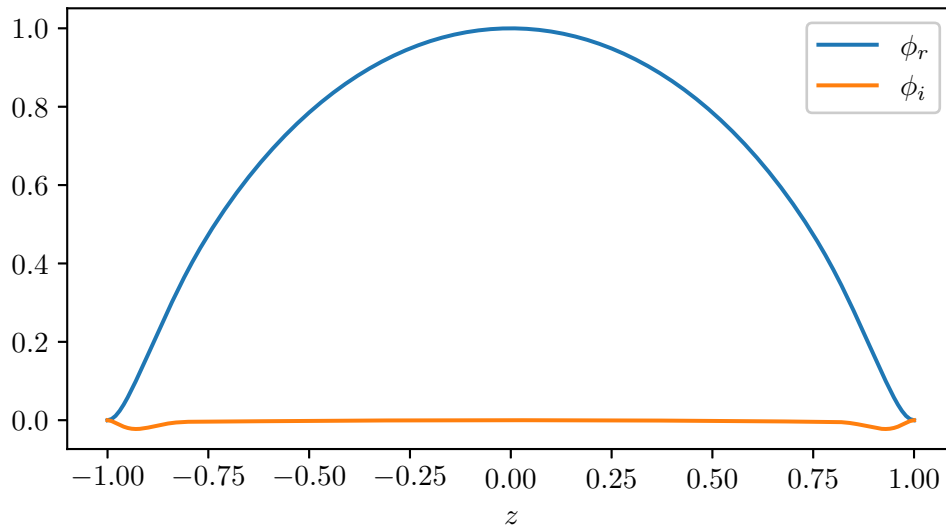


Figure 5.12: The eigenfunction for the critical mode for plane Poiseuille flow. ϕ_r is the real part and ϕ_i is the imaginary part.

We can also see that this eigenfunction satisfies the boundary conditions of the Orr-Sommerfeld equation, as it is zero at the boundaries. However, we should also check the derivative of the eigenfunction is zero at the boundaries so that all boundary conditions are satisfied. Using the

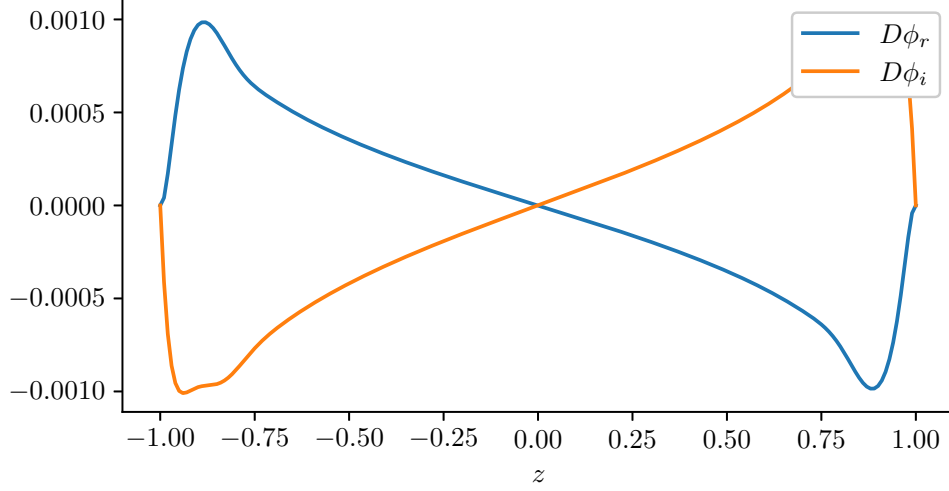


Figure 5.13: The un-normalized derivative of the eigenfunction for the critical mode of plane Poiseuille flow with $R = 10000$ and $M = 200$

first order method, all we need to do is to pick the part of the eigenvector corresponding to ϕ_2 as this is equivalent to $D\phi$. Figure 5.13 shows the plot of this (not normalised). As can be seen, it is zero at the endpoints, showing that all four boundary conditions have been satisfied!

We can take this eigenfunction a little further by computing $\psi'(z)$, the streamfunction solution we took in Chapter 2 (equation (18)) to get the Orr-Sommerfeld equation. Recall that

$$\psi'(x, z, t) = \phi(z) \exp\{i\alpha(x - ct)\},$$

where in this case, $\alpha = 1$ and c is our critical eigenvalue.

Figure 5.14 shows the real part of the streamfunction for the eigenfunction in Figure 5.12 plotted between $0 < x < 2\pi$. As we would expect, it is periodic over this range.

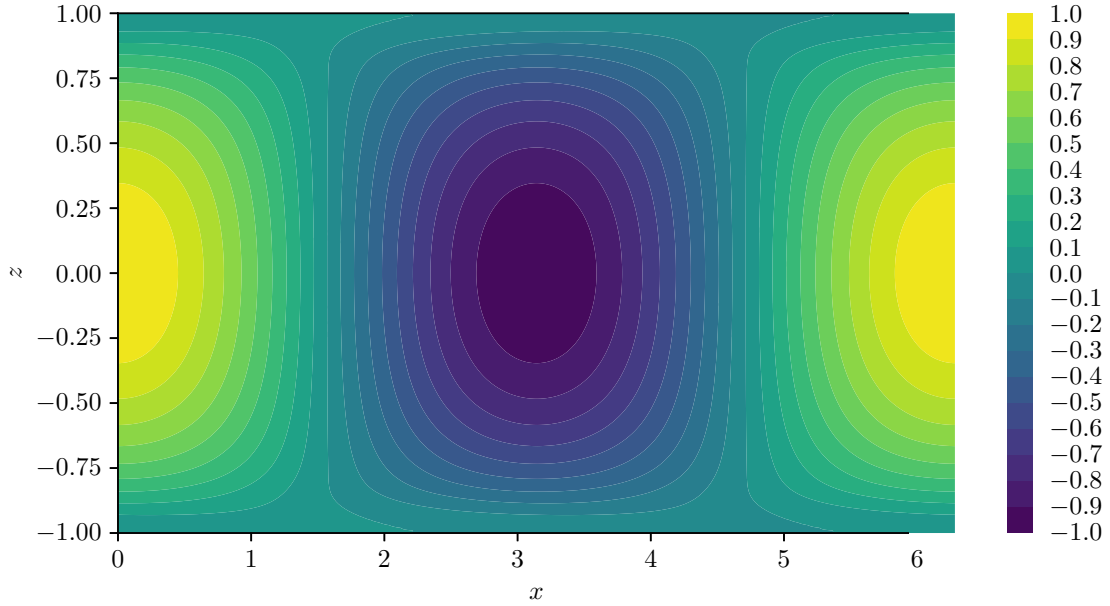


Figure 5.14: The real part of streamfunction for the critical mode of plane Poiseuille flow with $R = 10000$ and $M = 200$

Code for the eigenfunction plot can be found in Appendix A.12, and for the streamfunction plot can be found in Appendix A.13.

We can now move on to the most significant result this method can calculate, the critical Reynolds number for Poiseuille flow.

5.7 Critical Reynolds Number - Poiseuille flow

For a given α , we can compute the Reynolds number at which the corresponding mode becomes unstable (i.e. the imaginary part of the critical eigenvalue hits 0). We can use this to plot a curve of marginal stability which provides a bound on stability in the same way that the analytic results from Joseph (1968); Synge (1938) did. The Python program in Appendix A.14 calculates these values of R using a bisection technique. The result of this program is shown in Figure 5.15. Notice it takes a very similar shape to Joseph and Synge's bounds, with the unstable R value appearing to head to infinity as $\alpha \rightarrow 0$ and as $\alpha \rightarrow \infty$. However, the values of R found are much higher than those given by Joseph and Synge. This is likely because the analytic bounds were calculated by significant approximations, and are therefore extremely inaccurate compared to our numerical method.

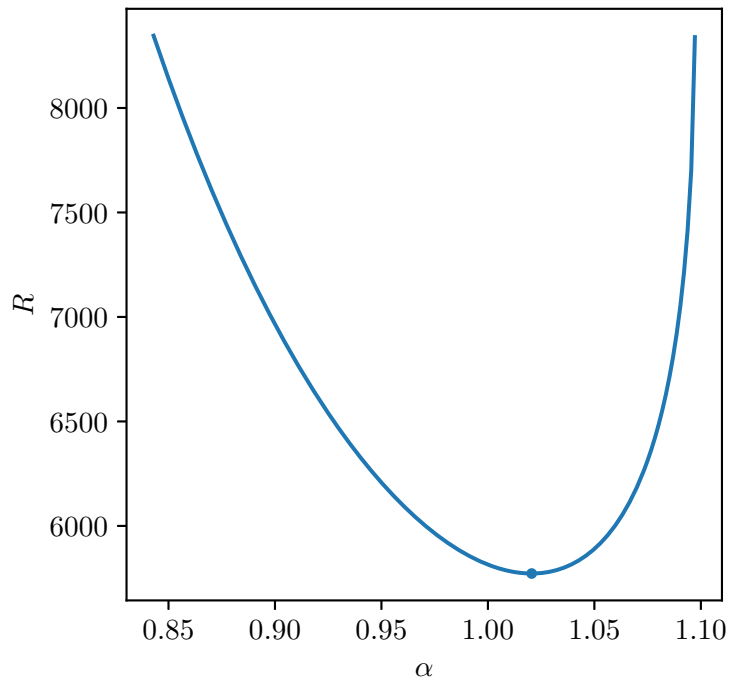


Figure 5.15: The marginal stability curve for Poiseuille flow. The dot indicates the point of critical R and α

We have a minimum point marked on the graph as a dot. This minimum corresponds to the famous *critical Reynolds number*, the Reynolds number above which Poiseuille flow is unstable. The program computes this by recursively searching for the smallest value near the minimum point. To 6 significant figures, the first order method with 50 polynomials finds

$$R = 5772.22, \text{ at } \alpha = 1.02055,$$

which matches the findings of Orszag (1971), who found that the first unstable mode of plane Poiseuille flow occurs at $R = 5772.2$ with $\alpha = 1.02056 \pm 0.00001$.

A value for the critical Reynolds number for Couette flow does exist, at about $R = 45000$ (Bukhari, 1997). Unfortunately, we cannot reproduce this value using this program as accuracy is lost so fast as R increases with Couette flow. Interestingly, Bukhari (1997) quotes a slightly lower value for plane Poiseuille flow, although this doesn't seem to be supported in other literature, and our program cannot reproduce this value either.

These results differ significantly from those predicted by our analytic bounds from Chapter 3. The critical Reynolds number computed numerically is much higher for Poiseuille flow than the value obtained analytically. Recall that Joseph's more accurate bound predicted a value of roughly 9! However, Joseph and Synge's bounds did expect that the critical Reynolds number would be found at a low wavenumber, which we have found to be the case.

Chapter 6

Conclusion

6.1 Summary of results

To conclude this report, let us summarise what has been shown. To begin, we introduced two classic types of plane flow, plane Poiseuille flow and plane Couette flow. We then used the governing equations for a plane channel flow to derive an important equation for linear stability, the Orr-Sommerfeld equation. The important parameters of the Orr-Sommerfeld equation were the Reynolds number R , the wavenumber α and its complex eigenvalue c . From examining the Orr-Sommerfeld equation, we determined a flow must be unstable if it has an eigenvalue with an imaginary part greater than 0.

From here we derived a few analytic bounds on the stability of a flow, the bounds first derived by Synge (1938) and then the improved bounds derived by Joseph (1968, 1969). When compared, Joseph's bounds had a higher minimum point (critical R value) than Synge's bounds. From this, we can conclude that a more accurate bound on stability would likely have a larger critical R value since these bounds were extremely approximate. To get a better bound on stability and to enable us to test whether a flow is stable for a specific value of R , we turned to a numerical method based on Chebyshev polynomials.

We showed that not only do Chebyshev polynomials accurately approximate functions, but they also have several extremely useful recursive relations for their derivatives. These relations allowed us to represent the differentiation of a function by multiplying a vector of coefficients by a differentiation matrix. We also needed a way to represent the basic flow of plane Poiseuille and plane Couette flow, which was done by the P and Q matrices of Bukhari (1997). We then showed that we could form a generalised eigenvalue problem by representing the whole fourth-order Orr-Sommerfeld equation using one matrix. However, as the matrix involved had terms that grew extremely fast we discussed two alternative methods. By splitting the Orr-Sommerfeld equation into two second-order differential equations or even four first-order differential equations, represented in a block matrix form. These two additional methods had matrices with slower-growing elements, and therefore higher numerical accuracy. To solve our eigenvalue problem, we implemented these methods in Python and used the implementation of the QZ algorithm of Moler and Stewart (1973) to get our system's eigenvalues.

When applied to Poiseuille flow, all of the methods were able to produce consistent spectra in classic cases with $R < 10000$. From these spectra, we were able to determine that for low values of R , Poiseuille flow is stable, whereas it is unstable for R larger than a certain critical value. Both the first and second-order methods were able to reproduce the value for the critical eigenvalue as calculated by Orszag (1971), however, the fourth-order method was unable to produce this value at any point.

When applied to Couette flow, we observed a large difference in the spectrum produced by each method, with the fourth-order method diverging particularly far from the expected spectrum. Each method was also significantly more sensitive to changes in the polynomial count.

In both cases, when R or M became large, a triangle (or diamond) of instability formed, showing that numerical error was present. While it was a smaller triangle for more accurate methods, for higher values of R there was no value of M for which the triangle disappeared. This indicates that these methods do not work very well for high R , i.e. less viscous fluids. We also observed that if the value of M was too low, the eigenvalues lower down the spectrum (i.e. with imaginary parts much below zero) would split away from the straight branch expected. However, for determining the critical eigenvalue of a flow it was better to use a lower, but not too low value of M , to not lose accuracy to numerical error.

We then showed that our method can be used to calculate the eigenfunction for a flow. To verify this, we compared it with the computed eigenfunction for the critical mode of $R = 10000$, $\alpha = 1$ of Poiseuille flow by Thomas (1953). We verified that this eigenfunction does satisfy our boundary conditions, showing that it does satisfy both the Orr-Sommerfeld equation and its boundary conditions. We were also able to use this eigenfunction to get a streamfunction for this mode.

The final result of the report was a bound on stability computed using a bisection method. We were able to get that the critical Reynolds number for plane Poiseuille flow is $R = 5772.22$ which matches up with the value found by Orszag (1971). While a critical value of R for Couette flow has been mentioned in some literature such as Bukhari (1997), this could not be reproduced by our methods. Our method concluded that Couette flow is always stable which is the accepted norm as mentioned in Trefethen et al. (1993).

6.2 Application of Results

How do these results relate to flows in the real world? The main conclusion we can draw is that for both Poiseuille and Couette flow, there appears to be a Reynolds number below which the flow is completely stable. We can use this to suggest possible ways in which a flow could be stabilised. For example, let's say we have Poiseuille flow in pipe flowing at a rate of 1 metre per second. Distilled water has a kinematic viscosity of about $1.0038e - 006$ metres squared per second at 20 degrees Celsius (from this table [here](#)). For this flow to have a Reynolds number below the critical value of 5772.22 we must have a characteristic length of below 0.00577222 metres or 5.77222 millimetres. If we recall the definition of the characteristic length in Chapter 2 then we recall that this is half of the width of our channel i.e. its radius, implying that water flowing at a velocity of 1 metre per second at 20 degrees Celsius through a pipe with a radius of less than 5.77222 mm would be a stable Poiseuille flow.

If however, we were considering a flow of kerosene at the same speed and temperature, we would be able to pass it stably through a pipe with a radius of roughly 15.6 mm. This is because kerosene has a higher kinematic viscosity of around $2.71e - 006$ metres squared per second.

Our method has shown that flows with a low Reynolds number are more stable than those with a high Reynolds number. We can therefore conclude that our three variables (characteristic velocity, characteristic length and kinematic viscosity) have the following effects on stability

1. High-velocity flows are less stable than low-velocity flows,
2. Flows travelling through a wide channel are less stable than flows travelling through a thin channel,
3. Flows of high viscosity fluid are more stable than flows of low viscosity fluids.

6.3 Further Research

The numerical method in this report is the Tau method of Lanczos (1988). However, this was not the only method that we could have used. Another method is the Chebyshev collocation method. In this method, the points in both the differentiation matrices and the vectors are points in physical space (Canuto et al., 2007). While this method may seem more intuitive and therefore preferable, Bukhari (1997) notes that this method cannot be used to represent diverging series, and can be more susceptible to round-off error if the problem is not well-posed. A proper comparison of these methods for the case of Poiseuille and Couette flow would have been interesting and would be a good next step for this project if it were to continue.

This numerical method can also be used to analyse more complex flows, such as a combination of Poiseuille and Couette flow and multi-layered analysis. In Bukhari (1997), these methods are used to solve problems in convection in magnetic and multi-layered cases. In Antar (1976), the method is applied to another classic case, the Blasius boundary layer flow. Most flow profiles that satisfy the governing equations for plane channel flow can be used, as all that needs to be changed are the P and Q matrices.

Finally, a discussion of the significance of the critical Reynolds number is in order. As pointed out in Trefethen et al. (1993), experimentally instability is seen at much lower Reynolds numbers than the critical Reynolds number would suggest. Both Couette and Poiseuille flow show instability at R values in the hundreds in experiments. When considering a culprit for this inaccuracy, our first thought would be that these bounds were on linear stability, implying that maybe the non-linear terms are to blame for these instabilities. However, Trefethen et al. (1993) points out that these are non-modal instabilities and therefore the issue lies with the normal mode analysis and eigenvalue method itself.

Appendix A

Code

These appendices contain all the code used to generate the figures and results seen throughout this project. Unless stated in the comments of the code, this code is the author's own work. Readers interested in running the code themselves can find it in the following [GitHub repository](#).

The code listed is not necessarily in the order it was used in the text. A recommended starting point is to run the `tau_tester.py` program in Appendix A.10 which will allow you to test all three methods of different orders and will also return the critical eigenvalue for a given flow.

A.1 .eps Saver

This is just a simple program used to save the plots generated as .eps files. A lot of the programs use this code to display the results, and as such anyone wishing to run these needs to have L^AT_EX installed on their machine.

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Wed Feb 8 20:00:02 2023
4
5  @author: oscar
6  """
7
8  import matplotlib
9  import matplotlib.pyplot as plt
10
11  ## code from https://stackoverflow.com/questions/67512225/saving-matplotlib-
12  ↪ graphs-with-latex-fonts-as-eps
13  ## turned into a class for general use
14  class saver():
15      def __init__(self, fig_x, fig_y, font_size=11):
16          '''
17          Parameters
18          -----
19          fig_x : float
20              Size of the figure - x dimension.
21          fig_y : float
22              Size of the figure - y dimension.
23          font_size : float, optional
```

```

24         Font size of the figure. The default is 11.
25
26     Returns
27     -----
28     None.
29
30     '''
31
32     ## may not be necessary - uncomment if issues with outputted plot
33     ## matplotlib.use('QtAgg')
34
35     ## reset defaults
36     plt.rcParamsdefaults()
37
38     ## Set up LaTeX fonts
39     plt.rcParams.update({
40         "text.usetex": True,
41         "font.family": "serif",
42         "font.serif": ["Computer Modern Roman"],
43         "font.size": font_size,
44     })
45
46     self.fig, self.ax = plt.subplots(figsize=(fig_x, fig_y))
47
48 def plot(self, x_data, y_data, **kwargs):
49     '''
50
51     plot data on the figure
52
53     Parameters
54     -----
55     x_data : array
56         A 1D array of the x-coordinates.
57     y_data : 2D array
58         A 1D array of the y-data. Must be same size as x_data.
59     **kwargs : args
60         args to be applied to the plotted data e.g. linestyle, marker,
61         ↪ linewidth.
62         Must be in the form arg=argvalue.
63
64     Returns
65     -----
66     None.
67
68     '''
69     self.ax.plot(x_data, y_data, **kwargs)
70
71 def contourf(self, x, z, vals, **kwargs):
72     self.contour = self.ax.contourf(x, z, vals, **kwargs)
73

```

```

74
75 def colorbar(self, levels):
76     self.fig.colorbar(self.contour, ticks=levels)
77
78 def set_args(self, **kwargs):
79     '''
80
81     set the general args for the plot
82
83     Parameters
84     -----
85     **kwargs : args
86         args to be applied to the whole plot e.g. xlim, ylim.
87         Must be in the form arg=argvalue.
88
89     Returns
90     -----
91     None.
92
93     '''
94     self.ax.set(**kwargs)
95
96 def legend(self):
97     '''
98     Enables legend
99
100    '''
101
102    self.ax.legend(loc='upper right')
103
104
105 def saveeps(self, file_name):
106     '''
107
108     Saves the figure as a .eps file
109
110     Parameters
111     -----
112     file_name : string
113         name of saved file.
114
115     Returns
116     -----
117     None.
118
119     '''
120
121     self.fig.savefig(file_name + ".eps", format="eps", dpi=1200,
        ↪ bbox_inches="tight", transparent=True)

```

A.2 Chebyshev Python Class

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Thu Nov 17 20:16:17 2022
4
5  @author: oscar
6  """
7  import numpy as np
8  import matplotlib.pyplot as plt
9  '''
10     A class that implements a Chebyshev polynomial
11     with functions to obtain its important properties
12  '''
13  class ChebyshevK():
14      def __init__(self, k, N):
15          '''
16              Initialise a Chebyshev polynomial of order k, with N points
17
18              Parameters
19              -----
20              k : INT
21                  The order of the Chebyshev polynomial
22              N : INT
23                  The number of points to initialise with
24
25              Returns
26              -----
27              None.
28
29              '''
30
31          self.k = k
32          self.N = N
33
34          self.set_points()
35          self.calculate_vals()
36
37      def evaluate(self, z):
38          '''
39              Evaluate the Chebyshev polynomial at a specific z value
40
41              Parameters
42              -----
43              z : FLOAT
44                  The z value to evaluate the polynomial at
45
46              Returns
47              -----
48              val : FLOAT
49                  The value of the polynomial at this point
50          '''
```

```

51         '''
52
53         theta = np.arccos(z)
54
55         val = np.cos(self.k * theta)
56
57         return val
58
59     def calculate_vals(self):
60         '''
61         Calculate the values the polynomial takes at its points
62
63         Returns
64         -----
65         None.
66
67         '''
68         vals = np.zeros(self.N)
69         for i in range(0, self.N):
70             z = self.points[i]
71             vals [i] = self.evaluate(z)
72
73         self.vals = vals
74
75     def get_vals(self):
76         '''
77         Get a list of all values this polynomial takes at its points
78
79         Returns
80         -----
81         List of floats
82             The pre calculated values
83
84         '''
85         return np.array(self.vals)
86
87     def set_points(self):
88         '''
89         Set the points to pre evaluate this polynomial at
90
91         Returns
92         -----
93         None.
94
95         '''
96         points = np.linspace(-1, 1, self.N)
97
98         self.points = points
99
100     def get_zeros_cosless(self):
101         '''

```



```

102     Get the zeros of this polynomial without applying the np.cos function
        ↪ to them
103
104     Returns
105     -----
106     zeros : LIST OF FLOATS
107         A list of the zeros of this polynomial
108
109     '''
110     zeros = np.zeros(self.k)
111
112     for i in range(0, self.k):
113         zeros[i] = (1 + 2*i) * np.pi / (2 * self.k)
114
115     return zeros
116
117 def gauss_lobatto(self):
118     '''
119     The Gauss-Lobatto points for this polynomial. Needed for the Chebyshev
        ↪ collocation method.
120
121     Returns
122     -----
123     LIST OF FLOATS
124         A list of the Gauss-Lobatto points of this polynomial
125
126     '''
127     zeros = np.pi * np.linspace(0, self.k, self.k, True) / self.k
128
129     return np.cos(zeros)
130
131 def plot(self):
132     '''
133     Plot the polynomial
134
135     Returns
136     -----
137     None.
138
139     '''
140     plt.plot(self.points, self.vals, '-', label=r'$T_{\text{' + str(self.k) + '}}(x)$'
        ↪ ')
141     plt.draw()

```

A.3 Chebyshev Plotter

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Sat Nov 19 20:09:00 2022
4
5  @author: oscar
6  """
7  import chebyshev as cheby
8  from matplotlib import pyplot as plt
9
10  '''
11     Plots a bunch of Chebyshev polynomials
12  '''
13  def main():
14
15     for i in range(0, 5):
16         Tk = cheby.ChebyshevK(i, 50)
17         Tk.plot()
18
19     plt.legend()
20
21     # the label may not work if you don't have latex installed
22     plt.set_args(xlim=[-1,1], ylim=[-1.05,1.05], xlabel=r'$x$')
23     plt.show()
24
25  main()
```

A.4 Chebyshev Function Approximator

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Sat Nov 19 19:59:44 2022
4
5  @author: oscar
6  """
7
8  import chebyshev as cheby
9  import numpy as np
10 from tau_matrices import D1, D2, D4
11 import epssaver as ep
12
13 def c_k(k):
14     if k < 0:
15         return 0
16     elif k == 0:
17         return 2
18     else:
19         return 1
20
21 def f(z):
22     # function to approximate
23
24     return np.cos(z)
25
26 def f_diff(z):
27     # first derivative of function
28     return -np.sin(z)
29
30 def f_diff2(z):
31     # second derivative of function
32     return -np.cos(z)
33
34 def f_diff4(z):
35     # fourth dericative of function
36     return np.cos(z)
37
38 def calculate_coeff(k, N, zeros):
39     # calculates coefficient for Chebyshev polynomial of specific order
40     f_zeros = np.array([f(zi) for zi in np.cos(zeros)])
41
42     elems = f_zeros * np.cos(k * zeros)
43
44     return (2 / (c_k(k) * (N + 1))) * np.sum(elems)
45
46 def get_coeffs(N):
47     # get the coefficients of the functions Chebyshev expansion
48     coeffs = np.zeros(N)
49
50     Tmax = cheby.ChebyshevK(N + 1, 32)
```

```

51     zeros = Tmax.get_zeros_cosless()
52
53     for i in range(0, N):
54         coeffs[i] = calculate_coeff(i, N, zeros)
55
56     return coeffs
57
58 def plot_comparison(points, func_vals, approx_vals, file_name):
59     # plot a comparison between actual function and the approximation
60     plt = ep.saver(3.75, 3.75)
61
62     plt.set_args(xlim=[-1, 1])
63     plt.set_args(ylim=[-1.02, 1.02])
64     plt.set_args(xlabel=r'$z$')
65
66     plt.plot(points, func_vals, linestyle='--', color='#ff7f0e')
67     plt.plot(points, approx_vals, linestyle='--', color='#1f77b4')
68
69     plt.saveeps(str(file_name))
70
71 def main():
72     order = 8
73     accuracy = 1000
74
75     D_1 = D1(order + 1)
76     D_2 = D2(order + 1)
77     D_4 = D4(order + 1)
78
79     # get the approximation coefficients
80     coeffs = get_coeffs(order + 1)
81
82     # work out the coefficients of the derivatives using D matrices
83     firstDiffCoeffs = np.matmul(D_1, coeffs)
84     secondDiffCoeffs = np.matmul(D_2, coeffs)
85     fourthDiffCoeffs = np.matmul(D_4, coeffs)
86
87     vals = np.zeros(accuracy)
88     firstDiffvals = np.zeros(accuracy)
89     secondDiffvals = np.zeros(accuracy)
90     fourthDiffvals = np.zeros(accuracy)
91
92     points = np.linspace(-1, 1, accuracy)
93     for i in range(0, order + 1):
94         Tk = cheby.ChebyshevK(i, accuracy)
95
96         tk_vals = Tk.get_vals()
97
98         vals = vals + coeffs[i] * tk_vals
99         firstDiffvals = firstDiffvals + firstDiffCoeffs[i] * tk_vals
100        secondDiffvals = secondDiffvals + secondDiffCoeffs[i] * tk_vals
101        fourthDiffvals = fourthDiffvals + fourthDiffCoeffs[i] * tk_vals

```

```

102
103     # plot approximation to main function
104     func_vals = np.array([f(point) for point in points])
105
106     plot_comparison(points, func_vals, vals, 'chebyapprox_order=' + str(order))
107
108     # plot approximation to first deriv
109     diff1_func_vals = np.array([f_diff(point) for point in points])
110
111     plot_comparison(points, diff1_func_vals, firstDiffvals, 'diff1approx_order=
    ↪ ' + str(order))
112
113     # plot approximation to second deriv
114     diff2_func_vals = np.array([f_diff2(point) for point in points])
115
116     plot_comparison(points, diff2_func_vals, secondDiffvals, 'diff2approx_order
    ↪ =' + str(order))
117
118     # plot approximation to fourth deriv
119     diff4_func_vals = np.array([f_diff4(point) for point in points])
120
121     plot_comparison(points, diff4_func_vals, fourthDiffvals, 'diff4approx_order
    ↪ =' + str(order))
122
123     main()

```

A.5 Tau Matrices

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Mar 13 11:20:18 2023
4
5  @author: oscar
6  """
7
8  import numpy as np
9
10 # matrices needed for the tau method
11
12 def c_k(k):
13     if k < 0:
14         return 0
15     elif k == 0:
16         return 2
17     else:
18         return 1
19
20 # first derivative matrix
21 def D1(M):
22     D1 = np.zeros([M, M])
23
24     for n in range(0, M):
25         for p in range(n + 1, M):
26             if (p + n) % 2 == 1:
27                 D1[n, p] = 2/c_k(n) * p
28
29     return D1
30
31 # second derivative matrix
32 def D2(M):
33     D2 = np.zeros([M, M])
34
35     for n in range(0, M):
36         for p in range(n + 2, M):
37             if (p + n) % 2 == 0:
38                 D2[n, p] = 1/c_k(n) * p * (p**2 - n**2)
39
40     return D2
41
42 # fourth derivative matrix
43 def D4(M):
44     D4 = np.zeros([M, M])
45
46     for n in range(0, M):
47         for p in range(n + 4, M):
48             if (p + n) % 2 == 0:
49                 q = p - n
50                 D4[n, p] = 1/(24 * c_k(n)) * p * (q**4 * (q**2 - 8) + n * q**3 *
```

```

51         ↪ (6 * q**2 - 32) + (12 * n**2 * q**2 + 8 * n**3 * q) * (q
52         ↪ **2 - 4) + 16 * q**2 + 32 * n * q)
53
54     return D4
55
56 # Orszag matrix for plane Poiseuille flow (NOT USED)
57 def y2_mat(N):
58     y2_mat = np.zeros([N, N])
59
60     for n in range(0, N):
61         if n - 2 >= 0:
62             y2_mat[n, n - 2] = c_k(n - 2)
63
64             y2_mat[n, n] = c_k(n) + c_k(n - 1)
65
66             if n + 2 < N:
67                 y2_mat[n, n + 2] = 1
68
69     return 1/4 * y2_mat
70
71 # Q as defined in Bukarri thesis
72 def Q_mat(M, flowtype="p"):
73     if flowtype == "p":
74         return Q_p(M)
75     elif flowtype == "c":
76         return Q_c(M)
77     else:
78         print("Invalid flow type entered, defaulting to Poiseuille flow")
79         return Q_p(M)
80
81 # for Poiseuille flow
82 def Q_p(M):
83     Q = np.zeros([M, M])
84
85     for i in range(0, M):
86         if i < M-2:
87             Q[i, i+2] = -1/4
88
89             if i > 1:
90                 Q[i, i-2] = -1/4
91
92             Q[i, i] = 1/2
93
94     Q[1, 1] = 1/4
95     Q[2, 0] = -1/2
96
97     return Q
98
99 # for Couette flow
100 def Q_c(M):
101     Q = np.zeros([M, M])

```

```

100
101     for i in range(0, M):
102         if i < M-1:
103             Q[i, i+1] = 1/2
104
105         if i > 0:
106             Q[i, i-1] = 1/2
107
108     Q[1, 0] = 1
109
110     return Q
111
112 # P as defined in Bukhari thesis
113 def P_mat(M, flowtype="p"):
114     if flowtype == "p":
115         return -2 * np.identity(M)
116     elif flowtype == "c":
117         return np.zeros([M, M]) + np.zeros([M, M]) * 1j
118     else:
119         print("Invalid flow type entered, defaulting to Poiseuille flow")
120         return -2 * np.identity(M)

```


A.6 Tau Method Class

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Mar 13 11:16:46 2023
4
5  @author: oscar
6  """
7
8  import numpy as np
9  from scipy.linalg import eig as sci_eig
10 from D1method import setup_matrices as D1_setup
11 from D2method import setup_matrices as D2_setup
12 from D4method import setup_matrices as D4_setup
13
14 '''
15     Implements the Chebyshev Tau method
16 '''
17 class ChebyshevTau():
18     def __init__(self, order, M, flowtype='p'):
19         '''
20         Initialise an instance of the Chebyshev Tau method with a specified
21             ↳ order for a specific flow
22
23         Parameters
24         -----
25         order : INT
26             The order of the method. 4 is fourth order, 2 is second order and 1
27             ↳ is first order
28
29         M : INT
30             The number of polynomials to use
31
32         flowtype : CHAR, optional
33             The type of flow, enter 'c' for Couette and 'p' for Poiseuille. The
34             ↳ default is 'p'.
35
36         Raises
37         -----
38         Exception
39             If an invalid order or flowtype is entered
40
41         Returns
42         -----
43         None.
44
45         '''
46
47         if not order in [1, 2, 4]:
48             raise Exception("Invalid order entered. Please enter either 1, 2 or
49                 ↳ 4")
50
51         if not flowtype in ['p', 'c']:
52             raise Exception("Invalid flowtype entered. Please enter either p or
```

```

    ↪ c")
47
48     self.order = order
49     self.M = M
50     self.flowtype = flowtype
51
52     self.eigs = np.zeros(0)
53     self.eig_vecs = np.zeros([M, 0]) + 1j * np.zeros([M, 0])
54
55 def get_eigvals_eigvecs(self, a, R, ci_lower=-1):
56     '''
57         Get the eigenvalues and eigenvectors of a flow with a specific a and
58         ↪ R
59
60     Parameters
61     -----
62     a : FLOAT
63         The wavenumber to solve for. Must be greater than 0
64     R : FLOAT
65         The Reynolds number of the flow.
66     ci_lower : FLOAT, optional
67         How far 'down' to truncate the eigvalues at. The default is -1.
68
69     Returns
70     -----
71     LIST OF FLOAT
72         Calculated eigenvalues
73     LIST OF LIST OF FLOATS
74         Calculated eigenvectors
75     '''
76
77     # setup matrices using entered parameters
78     lhs_matrix, rhs_matrix = self.setup_system(a, R)
79
80     # find the eigenvalues and eigenvectors using QZ algorithm
81     eigs_split, vecs = sci_eig(lhs_matrix, rhs_matrix, homogeneous_eigvals=
82         ↪ True)
83
84     # filter out relevant eigenvalues and their corresponding eigenvectors
85     self.eigs, self.eig_vecs = self.valid_eigs(eigs_split, vecs, a, R,
86         ↪ ci_lower)
87
88     return self.eigs, self.eig_vecs
89
90 def valid_eigs(self, eigs_split, eig_vecs, a, R, ci_lower):
91     '''
92         Uses the bounds provided by Joseph (1969) to get the valid
93         ↪ eigenvalues for the flow.
94         Also removes spurious eigenvalues returned by the QZ algorithm

```

```

93
94     Parameters
95     -----
96     eigs_split : LISTS OF FLOATS
97         Alpha and Beta returned by the QZ algorithm
98     eig_vecs : LIST OF LIST OF FLOATS
99         Eigenvectors returned by the QZ algorithm
100     a : FLOAT
101         Wavenumber of the flow
102     R : FLOAT
103         The Reynolds number of the flow.
104     ci_lower FLOAT:
105         Lower bound for c_i
106
107     Returns
108     -----
109     bound_eigs : LIST OF FLOATS
110         The eigenvalues within the bounds we are considering
111     bound_vecs : LIST OF LIST OF FLOATS
112         Corresponding eigenvectors to the bounded eigenvalues
113
114     '''
115     alpha = eigs_split[0]
116     beta = eigs_split[1]
117
118     cr_lower = 0
119     q = 0
120
121     # apply Joseph's bounds
122     if self.flowtype == 'p':
123         cr_lower = -4 / (np.pi**2 + a**2)
124         q = 2
125     elif self.flowtype == 'c':
126         cr_lower = -1
127         q = 1
128
129     ci_upper = q / 2*a - (a * R)**(-1) * ((np.pi**2 * (np.pi**2 + a**2))/ (
130         ↪ np.pi**2 + 4 * a**2) + a**2)
131
132     count = len(alpha)
133
134     bound_eigs = np.zeros(0)
135     bound_vecs = np.zeros([self.M, count]) + 1j * np.zeros([self.M, count])
136
137     vec_count = 0
138     for i in range(0, count):
139         if beta[i] != 0: # exclude spurious eigenvalues caused by QZ
140             ↪ algorithm
141             valid_eig = np.longcomplex(alpha[i]) / np.longcomplex(beta[i])
142
143             if valid_eig.real > cr_lower and valid_eig.real < 1:

```

```

142         if valid_eig.imag < ci_upper and valid_eig.imag > ci_lower:
143             bound_eigs = np.append(bound_eigs, valid_eig)
144             bound_vecs[vec_count] = eig_vecs[:, i]
145             vec_count += 1
146
147     return bound_eigs, bound_vecs[:vec_count]
148
149 def critical_eig(self):
150     '''
151         Get the critical eigenvalue for the system (the eigenvalue with the
152         ↪ greatest imaginary part)
153
154     Returns
155     -----
156     crit_eig : FLOAT
157         The critical eigenvalue of the system
158     '''
159
160     max_imag = 0
161     crit_eig = 0
162     for i in range(0, len(self.eigs)):
163         if self.eigs[i].imag > max_imag or i == 0:
164             max_imag = self.eigs[i].imag
165             crit_eig = self.eigs[i]
166
167     return crit_eig
168
169 def critical_vec(self):
170     '''
171         Get the eigenvector for the critical mode of the system
172
173     Returns
174     -----
175     crit_vec : LIST OF FLOATS
176         The critical eigenvector of the system
177     '''
178
179     max_imag = 0
180     crit_index = 0
181     for i in range(0, len(self.eigs)):
182         if self.eigs[i].imag > max_imag or i == 0:
183             max_imag = self.eigs[i].imag
184             crit_index = i
185
186     crit_vec = self.eig_vecs[crit_index, :self.M]
187
188     return crit_vec
189
190 def setup_system(self, a, R):
191     '''

```

```

192         Setup the matrix system using the correct order method
193
194     Parameters
195     -----
196     a : FLOAT
197         Wavenumber of the system
198     R : FLOAT
199         Reynolds number of the system
200
201     Returns
202     -----
203     lhs_matrix, rhs_matrix : MATRICES
204         Matrices making up the left and right hand sides of the generalised
205         ↪ eigenvalue problem for this system
206
207     '''
208     if self.order == 1:
209         return D1_setup(a, R, self.M, self.flowtype)
210     elif self.order == 2:
211         return D2_setup(a, R, self.M, self.flowtype)
212     elif self.order == 4:
213         return D4_setup(a, R, self.M, self.flowtype)

```

A.7 Fourth Order Method

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Tue Jan 3 15:58:51 2023
4
5  @author: oscar
6  """
7
8  import numpy as np
9  from tau_matrices import D2, D4, Q_mat, P_mat
10
11  def setup_matrices(a, R, M, flowtype):
12      '''
13          Sets up the matrices for the fourth order method
14
15          Parameters
16          -----
17          a : FLOAT
18              Wavenumber
19          R : FLOAT
20              Reynolds number
21          M : INT
22              Number of polynomials being used
23          flowtype : CHAR
24              'c' for Couette flow and 'p' for Poiseuille
25
26          Returns
27          -----
28          A : MATRIX
29              Left hand side matrix
30          B : MATRIX
31              Right hand side matrix
32
33      '''
34
35      D_2 = D2(M)
36      D_4 = D4(M)
37
38      P = P_mat(M, flowtype)
39      Q = Q_mat(M, flowtype)
40
41      I = np.identity(M)
42
43      A = (D_4 - 2 * a**2 * D_2 + a**4 * I) / (1j * a * R) - np.matmul(Q, D_2 - a
44          ↪ **2 * I) + P
45      B = -(D_2 - a**2 * I)
46
47      # apply boundary conditions
48      for k in range(0, M):
49          A[M - 4, k] = 0
50          A[M - 3, k] = 0
```

```

50     A[M - 2, k] = 0
51     A[M - 1, k] = 0
52
53     B[M - 4, k] = 0
54     B[M - 3, k] = 0
55     B[M - 2, k] = 0
56     B[M - 1, k] = 0
57
58     if k % 2 == 0:
59         A[M - 3, k] = 1
60         A[M - 1, k] = k**2
61     else:
62         A[M - 4, k] = 1
63         A[M - 2, k] = k**2
64
65     return A, B

```

A.8 Second Order Method

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Thu Jan 12 10:52:45 2023
4
5  @author: oscar
6  """
7
8  import numpy as np
9  from tau_matrices import D2, Q_mat, P_mat
10
11  def setup_matrices(a, R, M, flowtype):
12      '''
13          Sets up the matrices for the second order method
14
15          Parameters
16          -----
17          a : FLOAT
18              Wavenumber
19          R : FLOAT
20              Reynolds number
21          M : INT
22              Number of polynomials being used
23          flowtype : CHAR
24              'c' for Couette flow and 'p' for Poiseuille
25
26          Returns
27          -----
28          E : MATRIX
29              Left hand side matrix
30          F : MATRIX
31              Right hand side matrix
32
33      '''
34
35      N = 2*M
36
37      I = np.identity(M)
38
39      D_2 = D2(M)
40
41      P = P_mat(M, flowtype)
42
43      Q = Q_mat(M, flowtype)
44
45      E = np.zeros([N, N]) + 1j*np.zeros([N, N])
46      F = np.zeros([N, N]) + 1j*np.zeros([N, N])
47
48      E[:M, :M] = D_2
49      E[:M, M:] = -I
50      E[M:, :M] = (P + a**2 * Q) + a**3 * I / (1j * R)
```



```

51     E[M:, M:] = (D_2 - 2*a**2 * I) / (1j * a * R) - Q
52
53     F[M:, :M] = a**2 * I
54     F[M:, M:] = -I
55
56     # boundary conditions
57     E[M-2] = 0
58     F[M-2] = 0
59     for k in range(0, M):
60         if k % 2 == 0:
61             E[M-2, k] = 1
62
63     E[M-1] = 0
64     F[M-1] = 0
65     for k in range(0, M):
66         if k % 2 == 1:
67             E[M-1, k] = 1
68
69     E[N-2] = 0
70     F[N-2] = 0
71     for k in range(0, M):
72         if k % 2 == 0:
73             E[N-2, k] = k**2
74
75     E[N-1] = 0
76     F[N-1] = 0
77     for k in range(0, M):
78         if k % 2 == 1:
79             E[N-1, k] = k**2
80
81     return E, F

```

A.9 First Order Method

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Thu Jan 12 10:52:45 2023
4
5  @author: oscar
6  """
7
8  import numpy as np
9  from tau_matrices import D1, Q_mat, P_mat
10
11  def setup_matrices(a, R, M, flowtype):
12      '''
13          Sets up the matrices for the first order method
14
15          Parameters
16          -----
17          a : FLOAT
18              Wavenumber
19          R : FLOAT
20              Reynolds number
21          M : INT
22              Number of polynomials being used
23          flowtype : CHAR
24              'c' for Couette flow and 'p' for Poiseuille
25
26          Returns
27          -----
28          G : MATRIX
29              Left hand side matrix
30          H : MATRIX
31              Right hand side matrix
32
33      '''
34
35      N = 4*M
36
37      I = np.identity(M)
38
39      D_1 = D1(M)
40
41      P = P_mat(M, flowtype)
42
43      Q = Q_mat(M, flowtype)
44
45      G = np.zeros([N, N]) + 1j*np.zeros([N, N])
46      H = np.zeros([N, N]) + 1j*np.zeros([N, N])
47
48      G[:M, :M] = G[M:2*M, M:2*M] = G[2*M:3*M, 2*M:3*M] = D_1
49      G[3*M:, 3*M:] = D_1 / (1j * a * R)
50      G[:M, M:2*M] = G[M:2*M, 2*M:3*M] = G[2*M:3*M, 3*M:] = -I
```

```

51     G[3*M:, :M] = (P + a**2 * Q) + a**3 * I / (1j*R)
52     G[3*M:, 2*M:3*M] = -Q - 2 * a * I / (1j*R)
53
54     H[3*M:, :M] = a**2 * I
55     H[3*M:, 2*M:3*M] = -I
56
57     # boundary conditions
58     # z_1=0 on x=-1
59     G[M-1] = 0
60     H[M-1] = 0
61     for k in range(0, M):
62         if k % 2 == 0:
63             G[M-1, k] = 1
64
65     # z_1=0 on x=1
66     G[2*M-1] = 0
67     H[2*M-1] = 0
68     for k in range(0, M):
69         if k % 2 == 1:
70             G[2*M-1, k] = 1
71
72     # z_2 = 0 on x=-1
73     G[3*M - 1] = 0
74     H[3*M - 1] = 0
75     for k in range(0, M):
76         if k % 2 == 0:
77             G[3*M - 1, M + k] = 1
78
79     # z_2 = 0 on x=1
80     G[N-1] = 0
81     H[N-1] = 0
82     for k in range(0, M):
83         if k % 2 == 1:
84             G[N-1, M + k] = 1
85
86     return G, H

```

A.10 Tau Method Tester

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Thu Mar 16 16:15:04 2023
4
5  @author: oscar
6  """
7
8  from chebyshev_tau import ChebyshevTau
9  import epssaver
10 import numpy as np
11
12 # use this method to test the tau method for specific flows
13 # plots the spectrum and returns the critical eigenvalue
14 # along with whether the flow is stable or not
15
16 order = 1 # order of method: 4, 2 or 1
17 M = 140 # number of polynomials
18 a = 1 # wavenumber
19 R = 10000 # Reynolds number
20 flowtype = 'p' # 'c' or 'p'
21
22 tau = ChebyshevTau(order, M)
23
24 eigs, vecs = tau.get_eigvals_eigvecs(a, R)
25
26 print('Unstable: ' + str(tau.critical_eig().imag > 0))
27 print('Critical eigenvalue= ' + str(tau.critical_eig()))
28
29 plt = epssaver.saver(4, 4)
30
31 plt.plot(np.real(eigs), np.imag(eigs), marker='.', linestyle=' ')
32 plt.set_args(xlabel=r'$c_r$', ylabel=r'$c_i$')
33 plt.set_args(xlim=[0, 1], ylim=[-1, 0.05])
34
35 plt.saveeps(flowtype + 'R=' + str(R) + 'a=' + str(a) + 'M=' + str(M) + 'order='
36             + str(order))
```

A.11 Tau Method Time Comparison

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Thu Mar 16 11:45:26 2023
4
5  @author: oscar
6  """
7
8  from chebyshev_tau import ChebyshevTau
9  import time as time
10 import numpy as np
11 import matplotlib.pyplot as plt
12
13 # this program compares the times taken for the different order methods to run
14 # as such takes a long time to run
15
16 def get_time(order, M):
17     # get the time taken to run for specific M
18     tau = ChebyshevTau(order, M)
19
20     time_start = time.time()
21
22     tau.get_eigvals_eigvecs(1, 10000)
23
24     time_end = time.time()
25
26     return time_end - time_start
27
28 def compare_time(M):
29     time_D1 = get_time(1, M)
30     time_D2 = get_time(2, M)
31     time_D4 = get_time(4, M)
32
33     return [time_D1, time_D2, time_D4]
34
35 def main(maxM, step):
36     Mvals = np.linspace(0, maxM, int(maxM/step) + 1)
37
38     Mvals[0] = 10
39     print(Mvals)
40
41     N = len(Mvals)
42
43     times_D1 = np.zeros(N)
44     times_D2 = np.zeros(N)
45     times_D4 = np.zeros(N)
46
47     for i in range(0, N):
48         print("calculating times for M = " + str(Mvals[i]))
49         times_D1[i], times_D2[i], times_D4[i] = compare_time(int(Mvals[i]))
50
```

```
51     plt.plot(Mvals, times_D1,'-')
52     plt.plot(Mvals, times_D2,'-')
53     plt.plot(Mvals, times_D4,'-')
54     plt.show()
55
56 main(500, 50)
```

A.12 Eigenfunction Computation

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Fri Mar 17 17:00:42 2023
4
5  @author: oscar
6  """
7
8  import numpy as np
9  import chebyshev as cheby
10 import epssaver
11 from chebyshev_tau import ChebyshevTau
12
13 def plot_eigenfunction(crit_vec):
14     '''
15         Plots the eigenfunction from a given eigenvector
16
17         Parameters
18         -----
19         crit_vec : LIST OF FLOATS
20             Critical eigenvector
21
22         Returns
23         -----
24         None.
25
26         '''
27
28     accuracy = 201
29
30     vals = np.zeros(accuracy)
31     points = np.linspace(-1, 1, accuracy)
32     zero_val = 0
33     for i in range(0, len(crit_vec)):
34         Tk = cheby.ChebyshevK(i, accuracy)
35
36         zero_val += crit_vec[i] * Tk.evaluate(0)
37         tk_vals = Tk.get_vals()
38
39         vals = vals + crit_vec[i] * tk_vals
40
41     vals = vals / zero_val
42
43     real_vals = vals.real
44     imag_vals = vals.imag
45
46     plt = epssaver.saver(6, 3)
47     plt.set_args(xlabel=r'$z$')
48     plt.plot(points, real_vals, label=r'$\phi_r$')
49     plt.plot(points, imag_vals, label=r'$\phi_i$')
50
```

```
51     plt.legend()
52
53     plt.savefig('eigenfunction')
54
55 def main():
56     tau = ChebyshevTau(1, 200)
57
58     tau.get_eigvals_eigvecs(1, 10000)
59
60     crit_vec = tau.critical_vec()
61     plot_eigenfunction(crit_vec)
62
63 main()
```


A.13 Streamfunction Computation

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Fri Mar 17 17:00:42 2023
4
5  @author: oscar
6  """
7
8  import numpy as np
9  import chebyshev as cheby
10 from chebyshev_tau import ChebyshevTau
11 import epssaver
12
13 def plot_streamfunction(crit_vec, c, t):
14     '''
15         Plots the streamfunction from a given eigenvector
16
17         Parameters
18         -----
19         crit_vec : LIST OF FLOATS
20             Critical eigenvector
21
22         c : FLOAT
23             Critical eigenvalue
24
25         t : FLOAT
26             Time to display the streamfunction at
27
28         Returns
29         -----
30         None.
31
32     '''
33
34     z_accuracy = 200
35     x_accuracy = 400
36
37     z_vals = np.zeros(z_accuracy)
38     z = np.linspace(-1, 1, z_accuracy)
39     x = np.linspace(0, 2*np.pi, x_accuracy)
40     xz = np.zeros([z_accuracy, x_accuracy]) + 1j * np.zeros([z_accuracy,
41         ↪ x_accuracy])
42
43     zero_val = 0
44     for i in range(0, len(crit_vec)):
45         Tk = cheby.ChebyshevK(i, z_accuracy)
46
47         zero_val += crit_vec[i] * Tk.evaluate(0)
48         tk_vals = Tk.get_vals()
49
50         z_vals = z_vals + crit_vec[i] * tk_vals
```

```

50     z_vals = z_vals / zero_val
51
52     for j in range(0, x_accuracy):
53         xz[:, j] = z_vals * np.exp(1j * x[j])
54
55     t_xz = xz * np.exp(-1j * c * t)
56
57     levels = np.linspace(np.min(t_xz.real), np.max(t_xz.real), 21)
58
59     plt = epssaver.saver(8, 4)
60
61     plt.set_args(xlabel=r'$x$', ylabel=r'$z$')
62
63     plt.contourf(x, z, t_xz.real, levels=levels)
64     plt.colorbar(levels)
65
66     plt.saveeps('streamfunction')
67
68 def main(t):
69     tau = ChebyshevTau(1, 200)
70
71     tau.get_eigvals_eigvecs(1, 10000)
72
73     crit_vec = tau.critical_vec()
74     crit_eig = tau.critical_eig()
75
76     plot_streamfunction(crit_vec, crit_eig, t)
77
78 main(0)

```

A.14 Critical Reynolds Number Computation

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Apr 17 15:24:28 2023
4
5  @author: oscar
6  """
7
8  import numpy as np
9  from chebyshev_tau import ChebyshevTau
10 from epssaver import saver
11
12 # the following bijection code is modified from
13 # https://stackoverflow.com/questions/14392208/how-to-do-the-bisection-method-
14 # in-python
15 def samesign(a, b):
16     return a * b > 0
17
18 def bisect(a, lowR, highR):
19     '''
20     Find root of continuous function where f(low) and f(high) have opposite
21     signs
22
23     Parameters
24     -----
25     a : FLOAT
26         Wavenumber to calculate at
27     lowR : FLOAT
28         Lowest R value
29     highR : FLOAT
30         Highest R value
31
32     Returns
33     -----
34     margR : FLOAT
35         Marginal R value calculated
36
37     '''
38
39     tau = ChebyshevTau(4, 50)
40
41     tau.get_eigvals_eigvecs(a, lowR)
42     lowImag = tau.critical_eig().imag
43
44     tau.get_eigvals_eigvecs(a, highR)
45
46     for i in range(54):
47         midR = (lowR + highR) / 2.0
48
49         tau.get_eigvals_eigvecs(a, lowR)
50         lowImag = tau.critical_eig().imag
```

```

49
50     tau.get_eigvals_eigvecs(a, midR)
51     midImag = tau.critical_eig().imag
52
53     if samesign(lowImag, midImag):
54         lowR = midR
55     else:
56         highR = midR
57
58     margR = midR
59
60     return margR
61
62 def compute_marginal_r(low_a, high_a, points):
63     '''
64         Computes the marginal R value for a specified number of points between
65         ↪ specified values of a
66
67         Parameters
68         -----
69         low_a : FLOAT
70             Lowest a value
71         high_a : FLOAT
72             Highest a value
73         points : INT
74             Number of points to calculate at
75
76         Returns
77         -----
78         a_vals : LIST OF FLOATS
79             a values the marginal R values have been calculated at
80         r_vals : LIST OF FLOATS
81             the marginal R values calculated
82     '''
83     a_vals = np.linspace(low_a, high_a, points)
84     r_vals = np.zeros(points)
85
86     for i in range(0, points):
87         r_vals[i] = bisect(a_vals[i], 4000, 10000)
88
89     return a_vals, r_vals
90
91 def find_crit_r(a_min, a_max, recursion_count):
92     '''
93         Recursively searches further and further into minimum point to find
94         ↪ critical Reynolds number
95
96         Parameters
97         -----
98         a_min : FLOAT

```

```

98         Minimum wavenumber - the start of the search
99     a_max : FLOAT
100         Maximum wavenumber - the end of the search
101     recursion_count : INT
102         Maximum number of recursions to do
103
104     Returns
105     -----
106     Critical R value : FLOAT
107         Value of the critical Reynolds number found
108     Critical wavenumber : FLOAT
109         Value of the critical wavenumber found
110
111     '''
112     a_vals, r_vals = compute_marginal_r(a_min, a_max, 5)
113     min_index = np.argmin(r_vals)
114
115     # if our value is accurate enough, exit and return the values found
116     if abs(np.max(r_vals) - r_vals[min_index]) < 0.0001 or recursion_count >=
        ↳ 20:
117         return r_vals[min_index], a_vals[min_index]
118
119     if min_index == 0:
120         # the minimum value is at the minimum of our range of a
121         # search between that a value and the next
122
123         return find_crit_r(a_vals[min_index], a_vals[min_index + 1],
            ↳ recursion_count + 1)
124     elif min_index == len(a_vals) - 1:
125         # the minimum value is at the maximum of our range of a
126         # search between that a value and the previous a value
127
128         return find_crit_r(a_vals[min_index - 1], a_vals[min_index],
            ↳ recursion_count + 1)
129     else:
130         # the minimum value is not at either endpoint
131         # check between the a values on both sides of the minimum value found
132
133         left_r, left_a = find_crit_r(a_vals[min_index - 1], a_vals[min_index],
            ↳ recursion_count + 1)
134         right_r, right_a = find_crit_r(a_vals[min_index], a_vals[min_index +
            ↳ 1], recursion_count + 1)
135
136         if left_r < right_r:
137             return left_r, left_a
138         else:
139             return right_r, right_a
140
141 def main():
142     # THIS PROGRAM TAKES AGES TO RUN
143     # Comment out the section you don't want to calculate

```

```

144     # or reduce the M values used
145
146     # find the critical R and a values
147     crit_r, crit_a = find_crit_r(1.02, 1.03, 0)
148
149     print('Critical R=' + str(crit_r))
150     print('Critical alpha=' + str(crit_a))
151
152     # plot a curve of marginal stability
153     computed_a, computed_r = compute_marginal_r(0.843, 1.0972, 150)
154
155     plt = saver(4, 4)
156     plt.plot(computed_a, computed_r, linestyle='--')
157     plt.plot(crit_a, crit_r, color='#1f77b4', marker='.')
158     plt.set_args(xlabel=r'$\alpha$', ylabel=r'$R$')
159
160     plt.saveeps('marginal_stability_curve')
161
162     main()

```

Bibliography

- Antar, B. N. (1976). The eigenvalue spectrum of the orr-sommerfeld problem. Technical report.
- Bukhari, A.-F. K. A. (1997). *A Spectral Eigenvalue Method for Multilayered Continua*. University of Glasgow (United Kingdom).
- Canuto, C., Hussaini, M. Y., Quarteroni, A., and Zang, T. A. (2007). *Spectral methods: evolution to complex geometries and applications to fluid dynamics*. Springer Science & Business Media.
- Dongarra, J., Straughan, B., and Walker, D. (1996a). Chebyshev tau-qz algorithm methods for calculating spectra of hydrodynamic stability problems. *Applied Numerical Mathematics*, 22(4):399–434.
- Dongarra, J., Straughan, B., and Walker, D. (1996b). Chebyshev tau-qz algorithm methods for calculating spectra of hydrodynamic stability problems. *Applied Numerical Mathematics*, 22(4):399–434.
- Drazin, P. G. and Reid, W. H. (2004). *Hydrodynamic stability*. Cambridge university press.
- Fox, L. and Parker, I. B. (1968). Chebyshev polynomials in numerical analysis. Technical report.
- Herbert, T. (1977). *Die neutrale Fläche der ebenen Poiseuille-Strömung*. PhD thesis.
- Joseph, D. D. (1968). Eigenvalue bounds for the orr-sommerfeld equation. *Journal of Fluid Mechanics*, 33(3):617–621.
- Joseph, D. D. (1969). Eigenvalue bounds for the orr—sommerfeld equation. part 2. *Journal of Fluid Mechanics*, 36(4):721–734.
- Lanczos, C. (1988). *Applied analysis*. Courier Corporation.
- Mack, L. M. (1976). A numerical study of the temporal eigenvalue spectrum of the blasius boundary layer. *Journal of Fluid Mechanics*, 73(3):497–520.
- Moler, C. B. and Stewart, G. W. (1973). An algorithm for generalized matrix eigenvalue problems. *SIAM Journal on Numerical Analysis*, 10(2):241–256.
- Orr, M. W. (1907). The stability or instability of the steady motions of a perfect liquid and of a viscous liquid. Dublin Proc. 27, 9-68. [Nature 75, 119.] (1907).
- Orszag, S. A. (1971). Accurate solution of the orr—sommerfeld stability equation. *Journal of Fluid Mechanics*, 50(4):689–703.
- Pai, S. (1954). On a generalization of syngé’s criterion for sufficient stability of plane parallel flows. *Quarterly of Applied Mathematics*, 12(2):203–206.
- Rayleigh, L. (1880). On the stability, or instability, of certain fluid motions. *Proc. London Math. Soc.*, 9:57–70.

- Sommerfeld, A. (1909). *Ein beitrag zur hydrodynamischen erklärung der turbulenten flüssigkeitsbewegungen*.
- Squire, H. B. (1933). On the stability for three-dimensional disturbances of viscous fluid flow between parallel walls. *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character*, 142(847):621–628.
- Synge, J. L. (1938). Hydrodynamical stability. *Semicentenn. Publ. Amer. Math. Soc*, 2:227–269.
- Thomas, L. (1953). The stability of plane poiseuille flow. *Physical Review*, 91(4):780.
- Trefethen, L. N., Trefethen, A. E., Reddy, S. C., and Driscoll, T. A. (1993). Hydrodynamic stability without eigenvalues. *Science*, 261(5121):578–584.